



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Specifica Tecnica

Stato	In lavorazione
Versione	0.9.0
Autori	Aldo Bettega Davide Lorenzon Ana Maria Draghici Filippo Guerra Felician Mario Necsulescu Davide Testolin
Verificatori	Davide Lorenzon Filippo Guerra Felician Mario Necsulescu
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin BlueWind srl

Vers.	Data	Autore	Verificatore	Descrizione
0.9.0	2026-05-06	Ana Maria Draghici		Scomposizione sezioni a partire dalla <u>Sezione 5.2</u> : Import, ValutazioneRequisiti, Dashboard,Export e Frontend
0.8.0	2026-05-03	Ana Maria Draghici		Scomposizione sezioni a partire dalla <u>Sezione 5.2</u> : Device, Asset e Session
0.7.3	2026-04-22	Davide Testolin	Ana Maria Draghici	Migliorata sezione <u>Sezione 3.6</u>
0.7.2	2026-04-20	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern comportamentali e architeturali <u>Sezione 4.5</u> , <u>Sezione 4.2</u>
0.7.1	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern: creazionali e strutturali <u>Sezione 4.3</u> , <u>Sezione 4.4</u>
0.7.0	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei principi di design <u>Sezione 4.1</u>
0.6.4	2026-04-19	Davide Lorenzon	-	Revisione architeturale della sezione relativa alla modifica degli asset <u>Sezione 5.3</u>
0.6.3	2026-04-19	Davide Lorenzon	-	Bozza delle classi relative alla generazione del report di conformità
0.6.2	2026-04-19	Davide Lorenzon	-	Bozza delle classi relative a import ed export tramite file di dispositivo e modello

Vers.	Data	Autore	Verificatore	Descrizione
0.6.1	2026-04-17	Filippo Guerra	-	Bozza della classe valutazione <u>Sezione 5.6</u>
0.6.0	2026-04-17	Filippo Guerra	Davide Lorenzon	Stesura vista_dati <u>Sezione 5.3</u>
0.5.0	2026-04-16	Ana Maria Draghici	Davide Lorenzon	Stesura vista_dati <u>Sezione 3.6</u>
0.4.0	2026-04-16	Aldo Bettega	Davide Lorenzon	Stesura <u>Sezione 5.1</u> e <u>Sezione 5.2</u>
0.3.0	2026-04-15	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.4</u>
0.2.0	2026-04-10	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.1</u> e <u>Sezione 3.2</u>
0.1.2	2026-04-09	Davide Lorenzon	Filippo Guerra	Bozza iniziale della <u>Sezione 3.3</u> Architettura di deployment.
0.1.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Bozza iniziale della <u>Sezione 2</u> Tecnologie.
0.1.0	2026-04-08	Aldo Bettega	Felician Mario Necsulescu	Stesura introduzione
0.0.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Creazione del documento

Indice

1) Introduzione	1
1.1) Scopo del documento	1
1.2) Scopo del prodotto	1
1.3) Glossario	1
1.4) Riferimenti	2
1.4.1) Riferimenti normativi	2
1.4.2) Riferimenti informativi	2
2) Tecnologie	3
2.1) Backend	3
2.2) Frontend	5
2.3) Persistenza dei dati	6
3) Architettura di Sistema	7
3.1) Premessa: approccio di lavoro usato	7
3.2) Architettura Logica	7
3.2.1) Stile architetturale	7
3.2.2) Motivazioni della scelta architetturale	8
3.2.3) Limiti dell'architettura	8
3.2.4) Diagramma dei package	9
3.3) Architettura di Deployment	10
3.3.1) Motivazioni della scelta	10
3.3.2) Realizzazione Pratica e Topologia	10
3.4) Diagrammi di sequenza	11
3.4.1) UC05 - Importazione dispositivo	11
3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato	12
3.4.3) UC30 - Esportazione report di conformità	13
3.5) Diagrammi di attività	14
3.5.1) Navigazione degli alberi	14
3.6) Schema dati	15
3.6.1) Scelte Architetture e Formalismo	15
3.6.2) Schema Dati Device	16
3.6.3) Schema ComplianceStandard	18
3.6.4) Gestione degli Esiti e Storicità	20
4) Design Patterns	21
4.1) Principi di Design	21
4.1.1) Dependency Injection	21
4.2) Pattern Architetture	21
4.2.1) Repository	21
4.2.2) Unit of Work	22
4.3) Design Patterns Creazionali	22
4.3.1) Factory	22

4.4) Design Patterns strutturali	23
4.4.1) Adapter	23
4.4.2) Facade	23
4.5) Design Patterns Comportamentali	24
4.5.1) Strategy	24
4.5.2) Template Method	24
5) Diagrammi delle classi	25
5.1) Diagramma di dominio	25
5.2) Device	27
5.2.1) WriteDeviceModule	27
5.2.2) CreateDevice	27
5.2.2.1) WriteDeviceController	28
5.2.2.2) CreateDeviceUseCase	28
5.2.2.3) CreateDeviceCommand	29
5.2.2.4) CreateDeviceService	29
5.2.2.5) Device	30
5.2.2.6) RegisterDevicePort	31
5.2.2.7) MongoDeviceAdapter	31
5.2.3) DeleteDevice	32
5.2.3.1) DeleteDeviceUseCase	32
5.2.3.2) DeleteDeviceCommand	33
5.2.3.3) DeleteDeviceService	33
5.2.3.4) DeleteDevicePort	34
5.2.4) UpdateDevice	35
5.2.4.1) UpdateDeviceUseCase	35
5.2.4.2) SaveDeviceCommand	36
5.2.4.3) UpdateDeviceService	36
5.2.4.4) SaveDevicePort	37
5.2.5) ReadDeviceModule	38
5.2.6) GetDeviceDetail	39
5.2.6.1) QueryDeviceController	39
5.2.7) GetDeviceDetailUseCase	40
5.2.7.1) GetDeviceDetailService	40
5.2.7.2) GetDeviceDetailCommand	41
5.2.7.3) FindDevicePort	41
5.2.8) GetDeviceList	42
5.2.8.1) GetDeviceListUseCase	42
5.2.8.2) GetDeviceListService	43
5.2.8.3) DeviceSummary	43
5.2.8.4) FindAllDevicesPort	44
5.3) Asset	44

5.3.1)	CreateAsset	44
5.3.1.1)	WriteAssetController	45
5.3.1.2)	CreateAssetUseCase	45
5.3.1.3)	CreateAssetCommand	46
5.3.1.4)	CreateAssetService	46
5.3.1.5)	Asset	47
5.3.1.6)	AssetProprieties	48
5.3.1.7)	InMemoryEvaluationSessionCache	49
5.3.1.8)	SaveSessionPort	50
5.3.1.9)	GetSessionPort	50
5.3.2)	UpdateAsset	51
5.3.2.1)	UpdateAssetUseCase	52
5.3.2.2)	UpdateAssetCommand	52
5.3.2.3)	UpdateAssetService	53
5.3.3)	DeleteAsset	54
5.3.3.1)	DeleteAssetCommand	55
5.3.3.2)	DeleteAssetUseCase	55
5.3.3.3)	DeleteAssetService	56
5.3.4)	GetAssetDetail	57
5.3.4.1)	QueryDashboardController	57
5.3.4.2)	GetAssetDetailCommand	58
5.3.4.3)	GetAssetDetailUseCase	59
5.3.4.4)	GetAssetDetailService	59
5.3.4.5)	AssetDetail	60
5.3.4.6)	RequirementEval	61
5.4)	Import	61
5.4.1)	ImportDevice	61
5.4.1.1)	UploadFileController	62
5.4.1.2)	ImportDeviceController	62
5.4.1.3)	UploadFileController	63
5.4.1.4)	ImportDeviceUseCase	64
5.4.1.5)	ImportDeviceCommand	64
5.4.1.6)	AllowedDeviceFileExtension	65
5.4.1.7)	ImportDeviceService	65
5.4.1.8)	FileDeviceImporterPort	66
5.4.1.9)	FileDeviceImporterFactoryPort	66
5.4.1.10)	FileDeviceImporter	67
5.4.1.11)	XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter	68
5.4.1.12)	ConcreteFileDeviceImporterFactory	68
5.4.2)	ImportStandard	69

5.4.2.1)	ImportStandardController	70
5.4.2.2)	ImportStandardUseCase	71
5.4.2.3)	ImportStandardCommand	71
5.4.2.4)	AllowedStandardFileExtension	72
5.4.2.5)	ImportStandardService	72
5.4.2.6)	FileStandardImporterPort	73
5.4.2.7)	StandardSavePort	74
5.4.2.8)	FileStandardImporterFactoryPort	74
5.4.2.9)	FileStandardImporter	75
5.4.2.10)	JSONFileStandardImporter e XMLFileStandardImporter .	76
5.4.2.11)	ConcreteFileStandardImporterFactory	77
5.5)	Dashboard	78
5.5.1)	GetDeviceDashboard	78
5.5.1.1)	GetDeviceDashboardCommand	78
5.5.1.2)	GetDeviceDashboardUseCase	79
5.5.1.3)	GetDeviceDashboardService	79
5.5.1.4)	DashboardCommand	80
5.5.1.5)	AssetSummaryCommand	81
5.6)	Session	82
5.6.1)	OpenEvaluationSession	82
5.6.1.1)	EvaluationSessionController	83
5.6.1.2)	OpenEvaluationSessionCommand	83
5.6.1.3)	OpenEvaluationSessionUseCase	84
5.6.1.4)	OpenEvaluationSessionService	84
5.6.1.5)	SessionCoordinator	85
5.6.1.6)	CreateSessionPort	85
5.6.1.7)	FindStandardPort	86
5.6.1.8)	MongoStandardAdapter	86
5.6.2)	SaveEvaluationSession	87
5.6.2.1)	SaveEvaluationSessionUseCase	88
5.6.2.2)	SaveEvaluationSessionService	88
5.6.2.3)	SaveEvaluationSessionCommand	89
5.6.2.4)	SaveSessionPort	89
5.6.3)	CloseEvaluationSession	90
5.6.3.1)	CloseEvaluationSessionUseCase	91
5.6.3.2)	CloseEvaluationSessionService	91
5.6.3.3)	CloseEvaluationSessionCommand	92
5.6.3.4)	DeleteSessionPort	92
5.6.4)	CommitEvaluationSession	93
5.6.4.1)	CommitEvaluationSessionService	94
5.6.4.2)	CommitEvaluationSessionUseCase	94

5.6.4.3)	CommitEvaluationSessionCommand	95
5.6.4.4)	CommitCloseSession	96
5.6.4.5)	CommitCloseEvaluationSessionService	97
5.7)	Valutazione	98
5.7.1)	EvaluateDecisionNode	98
5.7.1.1)	EvaluationDecisionNodeController	99
5.7.1.2)	EvaluateDecisionNodeUseCase	99
5.7.1.3)	EvaluationNodeCommand	100
5.7.1.4)	EvaluateDecisionNodeService	100
5.7.2)	GetRequirement	102
5.7.2.1)	QueryEvaluationRequirementsController	103
5.7.2.2)	GetRequirementUseCase	103
5.7.2.3)	GetRequirementCommand	104
5.7.2.4)	GetRequirementService	104
5.7.2.5)	RequirementResponse	105
5.7.2.6)	DecisionTreeResponse	105
5.7.2.7)	NodeResponse	106
5.7.2.8)	DependencyResponse	106
5.7.3)	InsertJustification	107
5.7.3.1)	EvaluationJustificationController	108
5.7.3.2)	InsertJustificationUseCase	108
5.7.3.3)	InsertJustificationCommand	109
5.7.3.4)	EvaluationJustificationService	109
5.8)	Export	110
5.8.1)	ExportDevice	110
5.8.1.1)	ExportDeviceController	110
5.8.1.2)	DownloadFileController	111
5.8.1.3)	ExportDeviceUseCase	111
5.8.1.4)	ExportDeviceCommand	112
5.8.1.5)	AllowedDeviceFileExtension	112
5.8.1.6)	ExportDeviceService	112
5.8.1.7)	DeviceFileCommand, AssetFileCommand e AssessmentFileCommand	113
5.8.1.8)	ConcreteFileDeviceExporterFactory	113
5.8.1.9)	FileDeviceExporter	114
5.8.1.10)	JSONFileDeviceExporter e XMLFileDeviceExporter	114
5.8.1.11)	FileDeviceExporterFactoryPort	115
5.8.1.12)	FileDeviceExporterPort	116
5.8.2)	ExportReport	117
5.8.2.1)	ExportReportController	117
5.8.2.2)	ExportReportUseCase	118

5.8.2.3)	ExportReportCommand	118
5.8.2.4)	ExportReportService	119
5.8.2.5)	ReportGeneratorPort	119
5.8.2.6)	PdfReportGeneratorAdapter	120
5.8.3)	ExportStandard	121
5.8.3.1)	ExportStandardController	121
5.8.3.2)	ExportStandardUseCase	122
5.8.3.3)	ExportStandardCommand	122
5.8.3.4)	ExportStandardService	123
5.8.3.5)	StandardFileCommand	123
5.8.3.6)	AssetFileCommand	124
5.8.3.7)	AssessmentFileCommand	125
5.8.3.8)	FileStandardExporterFactoryPort	125
5.8.3.9)	FileStandardExporterPort	126
5.8.3.10)	MongoStandardAdapter	126
5.8.3.11)	ConcreteFileStandardExporterFactory	127
5.8.3.12)	JSONFileStandardExporter e XMLFileStandardExporter	127
5.9)	Architettura Frontend: Modulo Decision Tree	128
5.9.1)	Architettura Interattiva della Sidebar	129
5.9.1.1)	DecisionTreeStore	130
5.9.1.2)	TreeSidebar	131
5.9.1.3)	NodeDTO	131
5.9.2)	Architettura Visiva del Canvas (Tree Canvas)	132
5.9.2.1)	LayoutResult e i DTO Geometrici	133
5.9.2.2)	LayoutEngine e D3LayoutEngine	133
5.9.2.3)	TreeCanvas	134
5.9.2.4)	UiDecisionNode e UiLeafNode	135
5.9.3)	Core di Valutazione (Domain Logic Client-side)	136
5.9.3.1)	Interfaccia Node	136
5.9.3.2)	DecisionNode	137
5.9.3.3)	LeafNode	137
5.9.3.4)	TreeStructure	138
5.9.3.5)	EvaluationEngine	138
5.9.4)	DecisionTreeStore	139
5.9.5)	EvaluationApiClient	140
6)	Tracciamento	141
7)	Qualità architetturale	142
8)	Gestione Errori e Logging	143
9)	Sicurezza	144
10)	Performance e Scalabilità	145

Lista delle tabelle

Tabella 1	Backend	3
Tabella 2	Frontend	5
Tabella 3	Schema dati: Root — Device	17
Tabella 4	Schema dati: Sub-documento — assets[N]	17
Tabella 5	Schema dati: Sub-documento — evaluations[N]	17
Tabella 6	Schema dati: Root — ComplianceStandard	19
Tabella 7	Schema dati: Sub-documento — requirements[N]	19
Tabella 8	Schema dati: Sub-documento — DecisionTree	19
Tabella 9	Schema dati: Sub-documento — DecisionNode	20
Tabella 10	Schema dati: Sub-documento — LeafNode	20

Lista delle immagini

Figura 1	Schema logico: Documento Device	16
Figura 2	Schema logico: ComplianceStandard	18
Figura 3	Modulo di scrittura Dispositivi	27
Figura 4	Caso d'uso CreateDevice	27
Figura 5	WriteDeviceController	28
Figura 6	CreateDeviceUseCase	28
Figura 7	CreateDeviceCommand	29
Figura 8	CreateDeviceService	29
Figura 9	Device	30
Figura 10	RegisterDevicePort	31
Figura 11	MongoDeviceAdapter	31
Figura 12	Caso d'uso DeleteDevice	32
Figura 13	DeleteDeviceUseCase	32
Figura 14	DeleteDeviceCommand	33
Figura 15	DeleteDeviceService	33
Figura 16	DeleteDevicePort	34
Figura 17	Caso d'uso UpdateDevice	35
Figura 18	UpdateDeviceUseCase	35
Figura 19	UpdateDeviceCommand	36
Figura 20	UpdateDeviceService	36
Figura 21	SaveDevicePort	37
Figura 22	Modulo di lettura Dispositivi	38
Figura 23	GetDeviceDetail	39
Figura 24	QueryDeviceController	39
Figura 25	GetDeviceDetailUseCase	40
Figura 26	GetDeviceDetailService	40
Figura 27	GetDeviceDetailCommand	41
Figura 28	FindDevicePort	41
Figura 29	Caso d'uso GetDeviceList	42
Figura 30	GetDeviceListUseCase	42
Figura 31	GetDeviceListService	43
Figura 32	DeviceSummary	43
Figura 33	FindAllDevicesPort	44
Figura 34	Caso d'uso CreateAsset	44
Figura 35	WriteAssetController	45
Figura 36	CreateAssetUseCase	45
Figura 37	CreateAssetCommand	46
Figura 38	CreateAssetService	46
Figura 39	Asset	47
Figura 40	AssetAnagraphic	48

Figura 41	AssetProprieties	48
Figura 42	InMemoryEvaluationSessionCache	49
Figura 43	SaveSessionPort	50
Figura 44	GetSessionPort	50
Figura 45	Caso d'uso UpdateAsset	51
Figura 46	UpdateAssetUseCase	52
Figura 47	UpdateAssetCommand	52
Figura 48	UpdateAssetService	53
Figura 49	Caso d'uso DeleteAsset	54
Figura 50	DeleteAssetCommand	55
Figura 51	DeleteAssetUseCase	55
Figura 52	DeleteAssetService	56
Figura 53	Caso d'uso GetAssetDetail	57
Figura 54	QueryDashboardController	57
Figura 55	GetAssetDetailCommand	58
Figura 56	GetAssetDetailUseCase	59
Figura 57	GetAssetDetailService	59
Figura 58	AssetDetail	60
Figura 59	RequirementEval	61
Figura 60	Caso d'uso ImportDevice	61
Figura 61	UploadFileController	62
Figura 62	ImportDeviceController	62
Figura 63	UploadFileController	63
Figura 64	ImportDeviceUseCase	64
Figura 65	ImportDeviceCommand	64
Figura 66	AllowedDeviceFileExtension	65
Figura 67	ImportDeviceService	65
Figura 68	FileDeviceImporterPort	66
Figura 69	FileDeviceImporterFactoryPort	66
Figura 70	FileDeviceImporter	67
Figura 71	XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter	68
Figura 72	ConcreteFileDeviceImporterFactory	68
Figura 73	ImportStandard	69
Figura 74	ImportStandardController	70
Figura 75	ImportStandardUseCase	71
Figura 76	ImportStandardCommand	71
Figura 77	AllowedStandardFileExtension	72
Figura 78	ImportStandardService	72
Figura 79	FileStandardImporterPort	73
Figura 80	StandardSavePort	74

Figura 81	FileStandardImporterFactoryPort	74
Figura 82	FileStandardImporter	75
Figura 83	JSONFileStandardImporter e XMLFileStandardImporter	76
Figura 84	ConcreteFileStandardImporterFactory	77
Figura 85	Caso d'uso GetDeviceDashboard	78
Figura 86	GetDeviceDashboardCommand	78
Figura 87	GetDeviceDashboardUseCase	79
Figura 88	GetDeviceDashboardService	79
Figura 89	DashboardCommand	80
Figura 90	AssetSummaryCommand	81
Figura 91	Caso d'uso OpenEvaluationSession	82
Figura 92	EvaluationSessionController	83
Figura 93	OpenEvaluationSessionCommand	83
Figura 94	OpenEvaluationSessionUseCase	84
Figura 95	OpenEvaluationSessionService	84
Figura 96	SaveAssetService	85
Figura 97	CreateSessionPort	85
Figura 98	FindStandardPort	86
Figura 99	MongoStandardAdapter	86
Figura 100	Caso d'uso SaveEvaluationSession	87
Figura 101	SaveEvaluationSessionUseCase	88
Figura 102	SaveEvaluationSessionService	88
Figura 103	SaveEvaluationSessionCommand	89
Figura 104	SaveSessionPort	89
Figura 105	Caso d'uso CloseEvaluationSession	90
Figura 106	CloseEvaluationSessionUseCase	91
Figura 107	CloseEvaluationSessionService	91
Figura 108	CloseEvaluationSessionCommand	92
Figura 109	DeleteSessionPort	92
Figura 110	Caso d'uso CommitEvaluationSession	93
Figura 111	CommitEvaluationSessionService	94
Figura 112	CommitEvaluationSessionUseCase	94
Figura 113	CommitEvaluationSessionCommand	95
Figura 114	Caso d'uso CommitCloseSession	96
Figura 115	CommitCloseEvaluationSessionService	97
Figura 116	Caso d'uso EvaluateDecisionNode	98
Figura 117	EvaluationDecisionNodeController	99
Figura 118	EvaluateDecisionNodeUseCase	99
Figura 119	EvaluateNodeCommand	100
Figura 120	EvaluateDecisionNodeService	100
Figura 121	Caso d'uso GetRequirement	102

Figura 122	QueryEvaluationRequirementsController	103
Figura 123	GetRequirementUseCase	103
Figura 124	GetRequirementCommand	104
Figura 125	GetRequirementService	104
Figura 126	RequirementResponse	105
Figura 127	DecisionTreeResponse	105
Figura 128	NodeResponse	106
Figura 129	DependencyResponse	106
Figura 130	Caso d'uso InsertJustification	107
Figura 131	EvaluationJustificationController	108
Figura 132	InsertJustificationUseCase	108
Figura 133	InsertJustificationCommand	109
Figura 134	EvaluationJustificationService	109
Figura 135	Caso d'uso ExportDevice	110
Figura 136	ExportDeviceController	110
Figura 137	DownloadFileController	111
Figura 138	ExportDeviceUseCase	111
Figura 139	ExportDeviceCommand	112
Figura 140	AllowedDeviceFileExtension	112
Figura 141	ExportDeviceService	112
Figura 142	DDeviceFileCommand, AssetFileCommand e AssessmentFileCommand	113
Figura 143	ConcreteFileDeviceExporterFactory	113
Figura 144	FileDeviceExporter	114
Figura 145	JSONFileDeviceExporter e XMLFileDeviceExporter	114
Figura 146	FileDeviceExporterFactoryPort	115
Figura 147	FileDeviceExporterPort	116
Figura 148	Caso d'uso ExportReport	117
Figura 149	ExportReportController	117
Figura 150	ExportReportUseCase	118
Figura 151	ExportReportCommand	118
Figura 152	ExportReportService	119
Figura 153	ReportGeneratorPort	119
Figura 154	PdfReportGeneratorAdapter	120
Figura 155	Caso d'uso ExportStandard	121
Figura 156	ExportStandardController	121
Figura 157	ExportStandardUseCase	122
Figura 158	ExportStandardCommand	122
Figura 159	ExportStandardService	123
Figura 160	StandardFileCommand	123
Figura 161	AssetFileCommand	124

Figura 162	AssessmentFileCommand	125
Figura 163	FileStandardExporterFactoryPort	125
Figura 164	FileStandardExporterPort	126
Figura 165	MongoStandardAdapter	126
Figura 166	ConcreteFileStandardExporterFactory	127
Figura 167	JSON e XML FileStandardExporter	127
Figura 168	Architettura del componente TreeSidebar e del suo Store	129
Figura 169	DecisionTreeStore	130
Figura 170	TreeSidebar	131
Figura 171	NodeDTO	131
Figura 172	Architettura del componente TreeCanvas e del Layout Engine	132
Figura 173	LayoutResult, NodeDTO ed EdgeDTO	133
Figura 174	D3LayoutEngine	133
Figura 175	TreeCanvas	134
Figura 176	UiDecisionNode e UiLeafNode	135
Figura 177	Diagramma delle classi del Motore di Valutazione	136
Figura 178	Interfaccia Node	136
Figura 179	DecisionNode	137
Figura 180	LeafNode	137
Figura 181	TreeStructure	138
Figura 182	EvaluationEngine	138
Figura 183	Dettaglio del DecisionTreeStore (Pinia)	139
Figura 184	Dettaglio dell'EvaluationApiClient	140

1) Introduzione

1.1) Scopo del documento

Il presente documento intende fornire una descrizione approfondita dell'architettura del prodotto software, delineando con chiarezza le componenti che lo costituiscono, le loro responsabilità e le interazioni reciproche all'interno del sistema. La Specifica Tecnica funge da riferimento principale per la fase di progettazione e codifica, garantendo coerenza con i requisiti analizzati e consolidando la maturità architetturale del prodotto.

Nello specifico, questo documento si propone di:

- **Definire l'architettura logica e i design pattern:** descrivere l'organizzazione dei moduli e le logiche di interazione, evidenziando come i pattern adottati garantiscano un codice modulare e testabile.
- **Motivare lo stack tecnologico:** giustificare la scelta delle tecnologie utilizzate in funzione della scalabilità del sistema e della qualità complessiva del prodotto finale.
- **Delineare l'architettura di deployment:** illustrare la distribuzione delle componenti negli ambienti di esecuzione e le strategie di rilascio adottate.
- **Fornire un framework per l'evoluzione del software :** stabilire le linee guida necessarie per facilitare la comprensione del sistema, agevolando futuri interventi di estensione e manutenzione.

1.2) Scopo del prodotto

Il prodotto si prefigge di automatizzare e digitalizzare il processo di verifica della conformità alla normativa EN 18031, sostituendo le attuali procedure manuali, spesso onerose e soggette a errore umano, con una soluzione software interattiva.

Il sistema è progettato per guidare l'utente attraverso l'esecuzione di alberi decisionali (Decision Tree) complessi, permettendo la valutazione sistematica dei requisiti di sicurezza per dispositivi IoT e la generazione di esiti certi (Pass, Fail, N.A.). L'integrazione di funzionalità per l'importazione di dati strutturati e una dashboard di monitoraggio in tempo reale mira a garantire la massima tracciabilità del processo, ottimizzando i tempi operativi della proponente e assicurando un elevato standard di affidabilità e manutenibilità dei risultati ottenuti.

1.3) Glossario

Per garantire precisione terminologica senza appesantire la lettura, in questo documento i termini tecnici presenti nel Glossario sono segnalati con un pedice "G", ad esempio:

termine_G: indica che il termine è definito nel Glossario e può essere consultato per chiarimenti.

Questo metodo consente di mantenere il testo chiaro e tecnicamente corretto, permettendo al lettore di riferirsi al Glossario solo quando necessario, senza interrompere il flusso della lettura.

1.4) Riferimenti

1.4.1) Riferimenti normativi

- [Capitolato d'appalto C1 - Automated EN18031 Compliance Verification di BlueWind Srl](#)
Ultima consultazione: 8 aprile 2026;
- [Regolamento del progetto](#)
Ultima consultazione: 8 aprile 2026;
- [Standard ISO/IEC/IEEE 12207:2017](#)
Ultima consultazione: 10 gennaio 2026;
- [Standard ISO/IEC 9126](#)
Ultima consultazione: 10 gennaio 2026;

1.4.2) Riferimenti informativi

- [Glossario del gruppo](#)
Ultima consultazione: 8 aprile 2026;
- [Software Engineering, Ian Sommerville](#)
Ultima consultazione: 10 gennaio 2026;
- [Documentazione del gruppo GroupRubberDuck](#)
Ultima consultazione: 8 aprile 2026;

2) Tecnologie

In questa sezione vengono descritte le tecnologie utilizzate nello sviluppo del progetto **Automated EN18031 Compliance Verification**.

Per ogni tecnologia è riportata la versione di riferimento e il relativo ruolo all'interno del progetto

2.1) Backend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
Python	3.12.X	Linguaggio interpretato scelto per la rapidità di sviluppo, l'alta leggibilità e il vasto ecosistema.
Framework Principale		
Flask	3.1.3	Micro-framework web utilizzato come infrastruttura principale di backend. Consente una gestione flessibile e leggera del routing per esporre le interfacce API. Buona documentazione e possibilità di confronto tecnico con la proponente.
Dipendenze principali		
waitress	3.0.2	Server WSGI leggero. Gestisce la concorrenza delle richieste in modo affidabile.
fpdf		Libreria per la generazione dinamica di documenti e reportistica in formato PDF direttamente lato server.
pymongo		Driver ufficiale per l'integrazione con MongoDB.
pydantic		Utilizzato per la validazione rigida e type-safe dei dati in ingresso e delle variabili d'ambiente di sistema.
python-dotenv		Gestione semplificata delle configurazioni e delle variabili d'ambiente.
Jinja2		Template engine HTML con un'ottima integrazione con Flask.

Nome	Versione	Descrizione
werkzeug		Server di sviluppo locale.
watchdog		Permette l'hot reloading, passando le modifiche al server di sviluppo senza necessità di riavvio e preservando lo stato dell'applicazione.
Dynamic Testing		
pytest		Framework di testing adottato per la sua sintassi concisa. Utilizzato per automatizzare i «Sanity Tests» e validare l'integrazione di sistema alla radice del progetto.
Static Testing		
ruff		Lint e formater estremamente veloce (scritto in Rust). Impone e garantisce standard di codice puliti e uniformi senza rallentare lo sviluppo.
mypy		Analizzatore statico basato sul Type Hinting. Previene i bug e gli errori di tipo a tempo di sviluppo prima ancora dell'esecuzione
Sviluppo		
poetry		Gestore moderno delle dipendenze e degli ambienti virtuali (.venv). Garantisce build riproducibili tra i vari sviluppatori tramite il file di blocco (poetry.lock).

Tabella 1: Backend

2.2) Frontend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
JavaScript		Linguaggio client-side universale.
Framework Principale		
Vue.js		Framework progressivo con una curva di apprendimento morbida e un'ottima implementazione della reattività. L'adozione della Composition API permette di scrivere logica modulare e riutilizzabile.
Dipendenze principali		
pinia		Gestore dello stato ufficiale e leggero per Vue.
axios		Client HTTP flessibile. Preferito alla Fetch API nativa per l'ergonomia nella gestione automatica del JSON, la gestione degli errori e l'uso degli interceptor per le chiamate verso le API di Flask.
tailwindcss		Framework CSS utility-first per un'agile stilizzazione dell'interfaccia.
Dynamic Testing		
vitest		Framework di unit testing veloce e nativo per Vite. Utilizzato per validare la logica isolata dei componenti.
vue-test-utils		Libreria ufficiale affiancata a Vitest per montare e simulare le interazioni dell'utente sulle singole isole Vue durante i test.
Static Testing		
eslint		Linter per garantire standard di pulizia e coerenza nel codice JavaScript dei vari componenti Vue.
Sviluppo		

Nome	Versione	Descrizione
vite		Tool di build per minificare e raggruppare i file JS/CSS corrispondenti ai componenti Vue, ottimizzandoli per la produzione.

Tabella 2: Frontend

2.3) Persistenza dei dati

MongoDB è un database NoSQL orientato ai documenti. A differenza dei classici database relazionali (che usano tabelle e colonne rigide), archivia le informazioni in documenti flessibili molto simili al formato JSON.

La versione utilizzata è la 7.XX

Permette di aggiungere o rimuovere campi dai dati senza riscrivere l'intera organizzazione delle informazioni del database.

Nel progetto **Automated EN18031 Compliance Verification** viene utilizzato come sistema per la persistenza dei dati, in particolare per la rappresentazione dei dispositivi sottoposti alle valutazioni e per la rappresentazione del modello di standard tramite configurazioni esterne.

Il suo utilizzo permette di rappresentare facilmente strutture dati annidate, tramite una sintassi JSON.

Evita la gestione di join complessi tipici di un database relazionale, offre una maggiore sicurezza rispetto alla gestione manuale di file di rappresentazione interni.

È facilmente integrabile nel sistema backend

3) Architettura di Sistema

3.1) Premessa: approccio di lavoro usato

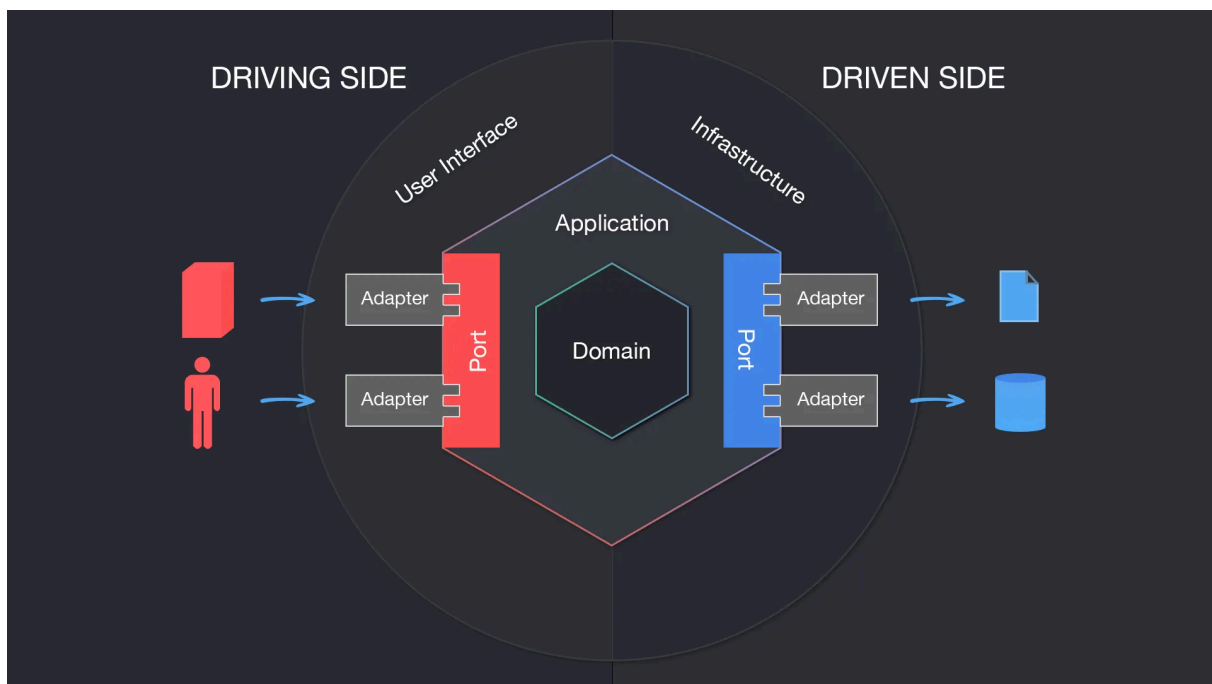
L'obiettivo della fase di progettazione è definire un'architettura software in grado di soddisfare pienamente i requisiti di sistema, operando secondo criteri di sostenibilità e ottimizzazione delle risorse (design-to-cost).

La metodologia adottata per la definizione dell'architettura è di tipo Top-Down. Il sistema viene inizialmente concepito nella sua interezza come un'unica entità astratta e, attraverso un processo di esplorazione funzionale, viene progressivamente decomposto in sotto-sistemi logici più piccoli e gestibili.

3.2) Architettura Logica

3.2.1) Stile architetturale

L'architettura adottata per realizzare il prodotto è l'architettura esagonale. Questo tipo di architettura ha come obiettivo primario isolare la logica di business dal resto. Questo approccio garantisce un'elevata testabilità del sistema e rende il nucleo del software completamente agnostico rispetto ai dettagli implementativi (framework, database o interfacce). Il sistema è strutturato in livelli concentrici, organizzati come segue:



- **Domain (Core):** Rappresenta il nucleo centrale dell'applicazione. Contiene la logica di business pura ed è rigorosamente privo di dipendenze esterne.
- **Ports:** Costituiscono il punto di connessione tra il nucleo e il mondo esterno, permettendo una comunicazione strutturata senza creare accoppiamento. Si suddividono in Inbound Ports (definiscono i casi d'uso accessibili dall'esterno) e Outbound Ports (permettono al nucleo di definire interfacce per interagire con i servizi esterni).

- **Adapters:** Rappresentano lo strato più esterno e fungono da traduttori tra le tecnologie specifiche e il nucleo. Si suddividono in Driving/Input Adapters (adattatori in entrata, che guidano l'applicazione ricevendo input e invocando le Inbound Ports) e Driven/Output Adapters (adattatori in uscita, che vengono guidati dall'applicazione per interagire con l'infrastruttura esterna tramite le Outbound Ports).

3.2.2) Motivazioni della scelta architetturale

Le motivazioni per le quali questa architettura è stata scelta sono le seguenti:

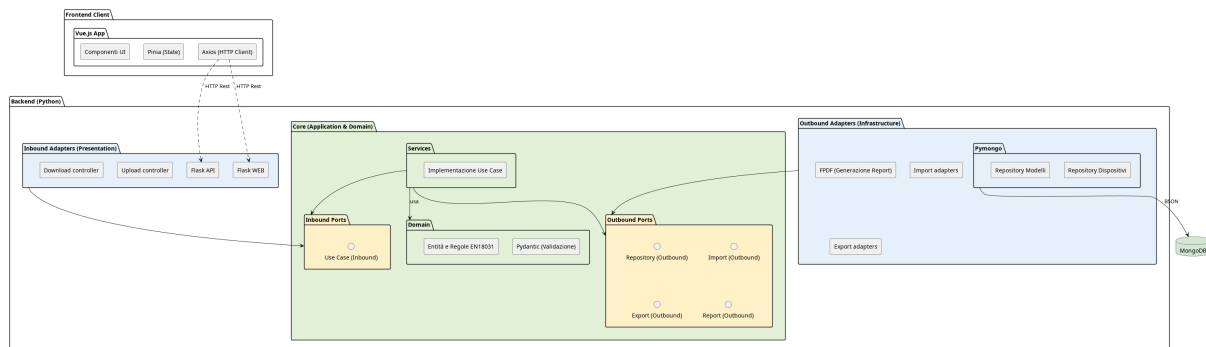
- **Testabilità:** l'assenza di dipendenze esterne nel livello di Dominio permette di scrivere Unit Test in puro Python in modo semplice e veloce. È possibile testare la logica di business in modo isolato, iniettando mock o stub per simulare le interfacce, senza la necessità di avviare il server Flask o connettersi al database MongoDB.
- **Divisione del nucleo:** il nucleo dell'applicazione è totalmente disaccoppiato dall'infrastruttura. Essendo ignaro del livello di presentazione (l'interfaccia utente in HTML, CSS e Vue.js) e del livello di persistenza (il database NoSQL MongoDB), il sistema garantisce un'alta tolleranza ai cambiamenti. È possibile sostituire o aggiornare le tecnologie esterne senza dover alterare la logica di business.
- **Sviluppo parallelo:** la rigorosa definizione dei contratti di comunicazione (le Porte) nelle fasi iniziali di progettazione permette al team di procedere in parallelo. Frontend, backend e integrazione con il database possono essere sviluppati simultaneamente da membri diversi del gruppo, riducendo i colli di bottiglia.
- **Inversione delle dipendenze:** l'architettura applica rigorosamente questo principio: è il Dominio a dettare i contratti (le interfacce) a cui i servizi esterni devono sottostare, e non viceversa. In questo modo, il nucleo dell'applicazione mantiene il controllo assoluto del flusso ed è protetto dai potenziali effetti collaterali (side effects) derivanti dall'infrastruttura.

3.2.3) Limiti dell'architettura

Gli aspetti negativi di questa scelta sono:

- **Ripida curva di apprendimento:** l'architettura esagonale richiede una profonda comprensione dei principi SOLID, in particolare la Dependency Injection e usare questo pattern per la prima volta richiede profondo studio. Questo rischio sarà mitigato da una precisa fase di progettazione che semplificherà la codifica.
- **Elevato overhead iniziale:** La ferrea separazione dei livelli impone la stesura di un'abbondante quantità di codice infrastrutturale (boilerplate). Risulta necessario definire contratti astratti (Porte), implementazioni concrete (Adattatori) e orchestratori (Servizi), allungando i tempi di sviluppo nelle prime fasi del progetto. Il gruppo ha tuttavia accettato questo costo iniziale, ritenendolo un investimento necessario a fronte del drastico abbattimento dei futuri costi di manutenzione e della massima testabilità garantita al nucleo applicativo.

3.2.4) Diagramma dei package



Il diagramma illustra l'organizzazione logica del sistema Automated EN18031 Compliance Verification, fondata sull'architettura esagonale. Il sistema è strutturalmente ripartito in un livello di presentazione esterno (Frontend Client), sviluppato tramite il framework Vue.js, e un nucleo applicativo (Backend), implementato in Python.

Per garantire una rigorosa separazione delle responsabilità e il pieno rispetto del principio di Inversione delle Dipendenze, il Backend si articola nei seguenti livelli tipici dell'architettura esagonale:

1. **Adapters:** Costituisce il confine tecnologico del sistema, responsabile della comunicazione con l'esterno. Si divide in:
 - **Inbound Adapters** (Driving Adapters): Agiscono da punto di ingresso per il sistema. Intercettano gli input esterni (es. le richieste HTTP/REST inviate dal Frontend), ne effettuano il parsing e traducono tali direttive in invocazioni dirette verso le Inbound Ports, senza mai accedere alla logica di business.
 - **Outbound Adapters** (Driven Adapters): Rappresentano le tecnologie concrete di uscita (es. il driver MongoDB o le librerie di generazione PDF). Il loro compito è implementare materialmente le interfacce definite nel livello Outbound Ports, traducendo i comandi astratti in operazioni specifiche per l'infrastruttura.
2. **Core** (Nucleo Applicativo): È il cuore del software, totalmente agnostico e privo di dipendenze verso framework web o database. Al suo interno è stratificato in:
 - **Ports:** Funge da barriera di isolamento. Definisce le interfacce in puro Python, disaccoppiando la tecnologia dall'applicazione:
 - **Inbound Ports** (Primary Ports): Definiscono le interfacce (i contratti) relative ai Casi d'Uso (Use Cases) che il sistema offre. Specificano «cosa» il sistema è in grado di fare, permettendo agli Inbound Adapters di invocare le operazioni senza dover conoscere l'implementazione sottostante.
 - **Outbound Ports** (Secondary Ports): Definiscono i contratti astratti di cui il dominio ha bisogno per comunicare con l'esterno (ad esempio, le interfacce del Repository Pattern per l'accesso ai dati). Queste porte vengono usate dal livello Services e implementate dagli Outbound Adapters.

- **Services:** Costituiscono il livello di orchestrazione. Implementano concretamente i Casi d'Uso definiti nelle Inbound Ports, coordinano il flusso di esecuzione richiamando le entità di dominio, e si avvalgono delle Outbound Ports per la persistenza dei dati.
- **Domain:** Incapsula le regole di business pure e la logica della norma EN18031. Contiene la modellazione formale delle entità e la validazione strutturale dei dati (supportata da Pydantic), operando in totale e completo isolamento.

3.3) Architettura di Deployment

Il sistema adotta un'architettura a Monolite Modulare.

L'intera applicazione è concepita, sviluppata e rilasciata come un'unica unità eseguibile.

3.3.1) Motivazioni della scelta

Questa scelta è coerente con lo sviluppo di una web app locale.

In questo contesto un'architettura a monolite modulare, rispetto all'architettura a microservizi, offre i seguenti vantaggi:

- **Semplicità di Deployment:** tutte le funzionalità sono contenute nella singola unità operativa;
- **Latenze ridotte:** lo scambio di informazioni avviene completamente in locale.

Non si è optato per una architettura monolitica tradizionale in cui spesso il codice è fortemente accoppiato.

La rigorosa divisione in moduli logici isolati permette di coniugare la semplicità di rilascio dell'applicazione come singola unità con alcuni dei vantaggi organizzativi tipici delle architetture a microservizi:

- **Separazione delle responsabilità** tra i vari ambiti del dominio.
- **Semplicità di sviluppo e manutenibilità** a lungo termine.
- **Facilità di testing**, permettendo di collaudare i singoli moduli in totale isolamento.

3.3.2) Realizzazione Pratica e Topologia

Per la distribuzione viene usato Docker come tool di containerizzazione e Docker Compose come tool di orchestrazione.

1. Nodi di Esecuzione:

- **Browser dell'utente**, l'interfaccia utente è realizzata tramite browser;
- **Server Backend**, contiene la core logic dell'applicazione e risponde alle richieste dell'utente;
- **Database MongoDB**, sistema di permanenza;

2. **Packaging e isolamento.** L'infrastruttura è orchestrata tramite Docker Compose e prevede i seguenti container:

- **Web-app**

Contiene l'ambiente Python, il framework Flask, gli asset statici del frontend e altre dipendenze¹

- **Mongodb**

Basato sull'immagine ufficiale di MongoDB. Per prevenire la perdita dei dati questo container è agganciato a un Docker Volume locale.

3. Comunicazioni e protocolli di rete

I nodi comunicano attraverso i seguenti canali e protocolli:

- **Tra Browser e Backend:**

La comunicazione avviene in chiaro tramite protocollo HTTP/REST

- Tra Backend e Database: Flask comunica con MongoDB utilizzando il protocollo binario proprietario.

La porta del database non è esposta verso il browser né verso l'esterno, garantendo l'integrità dei dati.

4. Gestione del Traffico Web

Tra il browser dell'utente e l'applicazione Python è interposto un server **WSGI**.

Questo strato intermedio garantisce stabilità, gestisce la concorrenza delle richieste e le instrada in modo sicuro verso la logica applicativa.

3.4) Diagrammi di sequenza

La seguente sezione illustra il comportamento dinamico del sistema tramite diagrammi di sequenza, focalizzandosi sui casi d'uso di maggiore interesse. Questi modelli descrivono l'ordine cronologico dei messaggi scambiati tra gli attori esterni, i componenti infrastrutturali e il nucleo applicativo.

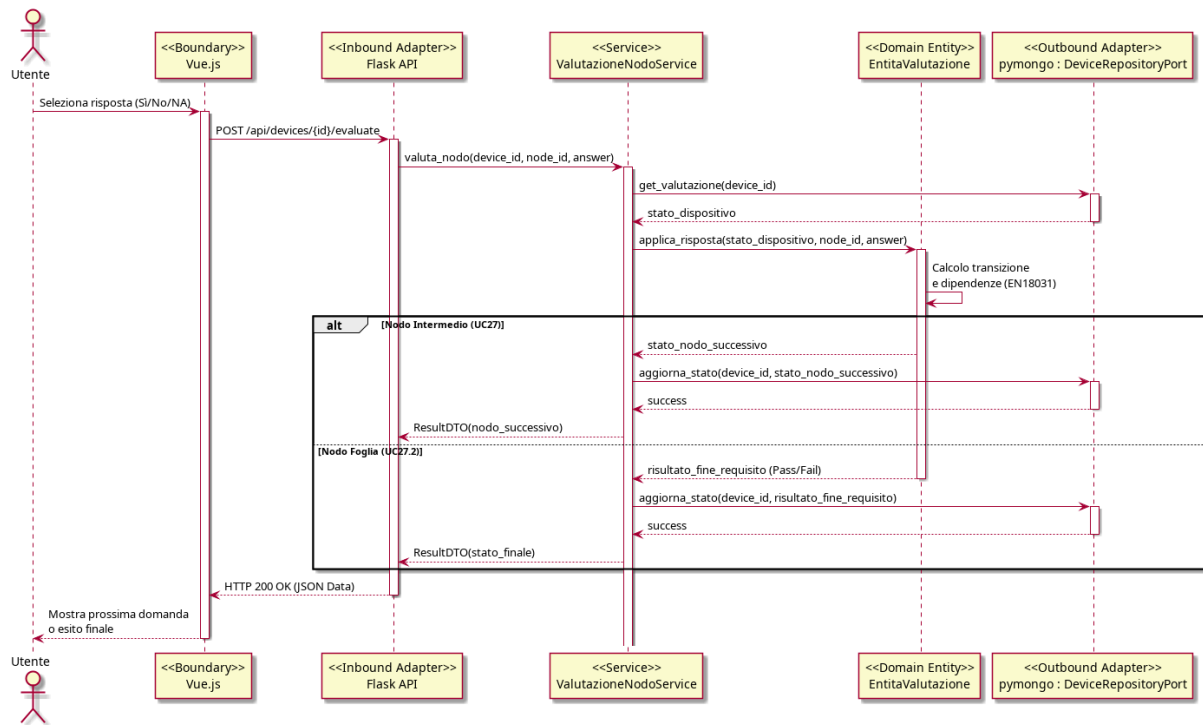
3.4.1) UC05 - Importazione dispositivo

Il diagramma illustra il processo di importazione e validazione strutturale di un dispositivo attraverso i layer dell'Architettura Esagonale. Il flusso adotta un approccio Fail-Fast diviso in due fasi. Inizialmente, l'Adattatore Inbound utilizza un Data Transfer Object (DTO) per eseguire una validazione strutturale sul formato del file in ingresso. Successivamente, il Servizio estrae i dati validati e li passa al Dominio, a cui è delegata esclusivamente la verifica delle regole normative.

Tramite le due aree alt viene modellata la gestione degli errori del caso (UC06): il primo blocco respinge i payload malformati fermandoli al confine del sistema (HTTP 400); il secondo gestisce le violazioni delle regole di business sollevate dal nucleo applicativo (HTTP 422). Solo se l'entità supera entrambi i controlli, il Servizio invoca l'Adattatore di persistenza per il salvataggio e completa l'operazione.

¹Vedi sezione [Tecnologie - Backend](#)

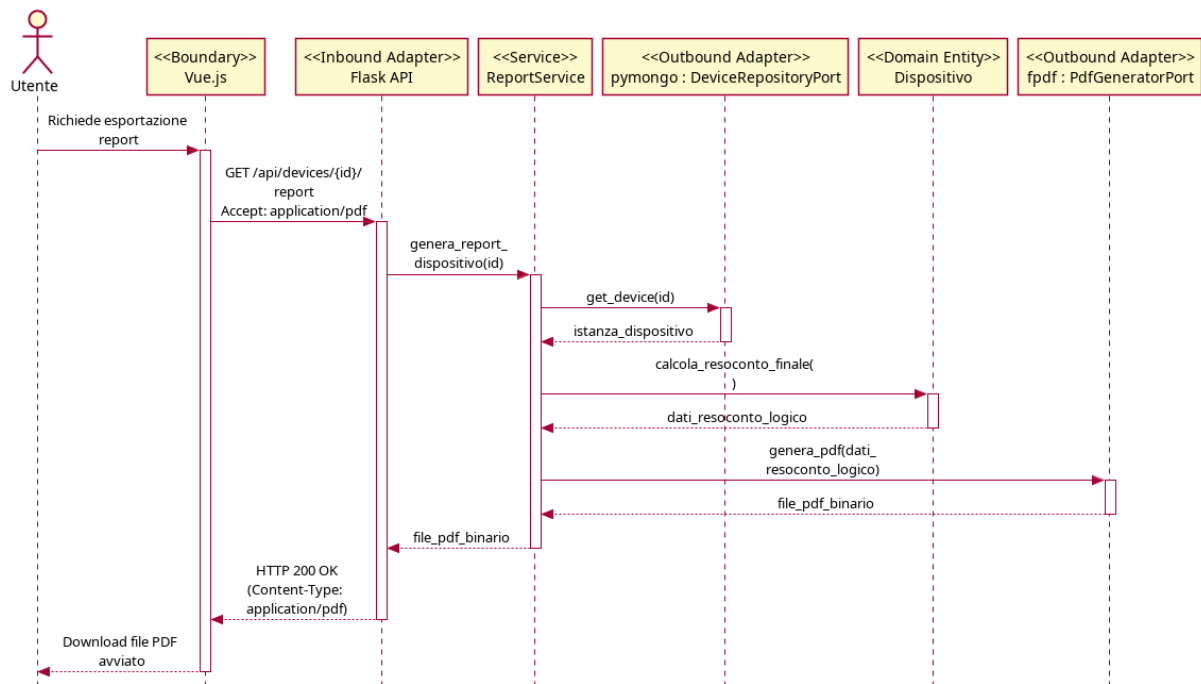
3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato



Il diagramma di sequenza illustra il flusso principale di interazione durante la valutazione di un dispositivo secondo la norma EN18031. Il diagramma evidenzia il rigoroso attraversamento dei layer architetturici, dal Boundary (Vue.js) fino al driver di persistenza (PyMongo).

Degno di nota è l'isolamento del livello Domain: il frontend e gli adapter non conoscono le regole normative. È il Servizio applicativo che recupera lo stato, lo inietta nel Dominio, e si affida al calcolo di quest'ultimo. Tramite l'uso di una sezione alt, il diagramma modella il comportamento polimorfico del flusso: se la risposta porta a un nodo intermedio, il sistema salva il progresso e restituisce la domanda successiva; se la risposta raggiunge una foglia dell'albero normativo, il sistema calcola la conformità finale (Pass/Fail) e chiude lo stato della valutazione.

3.4.3) UC30 - Esportazione report di conformità



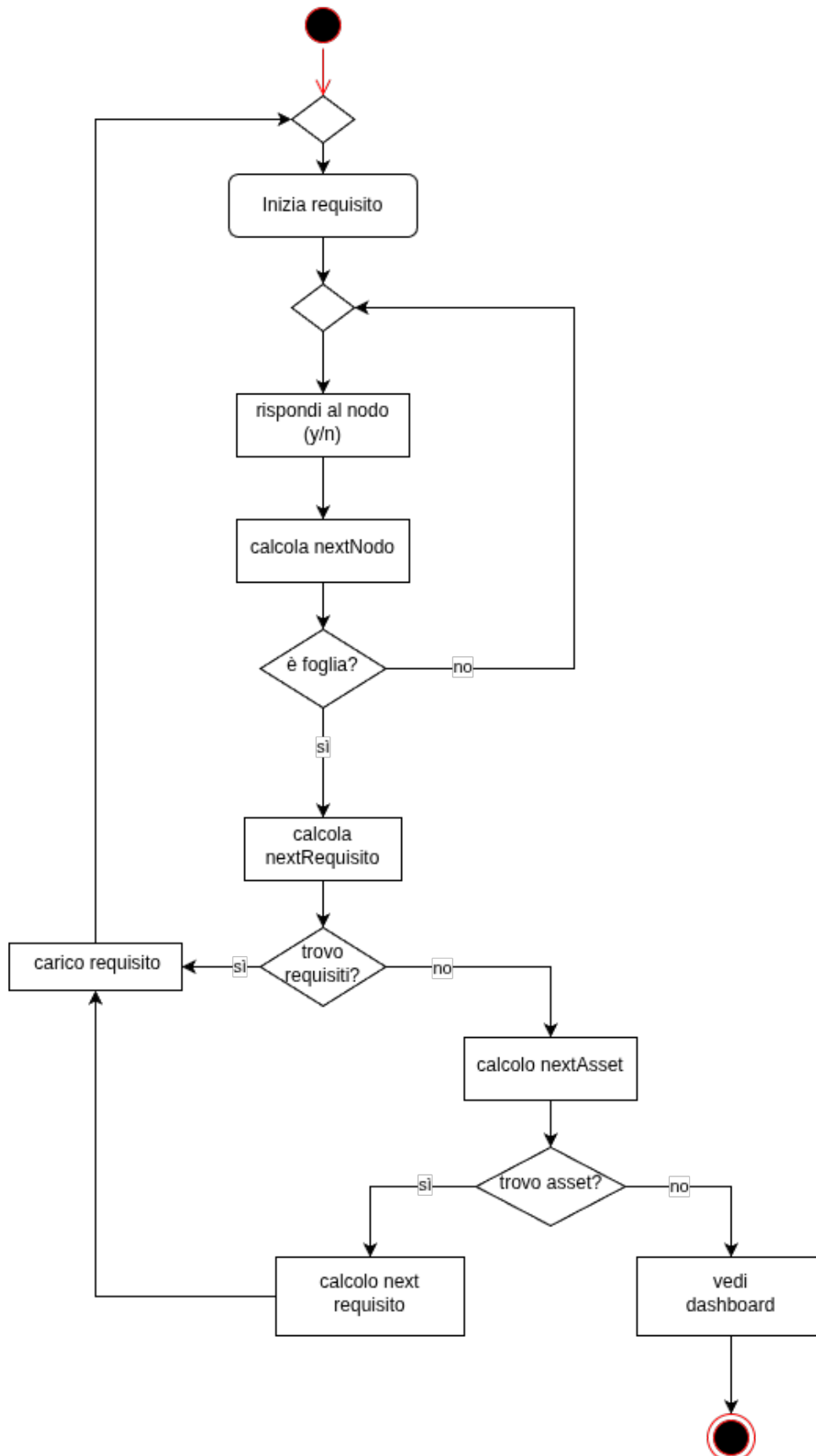
Il diagramma di sequenza illustra il processo di generazione ed esportazione del resoconto finale di conformità per un dispositivo valutato.

Il flusso è innescato da una chiamata HTTP gestita dall'Adattatore Inbound. Il Servizio applicativo avvia il recupero del documento che viene effettuato grazie all'outbound adapter (PyMongo) che estrae i dati dal database.

Una volta recuperato il dispositivo, il Servizio invoca l'aggregazione dei verdeti sul Dominio, il quale restituisce una struttura dati esclusivamente logica (Pass/Fail/NA) senza possedere alcuna conoscenza del rendering finale. Per la creazione del file fisico il Servizio usa la porta di generazione del pdf che traduce i dati puri in un layout grafico, restituendo il file che arriverà all'utente.

3.5) Diagrammi di attività

3.5.1) Navigazione degli alberi



Il diagramma di attività illustra l'algoritmo di navigazione dell'albero normativo.

Il flusso si basa su un ciclo continuo il cui innesco principale è la risposta dell'utente a uno specifico nodo (Sì/No). Dopo l'inserimento dell'input, il sistema calcola il nodo successivo e ne verifica la natura tramite un blocco decisionale («è foglia?»):

Se il nodo non è una foglia (nodo intermedio), il flusso torna indietro per sottoporre all'utente la nuova domanda appena calcolata.

Se il nodo è una foglia (ramo di valutazione concluso), il sistema innesca la logica di avanzamento gerarchico.

La fase di avanzamento procede per livelli. Dapprima, il sistema calcola e verifica se vi sono ulteriori requisiti da valutare per l'asset corrente («trovo requisiti?»). In caso positivo, il nuovo requisito viene caricato e il ciclo di domande riparte dall'inizio. In caso negativo, il sistema sale di livello verificando l'esistenza di ulteriori asset non ancora esaminati nel dispositivo («trovo asset?»). Se viene individuato un nuovo asset, ne vengono calcolati i relativi requisiti, che vengono caricati per riavviare la compilazione.

L'algoritmo fuoriesce da questo ciclo annidato solo ed esclusivamente quando sia i requisiti sia gli asset del dispositivo sono stati completamente esauriti. In questo scenario conclusivo, il sistema torna alla pagina di dashboard.

3.6) Schema dati

In questa sezione viene illustrata l'organizzazione logica dei dati all'interno del database MongoDB.

3.6.1) Scelte Architetture e Formalismo

A differenza dei sistemi relazionali basati su tabelle, MongoDB è un database **NoSQL orientato ai documenti** con struttura *schema-flexible*. Questa scelta architeturale è ideale per gestire strutture dati gerarchiche e dinamiche (come gli alberi decisionali), evitando il sovraccarico computazionale derivante da query di JOIN complesse.

Per la modellazione visiva non si utilizzano diagrammi Entity-Relationship, in quanto non idonei ai database documentali.

Si adotta invece il formalismo di Hackolade (consultabile al link: https://hackolade.com/schemas/Yelp_Challenge_dataset_documentation.html), che descrive chiaramente la gerarchia dei campi, i tipi di dato e le nidificazioni.

Nota sulla validazione dei dati: Benché i documenti vengano illustrati con tipi logici (incluso il tipo `enum` per chiarezza espositiva), la validazione strutturale di base sarà garantita dall'adapter predisposto alle comunicazioni col database

Le due *collection* principali del database sono:

- **Collection «Compliance Standards»:** Catalogo di sola lettura dei template. Contiene le definizioni degli standard, le versioni e la logica degli alberi decisionali.
- **Collection «Devices»:** Registro dinamico dei device censiti. Memorizza lo stato delle valutazioni e il riferimento puntuale al modello normativo applicato.

3.6.2) Schema Dati Device

Il documento *Device* è l'entità centrale del sistema. La sua struttura gerarchica rispecchia la scomposizione fisica e funzionale dell'oggetto analizzato in tre blocchi logici:

1. **Header Identificativo (Root):** Informazioni anagrafiche, tecniche e puntatore al modello normativo.
2. **Lista Asset (Nodi Figli):** Array di sub-documenti che modellano le singole componenti o funzionalità del dispositivo.
3. **Registro Valutazioni (Nodi Terminali):** Dizionario delle risposte fornite per i requisiti applicabili a ciascun asset.

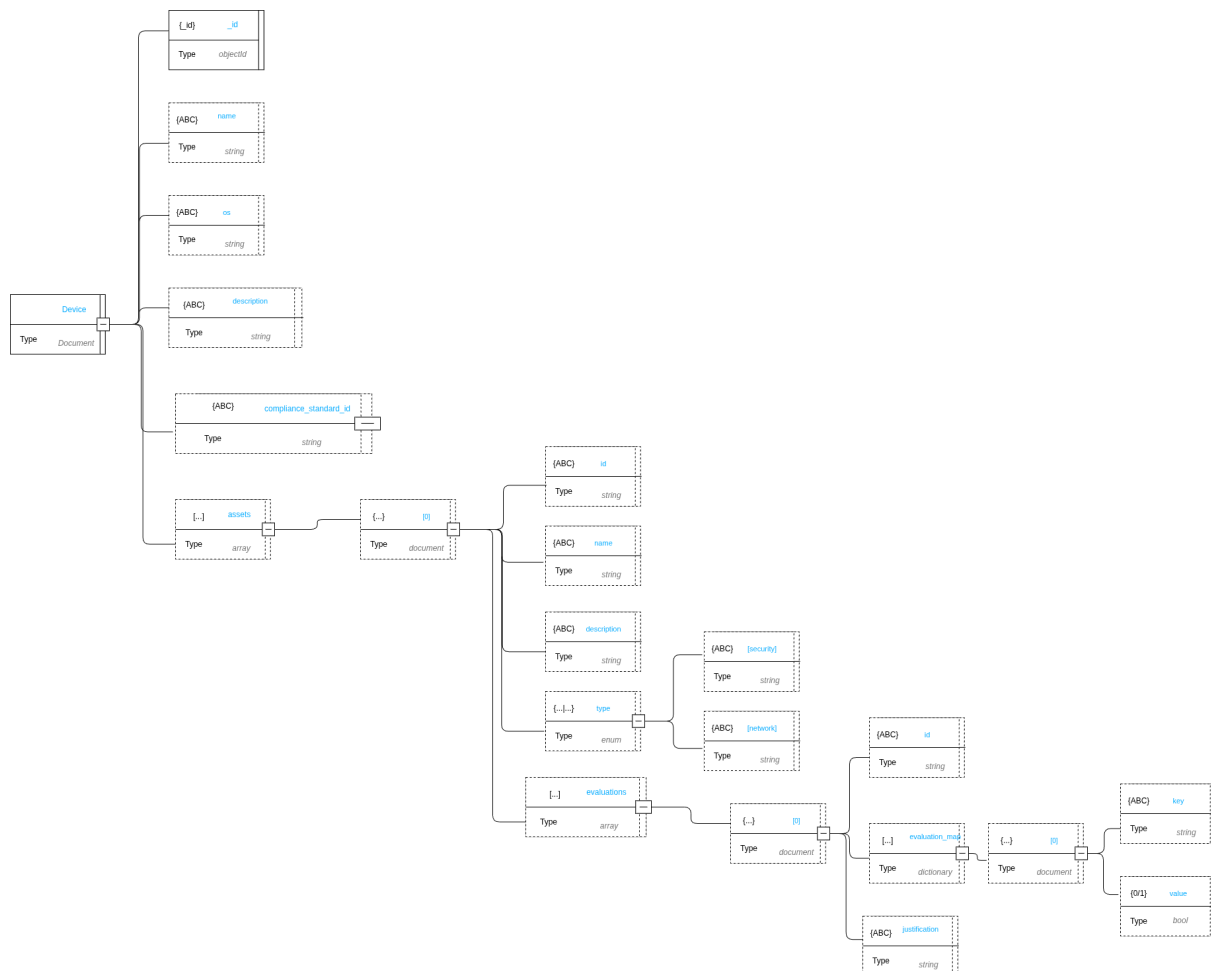


Figura 1: Schema logico: Documento Device

Root --- Device

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco generato dal database.
<code>name</code>	string	Nome assegnato dall'utente al dispositivo. Valore obbligatorio.
<code>os</code>	string	Sistema operativo o firmware installato.
<code>description</code>	string	Testo libero descrittivo del dispositivo.
<code>compliance_standard_id</code>	string	Id del modello di riferimento
<code>assets</code>	array	Elenco degli asset associati al dispositivo. Può essere inizializzato come array vuoto <code>[]</code> .

Tabella 3: Schema dati: Root — Device

Sub-documento --- assets[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	ID applicativo per l'identificazione univoca dell'asset nel dispositivo.
<code>name</code>	string	Nome descrittivo dell'asset.
<code>description</code>	string	Testo libero descrittivo dell'asset.
<code>type</code>	enum	Categoria logica dell'asset. Valori ammessi: <code>security</code> , <code>network</code> .
<code>evaluations</code>	array	Elenco delle valutazioni fornite per questo asset. Può essere vuoto <code>[]</code> .

Tabella 4: Schema dati: Sub-documento — assets[N]

Sub-documento --- evaluations[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Riferimento al codice del requisito presente nel modello.
<code>evaluation_map</code>	dictionary	Mappa chiave-valore delle risposte. La chiave (string) corrisponde al codice del nodo decisionale, il valore <code>bool</code> rappresenta la risposta associata.
<code>justification</code>	string	Testo descrittivo della motivazione. Obbligatorio a livello applicativo in caso di esito <code>FAIL</code> o <code>NA</code> .

Tabella 5: Schema dati: Sub-documento — evaluations[N]

3.6.3) Schema ComplianceStandard

La collection «Modelli» contiene la definizione strutturale degli standard normativi. A differenza del Dispositivo, orientato allo stato, il Modello descrive la logica di valutazione. Si suddivide in:

- **Metadati:** Versionamento e identificazione.
- **Albero dei Requisiti:** Struttura che definisce il testo della norma e la logica decisionale per guidare l'utente alla valutazione finale.

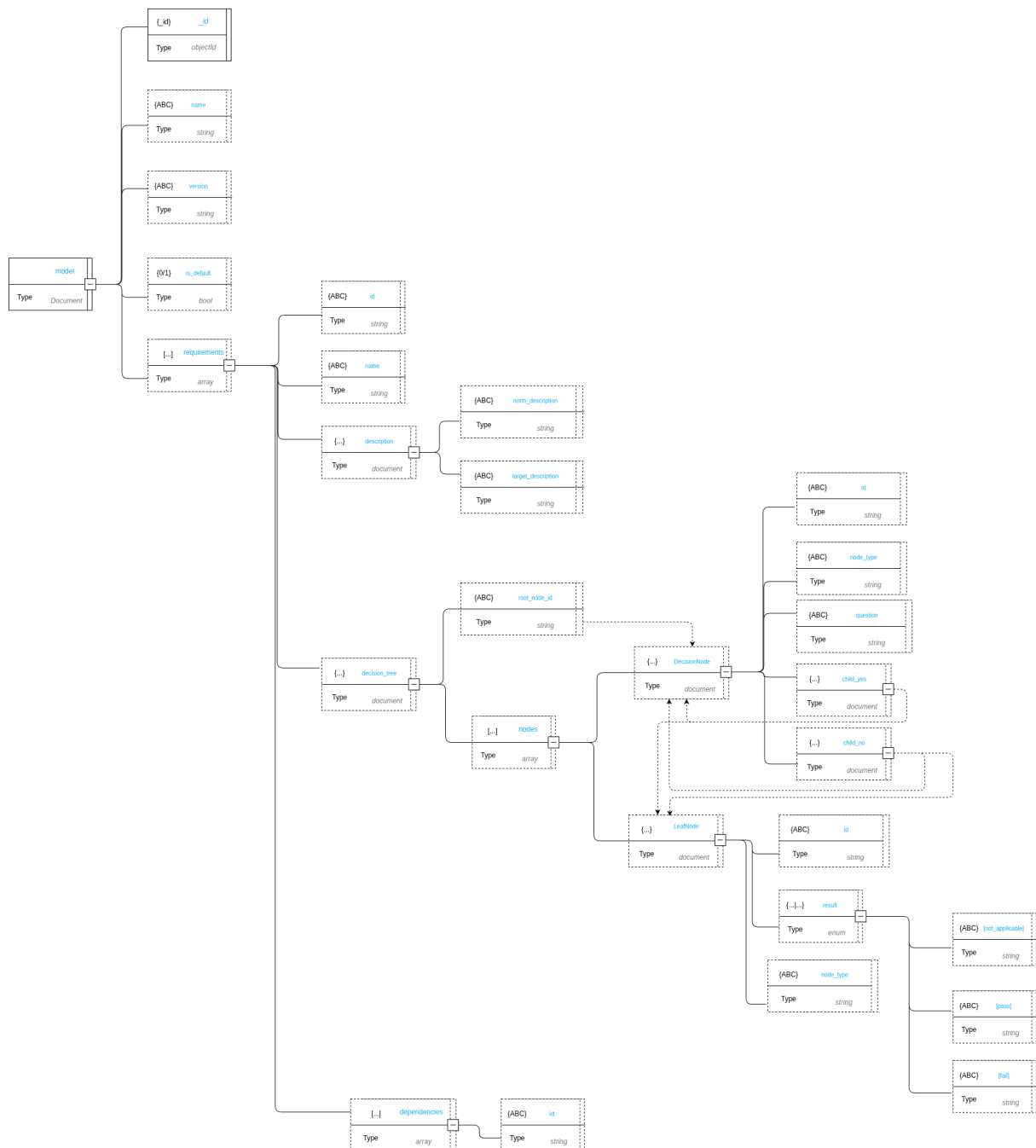


Figura 2: Schema logico: ComplianceStandard

Root --- ComplianceStandard

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco del modello.
<code>name</code>	string	Nome dello standard normativo.
<code>version</code>	string	Identificativo della versione del modello.
<code>is_default</code>	bool	Flag che indica se il modello è quello predefinito per nuove valutazioni.
<code>requirements</code>	array	Elenco dei requisiti che compongono la normativa.

Tabella 6: Schema dati: Root — ComplianceStandard

Sub-documento --- requirements[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Identificativo testuale univoco del requisito.
<code>name</code>	string	Titolo sintetico del requisito.
<code>description</code>	document	Testo descrittivi del contesto normativo (<code>norm_description</code> , <code>target_description</code>).
<code>decision_tree</code>	document	Albero decisionale per questo requisito.
<code>dependencies</code>	array	Elenco di codici di requisiti propedeutici da soddisfare prima della valutazione.

Tabella 7: Schema dati: Sub-documento — requirements[N]

Sub-documento --- DecisionNode

Campo	Tipo BSON	Descrizione
<code>root_node</code>	string	Codice identificativo del nodo logico iniziale.
<code>nodes</code>	array	Lista dei nodi che compongono il decision tree. Il <code>node_type</code> funge da discriminante tra nodi di decisione e nodi foglia. Valori ammessi da <code>node_type</code> : <code>decision_node</code> <code>leaf_node</code> .

Tabella 8: Schema dati: Sub-documento — DecisionTree

Sub-documento --- DecisionNode

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Codice identificativo del nodo logico.
<code>node_type</code>	enum	<code>decision_node</code> , identifica un nodo di decisione.
<code>question</code>	string	Testo della domanda posta all'utente.
<code>child_yes</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta affermativa.
<code>child_no</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta negativa.

Tabella 9: Schema dati: Sub-documento — DecisionNode

Sub-documento --- LeafNode

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Codice identificativo del nodo foglia.
<code>node_type</code>	enum	<code>leaf_node</code> , identifica un nodo foglia.
<code>result</code>	enum	Stato terminale del percorso logico. Valori ammessi: <code>PASS</code> , <code>FAIL</code> , <code>NA</code> .

Tabella 10: Schema dati: Sub-documento — LeafNode

3.6.4) Gestione degli Esiti e Storicità

La separazione netta tra Modello e Dispositivo ottimizza la memorizzazione e la gestione dei dati portando i seguenti vantaggi:

- **Zero Ridondanza:** Il documento Dispositivo salva unicamente il dizionario delle risposte fornite, mentre il documento Modello conserva esclusivamente la struttura statica dell'albero. Questo evita di duplicare l'intera gerarchia normativa per ogni dispositivo.
- **Disaccoppiamento:** Viene lasciato alla logica applicativa il compito di «fondere» i dati, compilando a runtime il template del Modello con le risposte storicizzate nel Dispositivo.
- **Versionamento:** La collection Modelli conserva tutte le versioni rilasciate di uno standard. Poiché il Dispositivo punta esplicitamente a una singola e specifica versione, lo storico delle certificazioni passate rimane inalterato e coerente anche a fronte di futuri aggiornamenti normativi.

4) Design Patterns

4.1) Principi di Design

4.1.1) Dependency Injection

Descrizione del pattern La dependency injection prevede che le dipendenze necessarie a una classe o a un modulo non vengano create internamente, ma fornite dall'esterno.

Generalmente questo avviene alla costruzione, tramite un setter o come parametro di un metodo.

Motivazioni dell'utilizzo del pattern La dependency injection permette di realizzare la Dependency Inversion e garantisce che le classi di dominio non dipendano mai da servizi secondari, framework o tecnologie esterne.

Utilizzo del pattern nel progetto All'interno del progetto viene usata la Constructor Injection. Il factory di Flask (`create_app()`) funge da **Composition Root**: è l'unico punto centralizzato del sistema a conoscere le implementazioni concrete delle porte e ad occuparsi di iniettarle nei service applicativi.

4.2) Pattern Architetture

4.2.1) Repository

Descrizione del pattern Il Repository è un pattern architetturale che astrae il sistema di persistenza dietro un'interfaccia che simula una collezione in memoria. Chi usa il Repository non conosce i dettagli del database sottostante.

Interagisce solo con metodi semantici come `findById()`, `save()`, `findAll()`.

Motivazioni dell'utilizzo del pattern Senza Repository la logica di business sarebbe mescolata con le query.

Inoltre i test unitari richiederebbero un database.

Con Repository il dominio e i service applicativi dipendono solo dall'interfaccia nei test si usa un mock.

Utilizzo del pattern nel progetto Nell'ambito del progetto viene utilizzata per gestire le comunicazioni con sistema di persistenza.

La lettura delle collection presenti su mongo DB avviene tramite porte dedicate.

DeviceRepositoryPort implementata da MongoDeviceRepository
e ComplianceStandardRepositoryPort implementata da
MongoComplianceStandardRepository

4.2.2) Unit of Work

Descrizione del pattern La Unit of Work è un pattern architetturale che traccia tutte le modifiche effettuate durante un'operazione e le applica al sistema di persistenza in modo atomico.

Se qualcosa va storto durante la scrittura, nessuna modifica parziale viene persistita.

Motivazioni dell'utilizzo del pattern Una sessione di valutazione può coinvolgere decine di modifiche e risposte ai nodi dei decision tree, aggiornamenti di stato, giustificazioni.

Scrivere ogni modifica immediatamente su MongoDB esporrebbe il sistema a stati inconsistenti se l'utente abbandona la sessione a metà.

La Unit of Work garantisce che il documento persistito sia sempre in uno stato coerente.

Utilizzo del pattern nel progetto Nel progetto, questo pattern regola il ciclo di sessione. Il ValutazioneService accumula e gestisce le modifiche di stato direttamente in memoria durante la navigazione.

Il documento su MongoDB non viene mai toccato o aggiornato durante i passaggi intermedi: la scrittura sul database avviene una sola volta, in modo atomico tramite il Repository, solo quando si raggiunge un nodo foglia o quando il progresso intermedio viene esplicitamente confermato.

4.3) Design Patterns Creazionali

4.3.1) Factory

Descrizione del pattern Il pattern Factory è un pattern creazionale che astrae il processo di creazione degli oggetti.

Delega a un componente specifico la responsabilità di creare oggetti complessi, restituendoli attraverso un'interfaccia comune.

Motivazioni dell'utilizzo del pattern In un'architettura esagonale, i layer di dominio devono dipendere da interfacce (Porte) e mai da implementazioni concrete.

Se un service dovesse istanziare direttamente le proprie dipendenze, finirebbe per accoppiarsi a librerie o tecnologie esterne.

Il Factory risolve questo problema incapsulando la conoscenza di «come» e «quale» oggetto concreto creare.

Utilizzo del pattern nel progetto Nel progetto il pattern è applicato a due livelli distinti:

1. **Application Factory:** A livello di framework, la funzione `create_app()` di Flask agisce come macro-factory, assemblando l'applicazione e le sue dipendenze principali all'avvio.

2. **Domain/Service Factory:** A livello applicativo, vengono utilizzate factory specifiche per costruire a runtime le implementazioni corrette di alcune interfacce che richiedono una selezione basata sulle azioni dell'utente (creare il FileExporter corrispondente al formato richiesto dall'utente), restituendo ai service oggetti pronti all'uso che rispettano l'interfaccia attesa.

4.4) Design Patterns strutturali

4.4.1) Adapter

Descrizione del pattern

L'Adapter è un pattern strutturale che traduce l'interfaccia di un componente esterno nell'interfaccia che il sistema si aspetta.

Permette a due componenti incompatibili di collaborare senza che nessuno dei due debba cambiare la propria interfaccia.

Motivazioni dell'utilizzo del pattern Il dominio definisce le proprie interfacce in termini di concetti di business.

Tramite l'uso sistematico di porte e adapter è possibile ridurre l'accoppiamento permettendo l'aggiornamento parallelo dei sistemi e lasciando all'adapter il compito di tradurre le nuove interfacce

Utilizzo del pattern nel progetto Al fine di implementare correttamente l'architettura esagonale nel sistema backend, il pattern adapter è stato usato in modo sistematico per gestire le comunicazioni verso servizi esterni.

Gli utilizzi principali sono nella comunicazione col database e con i servizi di interpretazione e generazione di file

4.4.2) Facade

Descrizione del pattern Il Facade è un pattern strutturale che fornisce un'interfaccia di alto livello e semplificata a un insieme di componenti complessi o a un intero sottosistema. L'obiettivo è nascondere la complessità interna esponendo al chiamante unicamente i metodi strettamente necessari all'azione richiesta.

Motivazioni dell'utilizzo del pattern In un'architettura a livelli, i Facade evitano che i layer esterni (come i controller o le interfacce utente) debbano orchestrare manualmente le dipendenze e la logica applicativa. Questo permette di mantenere i moduli di ingresso «sottili» e disaccoppiati dalle dinamiche interne del dominio.

Utilizzo del pattern nel progetto Nel progetto, questo pattern si riflette nei Service, come ad esempio il FileImporterService. Anche se espone un'interfaccia minimale, agisce da Facade orchestrando diverse operazioni sottostanti: riceve la richiesta, coordina i servizi di estrazione e parsing del file, e istruisce il sistema di persistenza per il salvataggio, nascondendo l'intera complessità della sequenza al chiamante.

4.5) Design Patterns Comportamentali

4.5.1) Strategy

Descrizione del pattern Lo Strategy è un pattern comportamentale che definisce una famiglia di algoritmi intercambiabili, incapsulandoli in classi separate che condividono la stessa interfaccia. Il chiamante seleziona quale algoritmo usare a runtime senza modificare il proprio codice.

Motivazioni dell'utilizzo del pattern Il sistema deve supportare più formati di import ed export — JSON, CSV, XML — e potenzialmente nuovi formati in futuro. Con Strategy supportare un nuovo formato consiste nel creare una nuova classe che implementa lo strategy pattern senza modificare il codice già esistente.

Utilizzo del pattern nel progetto Viene utilizzato nel processo di importazione del file

4.5.2) Template Method

Descrizione del pattern Il Template Method è un pattern comportamentale che definisce lo scheletro di un algoritmo in una classe astratta, delegando alle sottoclassi l'implementazione dei passi specifici.

La sequenza complessiva è fissa; ciò che cambia è il comportamento dei singoli passi.

Motivazioni dell'utilizzo del pattern Tutti i formati di export condividono la stessa sequenza operativa: preparare la struttura, scrivere i dati, finalizzare l'output.

Senza Template Method questa sequenza dovrebbe essere replicata in ogni exporter concreto, introducendo duplicazione e rischio di inconsistenze.

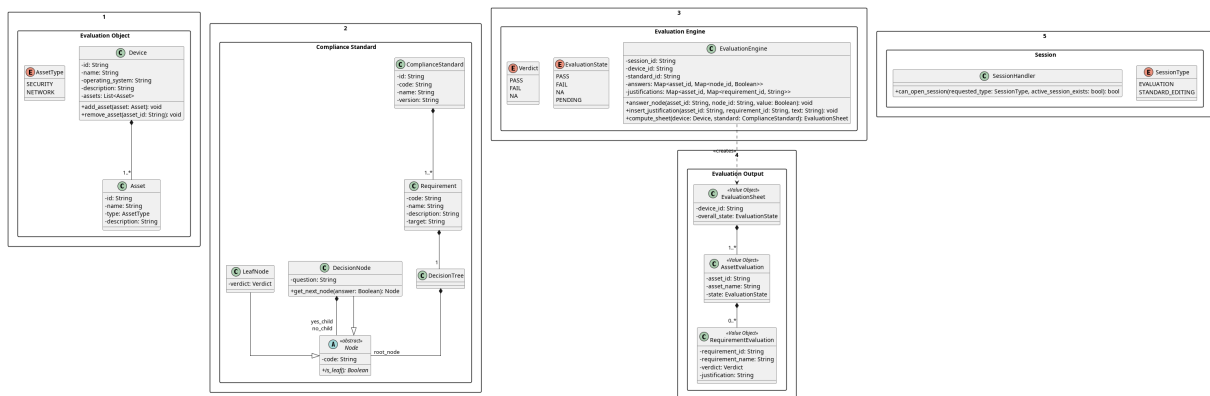
Il Template Method consente di scrivere la sequenza una sola volta nella classe astratta, lasciando alle sottoclassi solo i dettagli specifici del formato.

Utilizzo del pattern nel progetto Viene utilizzato durante l'esportazione di informazioni sotto formato di file in diversi formati.

5) Diagrammi delle classi

5.1) Diagramma di dominio

Il diagramma delle classi illustra il Modello di Dominio del nucleo applicativo. Progettato rispettando i principi del Domain-Driven Design e dell'Architettura Esagonale, il modello incapsula esclusivamente la logica di business pura, risultando del tutto agnostico rispetto ai dettagli infrastrutturali.



Per garantire coerenza logica e separazione delle responsabilità, le classi sono state logicamente organizzate in tre macro-package:

1. Oggetti di Valutazione

Questo package modella le entità concrete sottoposte alla valutazione:

- **Dispositivo:** mantiene lo stato globale dell'oggetto in esame e gestisce il ciclo di vita delle sue componenti.
- **Asset:** rappresenta i singoli componenti fisici o logici (classificati tramite l'enumerazione TipoAsset in Security o Network) che costituiscono il dispositivo.

La relazione tra Dispositivo e Asset è una composizione. La distruzione logica di un dispositivo all'interno del sistema comporta la distruzione dei relativi asset associati.

2. Modello Normativo

Questo package incapsula la struttura formale della norma di riferimento.

- **ModelloStandard e Requisito:** Il ModelloStandard definisce l'anagrafica della norma e si compone di molteplici Requisiti. La relazione riflessiva su Requisito («dipende da») permette di modellare i vincoli di dipendenza tra requisiti.
- **DecisionTree:** la struttura dell'albero di valutazione è stata modellata sfruttando il Design Pattern Composite. La classe astratta NodoNormativo definisce il contratto base, che viene implementato polimorficamente da:

- **NodoDecisionale**: nodi intermedi che incapsulano una domanda e definiscono una biforcazione logica (YES/NO) verso i nodi successivi.
- **NodoFoglia**: nodi terminali del ramo decisionale che emettono un Verdetto di conformità (PASS, FAIL, NA) per l'albero a cui appartengono

3. Navigazione e Valutazione

Questo package funge da area di interazione tra l'entità fisica e la regola normativa.

- **ValutazioneRequisito**: Agisce come Classe di Associazione tra uno specifico Asset e uno specifico Requisito. Questa scelta progettuale separa nettamente la definizione della norma dallo stato della valutazione corrente. Contiene una struttura dati (`mapRisposte`) fondamentale per memorizzare le risposte fornite dall'utente durante la navigazione del DecisionTree, garantendo la possibilità di sospendere e riprendere la compilazione.
- **ReportConformita**: È legato a **ValutazioneRequisito** tramite una relazione di aggregazione: il report colleziona le singole valutazioni per generare l'esito finale, ma una sua eventuale eliminazione o rigenerazione non invalida i dati delle valutazioni persistite nel sistema.

Viene fatto utilizzo di enumerazioni (`TipoAsset`, `StatoValutazione`, `Verdetto`). Questa scelta impone vincoli di dominio stringenti, garantendo la type-safety ed evitando stati di valutazione non previsti dal capitolato.

5.2) Device

5.2.1) WriteDeviceModule

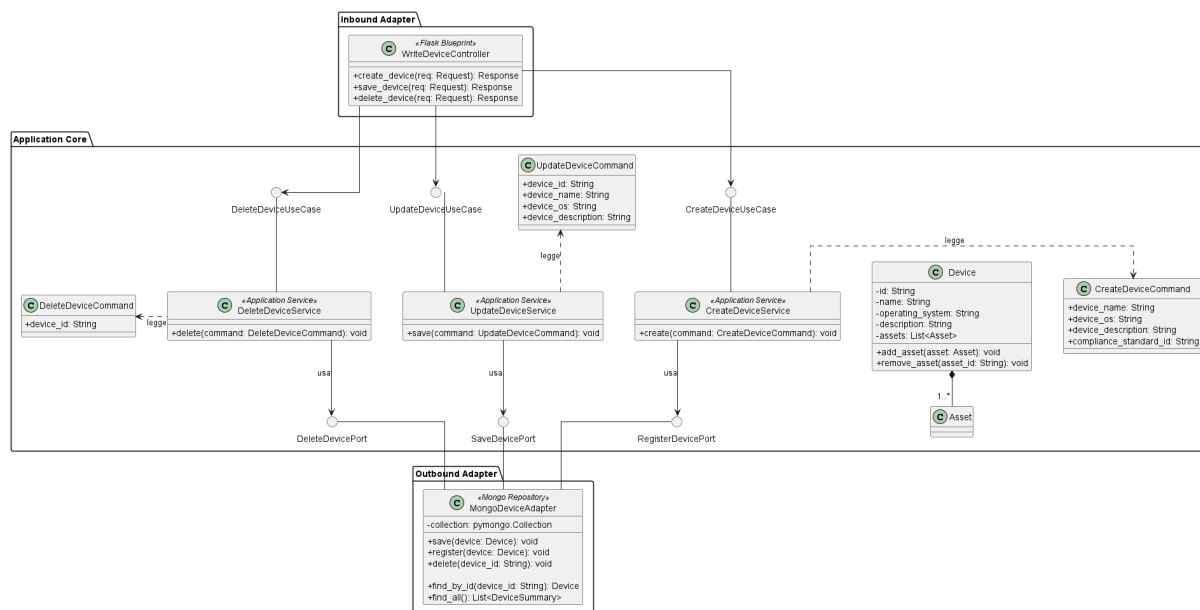


Figura 3: Modulo di scrittura Dispositivi

Il diagramma offre una visione d'insieme del modulo di scrittura per la gestione dei Dispositivi, mostrando come i tre casi d'uso — *CreateDevice*, *SaveDevice* e *DeleteDevice* — condividano gli stessi componenti infrastrutturali (*WriteDeviceController* e *MongoDeviceAdapter*) pur introducendo ciascuno le proprie interfacce e service dedicati. I componenti sono descritti in dettaglio nelle sezioni seguenti.

5.2.2) CreateDevice



Figura 4: Caso d'uso CreateDevice

Il diagramma illustra l'architettura del modulo di scrittura per la gestione dei Dispositivi, coprendo le operazioni di creazione, modifica ed eliminazione secondo i principi dell'architettura esagonale.

5.2.2.1) WriteDeviceController

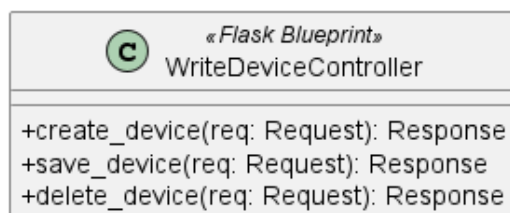


Figura 5: WriteDeviceController

Descrizione

WriteDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione dei Dispositivi e le inoltra al livello applicativo.

Attributi

WriteDeviceController non definisce attributi propri.

Metodi e funzioni

- + `create_device(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo Dispositivo, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `save_device(req: Request): Response` — riceve la richiesta HTTP di modifica di un Dispositivo esistente, estrae i dati aggiornati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `delete_device(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un Dispositivo, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.2.2.2) CreateDeviceUseCase

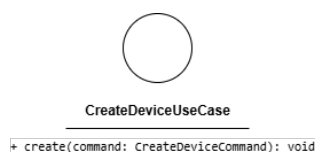


Figura 6: CreateDeviceUseCase

Descrizione

CreateDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la creazione di un nuovo Dispositivo. Viene implementata da *CreateDeviceService* e utilizzata da *WriteDeviceController*.

Attributi

CreateDeviceUseCase non definisce attributi.

Metodi e funzioni

- + create(command: CreateDeviceCommand): void — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.2.2.3) CreateDeviceCommand

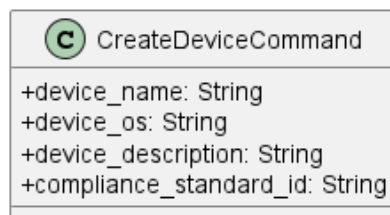


Figura 7: CreateDeviceCommand

Descrizione

CreateDeviceCommand è il Command Object che veicola i dati necessari alla creazione di un Dispositivo dal controller al service. L'uso di questo pattern separa la struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- + device_name: String — nome del nuovo Dispositivo.
- + device_os: String — sistema operativo del Dispositivo.
- + device_description: String — descrizione testuale del Dispositivo.
- + compliance_standard_id: String — identificativo dello standard di conformità associato.

Metodi e funzioni

CreateDeviceCommand non definisce metodi.

5.2.2.4) CreateDeviceService



Figura 8: CreateDeviceService

Descrizione

CreateDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un Dispositivo. Implementa l'interfaccia *CreateDeviceUseCase*, riceve il comando in ingresso, costruisce l'entità di dominio e ne coordina la persistenza tramite *RegisterDevicePort*.

Attributi

- - register_device_port: RegisterDevicePort — porta outbound utilizzata per la persistenza del nuovo Dispositivo.

Metodi e funzioni

- + `create(command: CreateDeviceCommand): void` — concretizza il contratto definito da *CreateDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede la registrazione tramite *RegisterDevicePort*.

5.2.2.5) Device

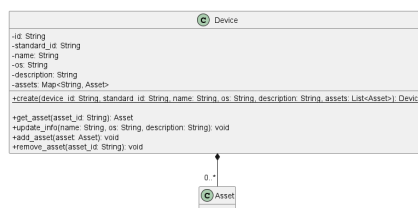


Figura 9: Device

Descrizione

Device è l'entità centrale del dominio che rappresenta il Dispositivo oggetto della valutazione di conformità. È associata alla classe *Asset* con una relazione di composizione avente cardinalità `0..*`.

Attributi

- - `id: String` — identificativo univoco del Dispositivo.
- - `standard_id: String` — identificativo dello standard di conformità associato al dispositivo.
- - `name: String` — nome del Dispositivo.
- - `os: String` — sistema operativo del Dispositivo.
- - `description: String` — descrizione testuale del Dispositivo.
- - `assets: Map<String, Asset>` — mappa degli asset associati al Dispositivo, indicizzati per il loro identificativo.

Metodi e funzioni

- + `create(device_id: String, standard_id: String, name: String, os: String, description: String, assets: List<Asset>): Device` — metodo per la creazione e istanziazione di un nuovo oggetto Dispositivo.
- + `get_asset(asset_id: String): Asset` — recupera l'Asset corrispondente all'identificativo fornito.
- + `update_info(name: String, os: String, description: String): void` — aggiorna le informazioni anagrafiche del Dispositivo (nome, sistema operativo e descrizione).
- + `add_asset(asset: Asset): void` — aggiunge un nuovo *Asset* alla mappa del Dispositivo.
- + `remove_asset(asset_id: String): void` — rimuove l'Asset identificato da `asset_id` dalla mappa del Dispositivo.

5.2.2.6) RegisterDevicePort

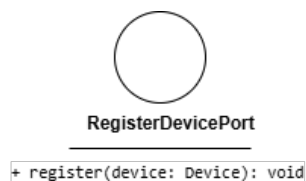


Figura 10: RegisterDevicePort

Descrizione

RegisterDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per la registrazione di un nuovo Dispositivo nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *CreateDeviceService*.

Attributi

RegisterDevicePort non definisce attributi.

Metodi e funzioni

- + register(device: Device): void — firma del metodo che esegue l'inserimento fisico del Dispositivo nel sistema di persistenza.

5.2.2.7) MongoDeviceAdapter



Figura 11: MongoDeviceAdapter

Descrizione

MongoDeviceAdapter è la classe dell'Outbound Adapter che implementa le porte outbound del sistema, traducendo le operazioni del dominio in interazioni concrete con MongoDB tramite `pymongo.Collection`.

Attributi

- - collection: pymongo.Collection — riferimento alla collezione MongoDB su cui vengono eseguite le operazioni di persistenza.

Metodi e funzioni

- + `save(device: Device): void` — salva le modifiche a un Dispositivo esistente nella collezione.
- + `register(device: Device): void` — inserisce un nuovo Dispositivo nella collezione.
- + `delete(device_id: String): void` — rimuove il Dispositivo identificato da `device_id` dalla collezione.
- + `find_by_id(device_id: String): Device` — recupera il Dispositivo corrispondente all'identificativo fornito.
- + `find_all(): List<DeviceSummary>` — recupera la lista sintetica di tutti i Dispositivi presenti nella collezione.

5.2.3) DeleteDevice

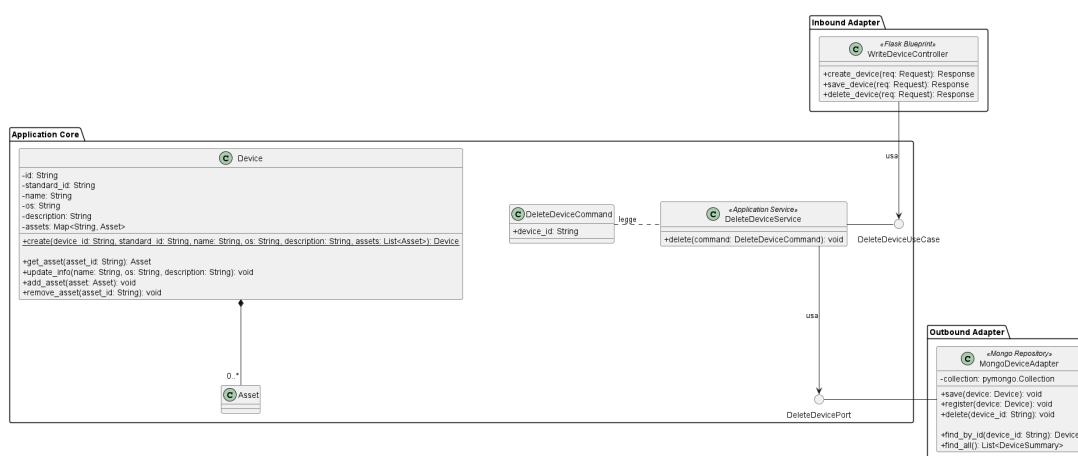


Figura 12: Caso d'uso DeleteDevice

Il diagramma illustra l'architettura del modulo dedicato all'eliminazione di un Dispositivo.

- Per la definizione di *WriteDeviceController*, vedere la sezione **Sezione 5.2.2.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.3.1) DeleteDeviceUseCase

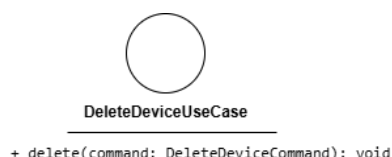


Figura 13: DeleteDeviceUseCase

Descrizione

DeleteDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'eliminazione di un Dispositivo. Viene implementata da *DeleteDeviceService* e utilizzata da *WriteDeviceController*.

Attributi

DeleteDeviceUseCase non definisce attributi.

Metodi e funzioni

- + delete(device_id: String): void — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dall'identificativo del Dispositivo.

5.2.3.2) DeleteDeviceCommand

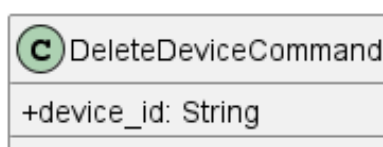


Figura 14: DeleteDeviceCommand

Descrizione

DeleteDeviceCommand è il Command Object utilizzato per trasportare i dati necessari all'eliminazione di un Dispositivo. Segue il pattern Command, separando i dati di input dalla logica applicativa del servizio.

Attributi

- + device_id: String — identificativo univoco del Dispositivo da eliminare.

Metodi e funzioni

DeleteDeviceCommand non definisce metodi propri.

5.2.3.3) DeleteDeviceService

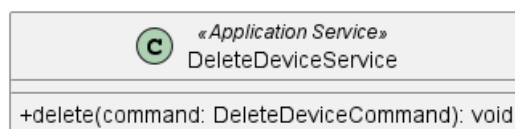


Figura 15: DeleteDeviceService

Descrizione

DeleteDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Dispositivo. Implementa l'interfaccia *DeleteDeviceUseCase* e riceve un *DeleteDeviceCommand* contenente l'identificativo del Dispositivo, coordinandone la rimozione dal sistema.

Attributi

DeleteDeviceService non definisce attributi propri.

Metodi e funzioni

- + `delete(command: DeleteDeviceCommand): void` — riceve il Command Object contenente l'identificativo univoco del Dispositivo e ne coordina la rimozione tramite *DeleteDevicePort*.

5.2.3.4) DeleteDevicePort

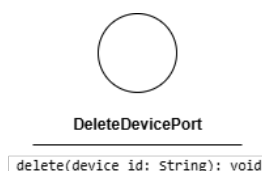


Figura 16: DeleteDevicePort

Descrizione

DeleteDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per la rimozione fisica di un Dispositivo dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *DeleteDeviceService*.

Attributi

DeleteDevicePort non definisce attributi.

Metodi e funzioni

- + `delete(device_id: String): void` — firma del metodo che esegue la rimozione fisica del Dispositivo identificato da `device_id` dal sistema di persistenza.

5.2.4) UpdateDevice

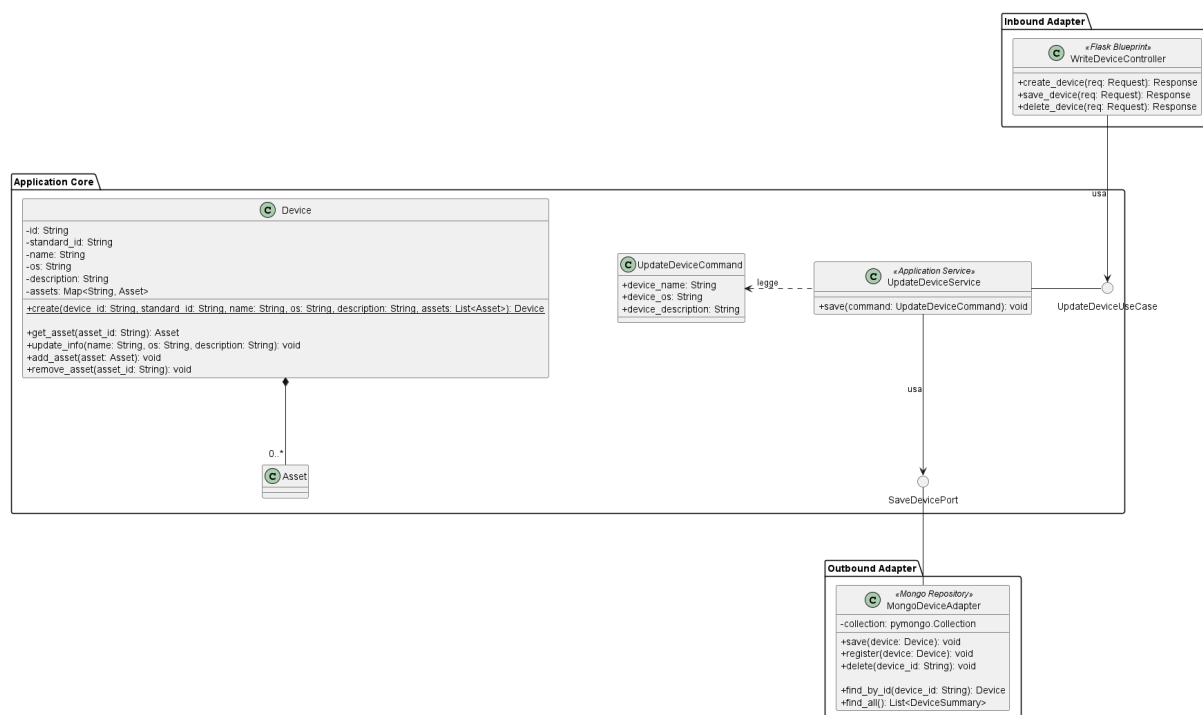


Figura 17: Caso d'uso UpdateDevice

Il diagramma illustra l'architettura del modulo dedicato alla modifica e al salvataggio dello stato di un Dispositivo esistente.

- Per la definizione di *WriteDeviceController*, vedere la sezione **Sezione 5.2.2.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.4.1) UpdateDeviceUseCase



Figura 18: UpdateDeviceUseCase

Descrizione

UpdateDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la modifica di un Dispositivo esistente. Viene implementata da *UpdateDeviceService* e utilizzata da *WriteDeviceController*.

Attributi

UpdateDeviceUseCase non definisce attributi.

Metodi e funzioni

- `save(command: UpdateDeviceCommand)` — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.2.4.2) SaveDeviceCommand

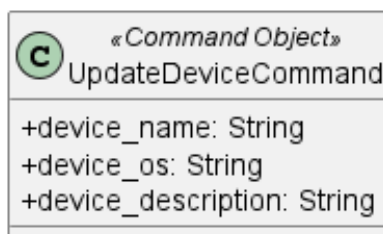


Figura 19: UpdateDeviceCommand

Descrizione

UpdateDeviceCommand è il Command Object che veicola i dati necessari alla modifica di un Dispositivo dal controller al service. Analogamente a *CreateDeviceCommand*, separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- `+ device_name: String` — nome aggiornato del Dispositivo.
- `+ device_os: String` — sistema operativo aggiornato.
- `+ device_description: String` — descrizione testuale aggiornata.

Metodi e funzioni

UpdateDeviceCommand non definisce metodi.

5.2.4.3) UpdateDeviceService

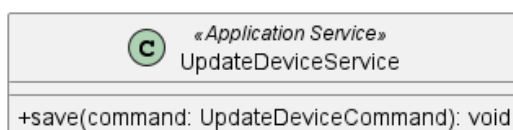


Figura 20: UpdateDeviceService

Descrizione

UpdateDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un Dispositivo esistente. Implementa l'interfaccia *UpdateDeviceUseCase*, riceve il comando in ingresso e ne coordina la persistenza tramite *SaveDevicePort*.

Attributi

UpdateDeviceService non definisce attributi propri.

Metodi e funzioni

- + `save(command: UpdateDeviceCommand): void` — concretizza il contratto definito da *SaveDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede l'aggiornamento tramite *SaveDevicePort*.

5.2.4.4) SaveDevicePort

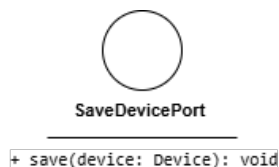


Figura 21: SaveDevicePort

Descrizione

SaveDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per l'aggiornamento fisico di un Dispositivo nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *UpdateDeviceService*.

Attributi

UpdateDevicePort non definisce attributi.

Metodi e funzioni

- + `save(device: Device): void` — firma del metodo che esegue l'aggiornamento fisico del Dispositivo nel sistema di persistenza.

5.2.5) ReadDeviceModule

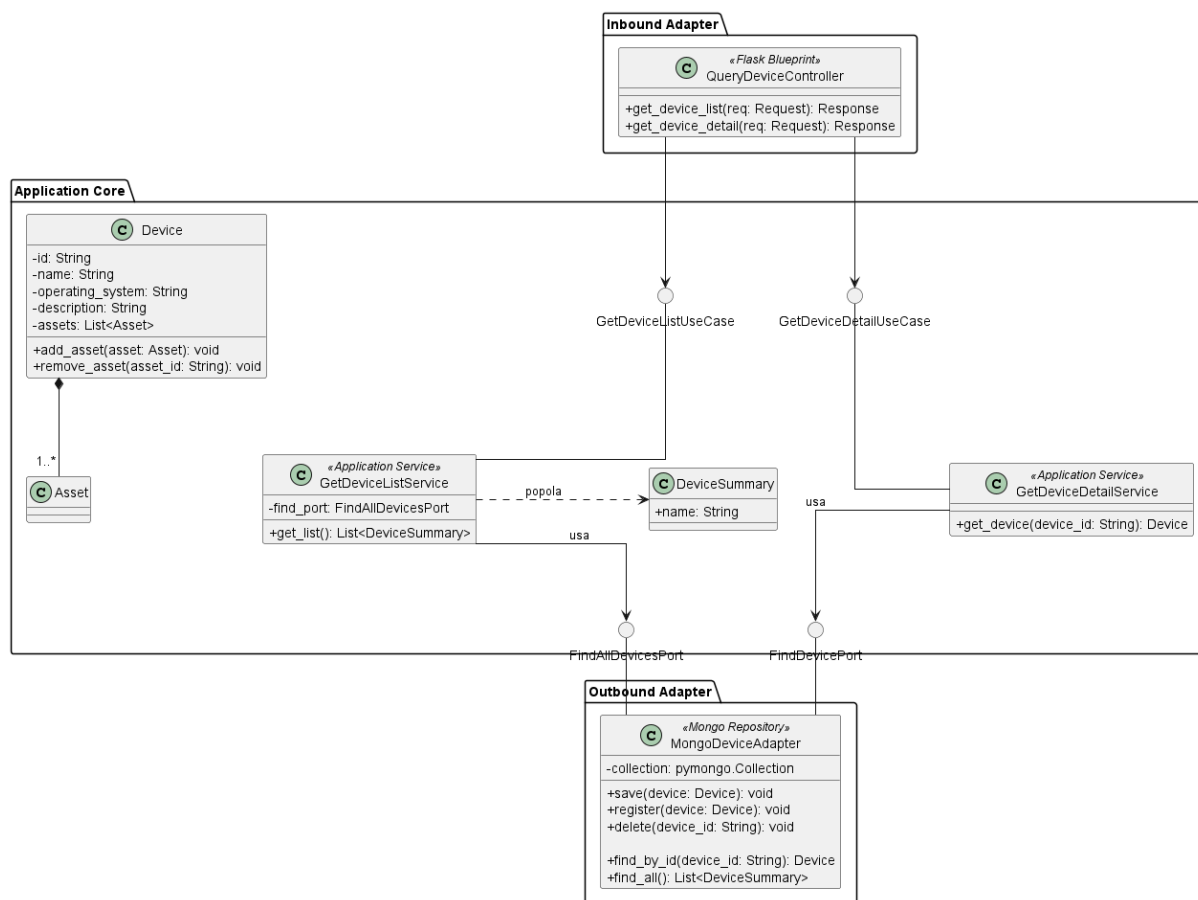


Figura 22: Modulo di lettura Dispositivi

Il diagramma illustra l'architettura del modulo di lettura per la gestione dei Dispositivi, coprendo le operazioni di recupero del dettaglio di un singolo Dispositivo e della lista sintetica di tutti i Dispositivi registrati.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo modulo.

5.2.6) GetDeviceDetail

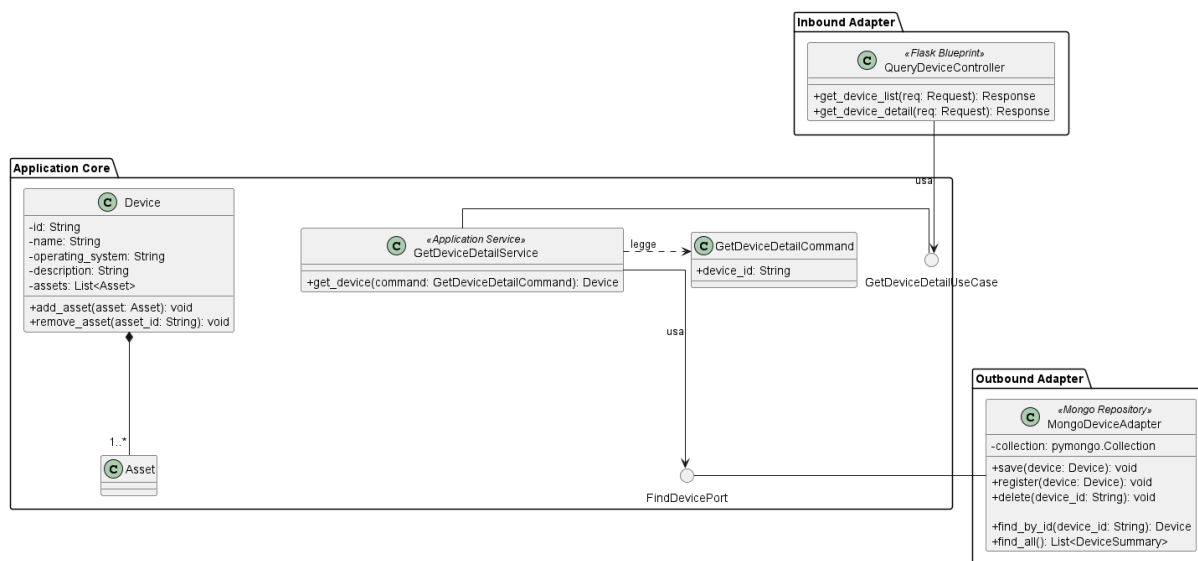


Figura 23: GetDeviceDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un Dispositivo.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.6.1) QueryDeviceController



Figura 24: QueryDeviceController

Descrizione

QueryDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di lettura relative ai Dispositivi e le inoltra al livello applicativo.

Attributi

QueryDeviceController non definisce attributi propri.

Metodi e funzioni

- `+ get_device_list(req: Request): Response` — riceve la richiesta HTTP di recupero della lista dei Dispositivi e restituisce una risposta HTTP con l'elenco sintetico.

- + `get_device_detail(req: Request): Response` — riceve la richiesta HTTP di recupero del dettaglio di un Dispositivo specifico e restituisce una risposta HTTP con i dati completi.

5.2.7) GetDeviceDetailUseCase

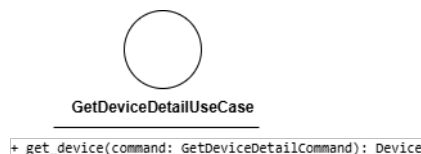


Figura 25: GetDeviceDetailUseCase

Descrizione

GetDeviceDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Dispositivo. Viene implementata da *GetDeviceDetailService* e utilizzata da *QueryDeviceController*.

Attributi

GetDeviceDetailUseCase non definisce attributi.

Metodi e funzioni

- + `get_device(command: GetDeviceDetailCommand): Device` — firma del metodo delegato al recupero del Dispositivo corrispondente al Command fornito.

5.2.7.1) GetDeviceDetailService

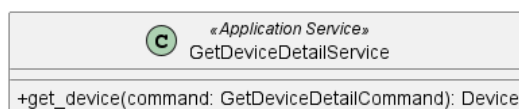


Figura 26: GetDeviceDetailService

Descrizione

GetDeviceDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Dispositivo. Implementa l'interfaccia *GetDeviceDetailUseCase* e coordina il recupero dei dati tramite la porta outbound *FindDevicePort*.

Attributi

GetDeviceDetailService non definisce attributi propri.

Metodi e funzioni

- + `get_device(command: GetDeviceDetailCommand): Device` — concretizza il contratto definito da *GetDeviceDetailUseCase*. Recupera il Dispositivo corrispondente all'identificativo contenuto nel Command tramite *FindDevicePort*.

5.2.7.2) GetDeviceDetailCommand

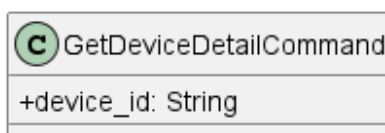


Figura 27: GetDeviceDetailCommand

Descrizione

GetDeviceDetailCommand è utilizzato per trasportare i dati necessari al recupero del dettaglio di un Dispositivo. Incapsula i parametri di input del metodo esposto da *GetDeviceDetailUseCase*.

Attributi

- + `device_id: String` — identificativo univoco del Dispositivo di cui recuperare il dettaglio.

Metodi e funzioni

GetDeviceDetailCommand non definisce metodi propri.

5.2.7.3) FindDevicePort

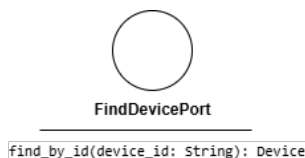


Figura 28: FindDevicePort

Descrizione

FindDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero di un Dispositivo dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *GetDeviceDetailService*.

Attributi

FindDevicePort non definisce attributi.

Metodi e funzioni

- + `find_by_id(device_id: String): Device` — firma del metodo che recupera il Dispositivo corrispondente all'identificativo fornito dal sistema di persistenza.

5.2.8) GetDeviceList

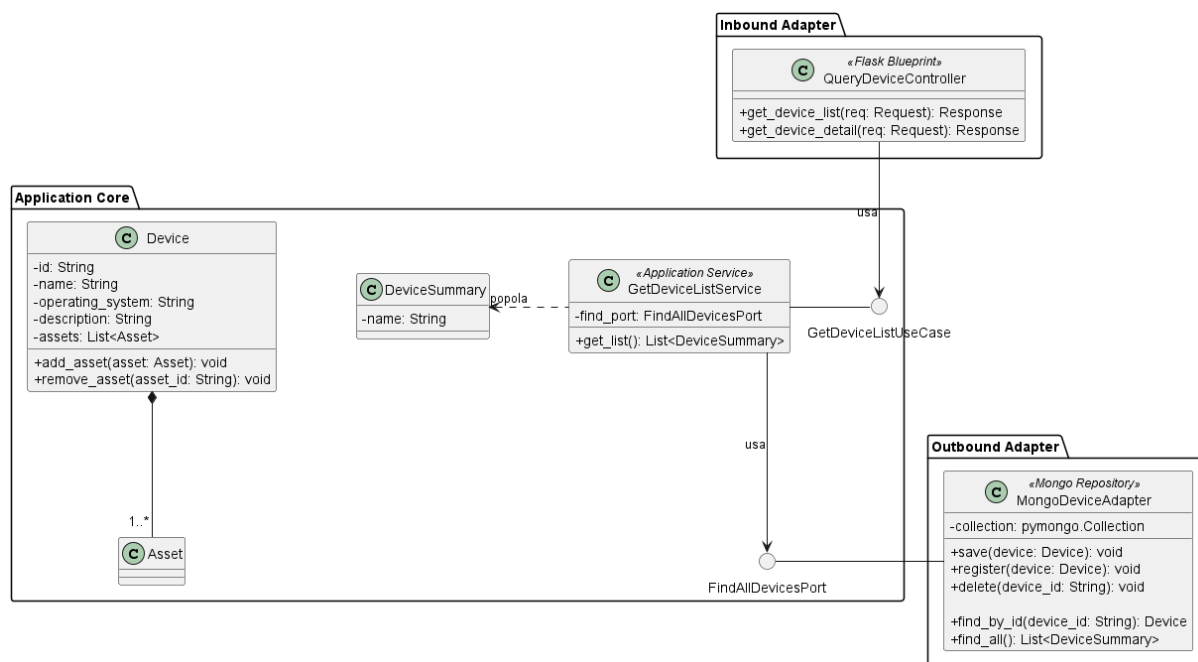


Figura 29: Caso d'uso GetDeviceList

Il diagramma illustra l'architettura del modulo dedicato al recupero della lista sintetica dei Dispositivi.

Per la definizione di *QueryDeviceController*, vedere la sezione **Sezione 5.2.6.1**.

Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.

Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.8.1) GetDeviceListUseCase

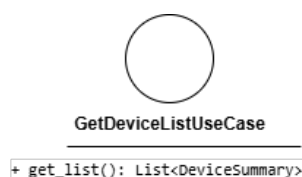


Figura 30: GetDeviceListUseCase

Descrizione

GetDeviceListUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero della lista sintetica dei Dispositivi. Viene implementata da *GetDeviceListService* e utilizzata da *QueryDeviceController*.

Attributi

GetDeviceListUseCase non definisce attributi.

Metodi e funzioni

- + `get_list(): List<DeviceSummary>` — firma del metodo delegato al recupero della lista sintetica di tutti i Dispositivi presenti nel sistema.

5.2.8.2) GetDeviceListService

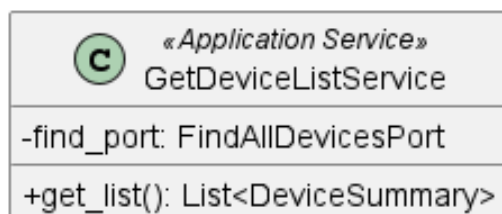


Figura 31: GetDeviceListService

Descrizione

GetDeviceListService è il service applicativo appartenente all'Application Core responsabile del recupero della lista sintetica dei Dispositivi. Implementa l'interfaccia *GetDeviceListUseCase* e coordina il recupero tramite la porta outbound *FindAllDevicesPort*.

Attributi

- - `find_port: FindAllDevicesPort` — porta outbound utilizzata per il recupero della lista dei Dispositivi dal sistema di persistenza.

Metodi e funzioni

- + `get_list(): List<DeviceSummary>` — concretizza il contratto definito da *GetDeviceListUseCase*. Recupera la lista sintetica di tutti i Dispositivi tramite *FindAllDevicesPort*.

5.2.8.3) DeviceSummary

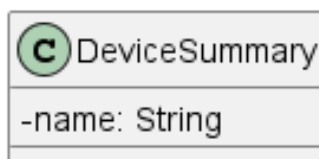


Figura 32: DeviceSummary

Descrizione

DeviceSummary è il Data Transfer Object che veicola la rappresentazione sintetica di un Dispositivo verso il livello di presentazione. Espone esclusivamente le informazioni necessarie alla visualizzazione in lista, evitando di esporre l'intera entità di dominio.

Attributi

- - `name: String` — nome del Dispositivo.

Metodi e funzioni

DeviceSummary non definisce metodi.

5.2.8.4) FindAllDevicesPort

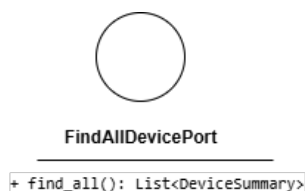


Figura 33: FindAllDevicesPort

Descrizione

FindAllDevicesPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero della lista sintetica di tutti i Dispositivi dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *GetDeviceListService*.

Attributi

FindAllDevicesPort non definisce attributi.

Metodi e funzioni

- + find_all(): List<DeviceSummary> — firma del metodo che recupera la lista sintetica di tutti i Dispositivi presenti nel sistema di persistenza.

5.3) Asset

5.3.1) CreateAsset

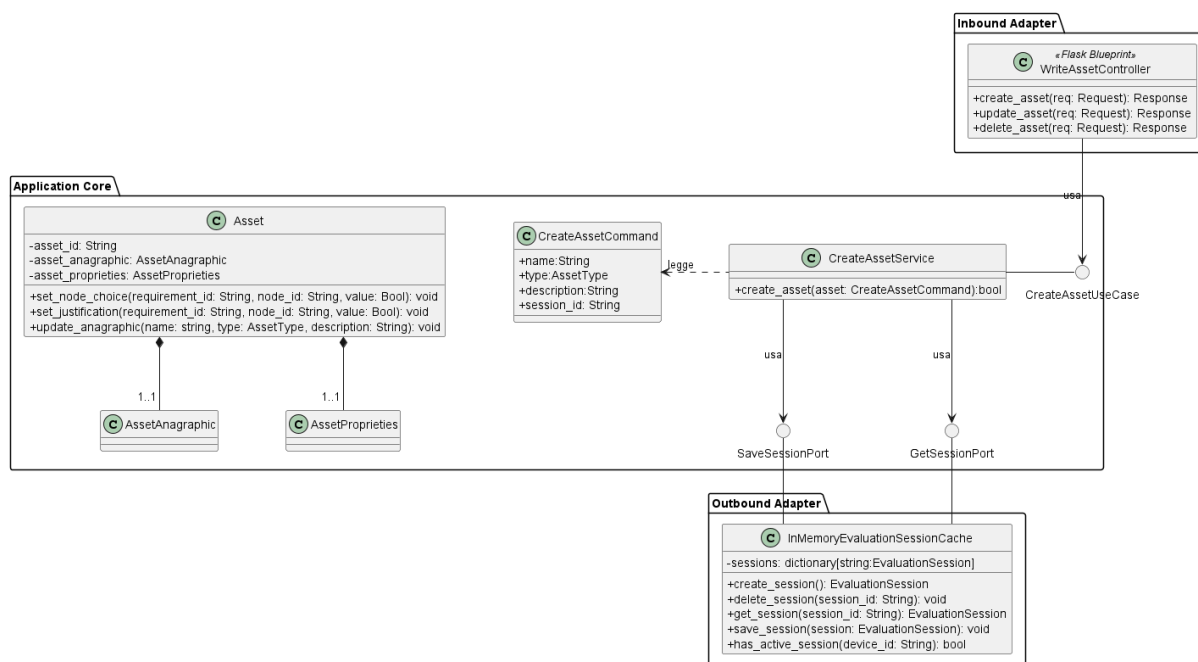


Figura 34: Caso d'uso CreateAsset

Il diagramma illustra l'architettura del modulo di creazione di un Asset all'interno di una sessione di valutazione attiva.

5.3.1.1) WriteAssetController



Figura 35: WriteAssetController

Descrizione

WriteAssetController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione (scrittura) degli Asset e le inoltra al livello applicativo.

Attributi

WriteAssetController non definisce attributi propri.

Metodi e funzioni

- `+ create_asset(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo Asset, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- `+ update_asset(req: Request): Response` — riceve la richiesta HTTP di aggiornamento di un Asset esistente, estrae i dati modificati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- `+ delete_asset(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un Asset, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.3.1.2) CreateAssetUseCase

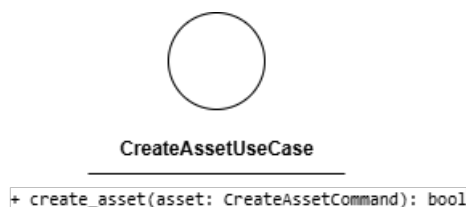


Figura 36: CreateAssetUseCase

Descrizione

CreateAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la creazione di un nuovo Asset all'interno della sessione di valutazione. Viene implementata da *CreateAssetService* e utilizzata da *WriteAssetController*.

Attributi

CreateAssetUseCase non definisce attributi.

Metodi e funzioni

- + `create_asset(asset: CreateAssetCommand): bool` — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.3.1.3) CreateAssetCommand

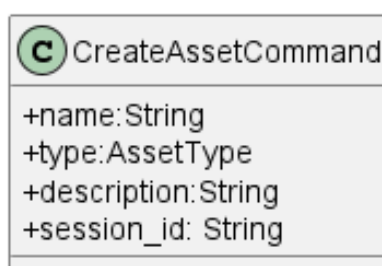


Figura 37: CreateAssetCommand

Descrizione

CreateAssetCommand è il Command Object che veicola i dati necessari alla creazione di un Asset dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- + `name: String` — nome del nuovo Asset.
- + `type: AssetType` — tipo dell'Asset.
- + `description: String` — descrizione testuale dell'Asset.
- + `session_id: String` — identificativo della sessione di valutazione attiva a cui l'Asset viene associato.

Metodi e funzioni

CreateAssetCommand non definisce metodi.

5.3.1.4) CreateAssetService

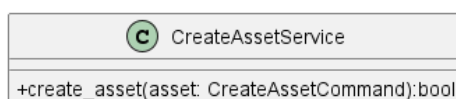


Figura 38: CreateAssetService

Descrizione

CreateAssetService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un Asset. Implementa l'interfaccia *CreateAssetUseCase*, recupera la sessione attiva tramite *GetSessionPort*, aggiunge il nuovo *Asset* e persiste la sessione aggiornata tramite *SaveSessionPort*.

Attributi

CreateAssetService non definisce attributi propri.

Metodi e funzioni

- + `create_asset(asset: CreateAssetCommand): bool` — concretizza il contratto definito da *CreateAssetUseCase*. Recupera la sessione attiva, vi aggiunge il nuovo Asset e ne persiste lo stato aggiornato.

5.3.1.5) Asset

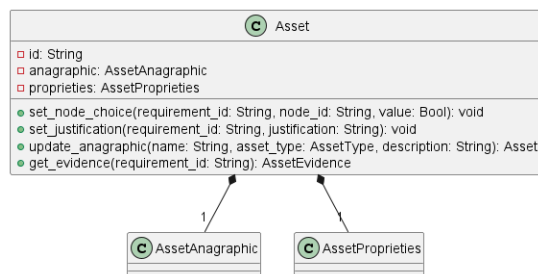


Figura 39: Asset

Descrizione

Asset è l'entità di dominio che rappresenta un asset oggetto di valutazione di conformità all'interno di una sessione.

Attributi

- - `asset_id: String` — identificativo univoco dell'Asset.
- - `asset_anagraphic: AssetAnagraphic` — oggetto che incapsula i dati anagrafici dell'asset.
- - `asset_proprieties: AssetProprieties` — oggetto che incapsula le proprietà e lo stato valutativo dell'asset.

Metodi e funzioni

- + `set_node_choice(requirement_id: String, node_id: String, value: Bool): void` — imposta o aggiorna la scelta (risposta) effettuata per un determinato nodo decisionale relativo a un requisito.
- + `set_justification(requirement_id: String, node_id: String, value: Bool): void` — imposta la giustificazione per un determinato nodo di un requisito.
- + `update_anagraphic(name: String, type: AssetType, description: String): void` — aggiorna le informazioni anagrafiche dell'asset (nome, tipologia e descrizione), delegando l'aggiornamento all'istanza interna di *AssetAnagraphic*.

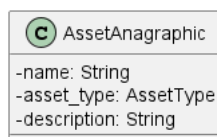


Figura 40: AssetAnagraphic

Descrizione

AssetAnagraphic è l'entità delegata alla gestione e all'incapsulamento delle informazioni anagrafiche e dei metadati di base di un generico asset.

Attributi

- - name: String — stringa di testo che rappresenta il nome identificativo dell'asset.
- - type: AssetType — attributo che definisce la tipologia o la categoria di appartenenza dell'asset.
- - description: String — stringa di testo destinata a contenere una descrizione estesa, note o dettagli aggiuntivi riguardanti le caratteristiche fisiche o logiche dell'asset.

Metodi e funzioni

AssetAnagraphic non definisce metodi.

5.3.1.6) AssetProprieties

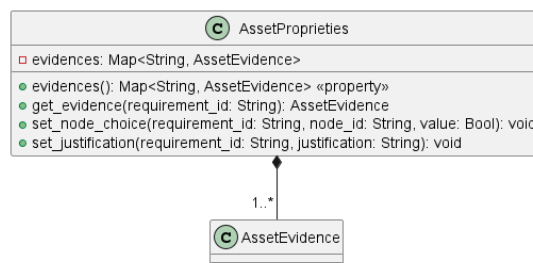


Figura 41: AssetProprieties

Descrizione

AssetProprieties è l'entità delegata alla gestione dello stato valutativo e delle proprietà specifiche di un asset. Presenta una relazione di composizione con la classe *AssetEvidence* con cardinalità 1..*, gestendone il ciclo di vita all'interno di una lista.

Attributi

- - asset_evidence_list: List<AssetEvidence> — struttura dati che incapsula e gestisce l'elenco delle evidenze (scelte e giustificazioni) associate all'asset.

Metodi e funzioni

- + `set_node_choice(requirement_id: String, node_id: String, value: Bool): void` — imposta o aggiorna la scelta (risposta) effettuata per un determinato nodo decisionale relativo a un requisito.
- + `set_justification(requirement_id: String, node_id: String, value: Bool): void` — imposta o aggiorna la giustificazione testuale per un determinato nodo di un requisito.
- + `get_evidence(requirement_id: String): AssetEvidence | void` — recupera l'oggetto *AssetEvidence* associato a un determinato identificativo di requisito. Restituisce l'evidenza se presente, altrimenti void (nessun valore/null).

5.3.1.7) InMemoryEvaluationSessionCache

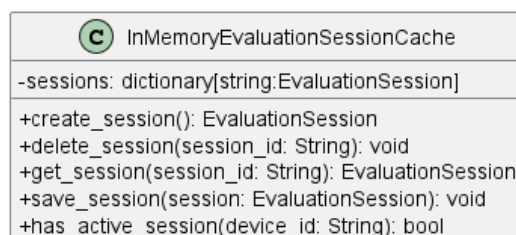


Figura 42: InMemoryEvaluationSessionCache

Descrizione

InMemoryEvaluationSessionCache è la classe dell'Outbound Adapter annotata come *Session Cache* che implementa entrambe le porte outbound *SaveSessionPort* e *GetSessionPort*. Gestisce la persistenza in memoria delle sessioni di valutazione tramite un dizionario indicizzato per `session_id`.

Attributi

- - `sessions: dictionary[string: EvaluationSession]` — struttura dati in memoria che mantiene le sessioni di valutazione attive, indicizzate per `session_id`.

Metodi e funzioni

- + `create_session(): EvaluationSession` — crea e registra una nuova sessione di valutazione.
- + `delete_session(session_id: String): void` — rimuove la sessione identificata da `session_id`.
- + `get_session(session_id: String): EvaluationSession` — recupera la sessione corrispondente all'identificativo fornito.
- + `save_session(session: EvaluationSession): void` — aggiorna la sessione nel dizionario in memoria.
- + `has_active_session(device_id: String): bool` — verifica se esiste una sessione attiva per il Dispositivo identificato da `device_id`.

5.3.1.8) SaveSessionPort



Figura 43: SaveSessionPort

Descrizione

SaveSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per il salvataggio della sessione di valutazione nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del modulo che modificano lo stato della sessione, tra cui *CreateAssetService*, *SaveAssetService* e *DeleteAssetService*.

Attributi

SaveSessionPort non definisce attributi.

Metodi e funzioni

- `+ save_session(session: EvaluationSession): void` — firma del metodo che persiste la sessione aggiornata nel sistema in memoria.

5.3.1.9) GetSessionPort

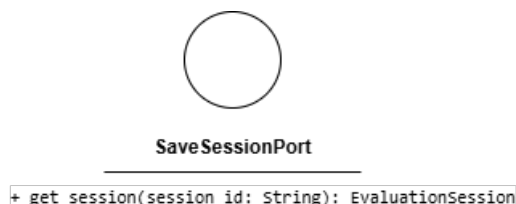


Figura 44: GetSessionPort

Descrizione

GetSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero della sessione di valutazione dal sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del modulo che necessitano di accedere alla sessione corrente, tra cui *CreateAssetService*, *SaveAssetService*, *DeleteAssetService* e *GetAssetDetailService*.

Attributi

GetSessionPort non definisce attributi.

Metodi e funzioni

- + get_session(session_id: String): EvaluationSession — firma del metodo che recupera la sessione di valutazione attiva corrispondente all'identificativo fornito.

5.3.2) UpdateAsset

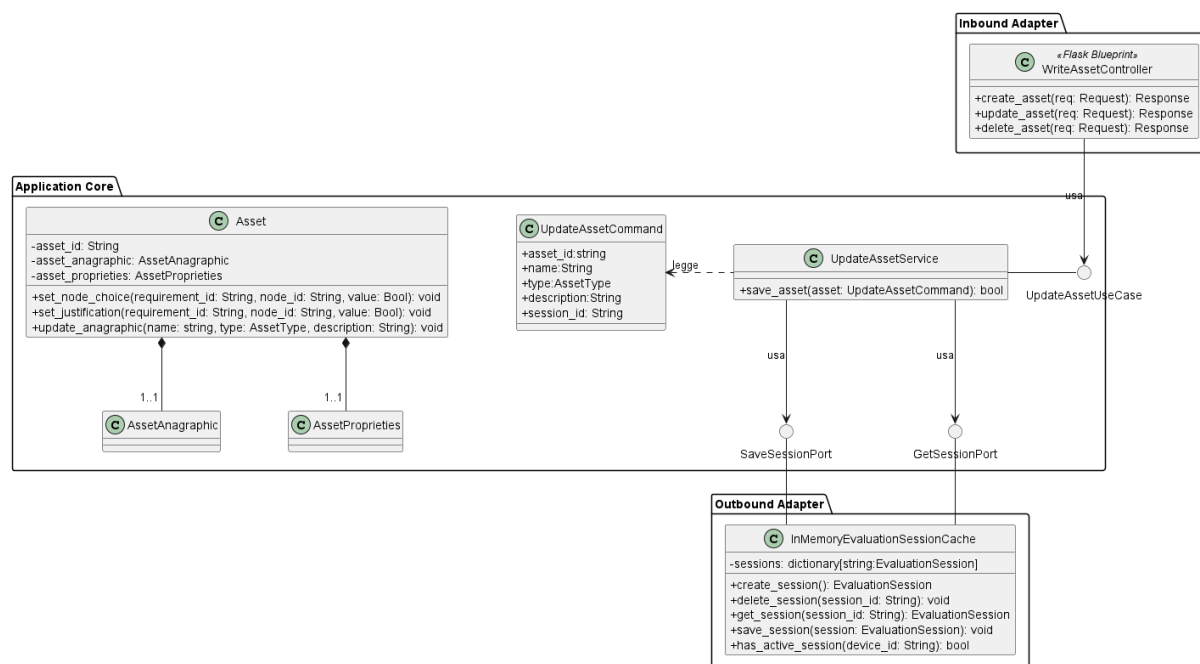


Figura 45: Caso d'uso UpdateAsset

Il diagramma illustra l'architettura del modulo di modifica di un Asset esistente all'interno di una sessione di valutazione attiva.

- Per la definizione di *WriteAssetController*, vedere la sezione **Sezione 5.3.1.1**.
- Per la definizione di *Asset*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *SaveSessionPort*, vedere la sezione **Sezione 5.3.1.8**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.2.1) UpdateAssetUseCase

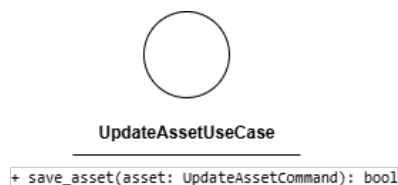


Figura 46: UpdateAssetUseCase

Descrizione

UpdateAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la modifica di un Asset esistente all'interno della sessione di valutazione. Viene implementata da *UpdateAssetService* e utilizzata da *WriteAssetController*.

Attributi

UpdateAssetUseCase non definisce attributi.

Metodi e funzioni

- + save_asset(asset: UpdateAssetCommand): bool — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.3.2.2) UpdateAssetCommand

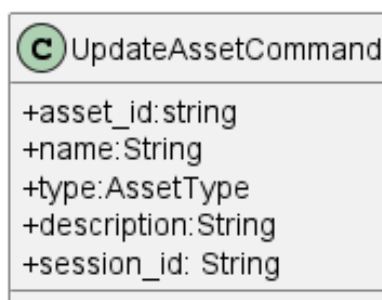


Figura 47: UpdateAssetCommand

Descrizione

UpdateAssetCommand è il Command Object che veicola i dati necessari alla modifica di un Asset dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- + asset_id: String — identificativo univoco dell'Asset da modificare.
- + name: String — nome aggiornato dell'Asset.
- + type: AssetType — tipo aggiornato dell'Asset.
- + description: String — descrizione testuale aggiornata dell'Asset.
- + session_id: String — identificativo della sessione di valutazione attiva.

Metodi e funzioni

UpdateAssetCommand non definisce metodi.

5.3.2.3) UpdateAssetService

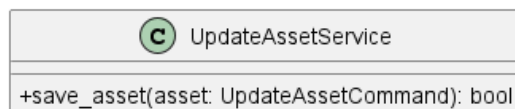


Figura 48: UpdateAssetService

Descrizione

UpdateAssetService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un Asset esistente. Implementa l'interfaccia *UpdateAssetUseCase*, recupera la sessione attiva tramite *GetSessionPort*, aggiorna l'Asset corrispondente e persiste la sessione modificata tramite *SaveSessionPort*.

Attributi

UpdateAssetService non definisce attributi propri.

Metodi e funzioni

- + *save_asset(asset: UpdateAssetCommand): bool* — concretizza il contratto definito da *UpdateAssetUseCase*. Recupera la sessione attiva, individua l'Asset da aggiornare tramite *asset_id* e ne persiste lo stato modificato.

5.3.3) DeleteAsset

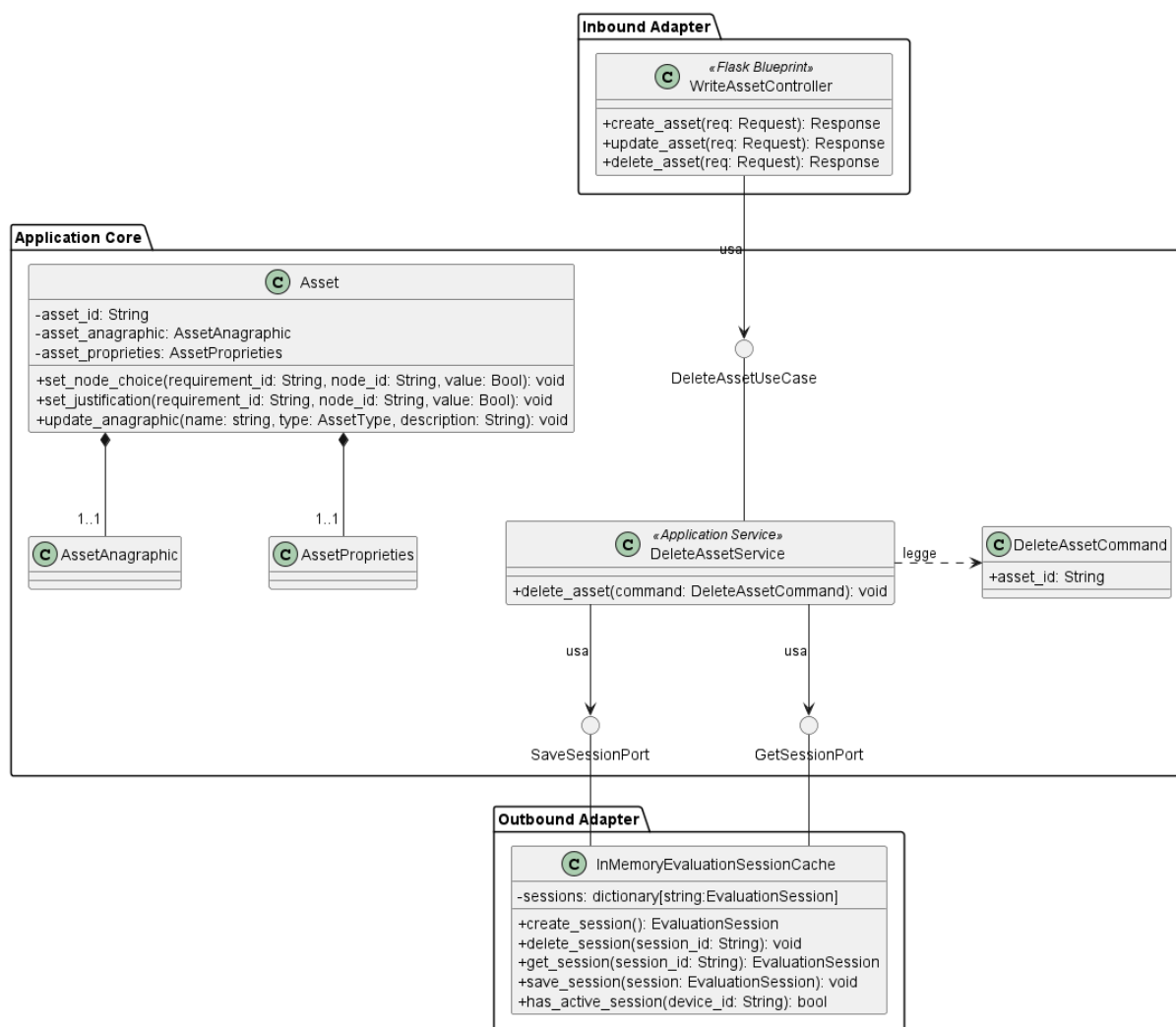


Figura 49: Caso d'uso DeleteAsset

Il diagramma illustra l'architettura del modulo di eliminazione di un Asset esistente all'interno di una sessione di valutazione attiva.

- Per la definizione di *WriteAssetController*, vedere la sezione **Sezione 5.3.1.1**.
- Per la definizione di *Asset*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *SaveSessionPort*, vedere la sezione **Sezione 5.3.1.8**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.3.1) DeleteAssetCommand

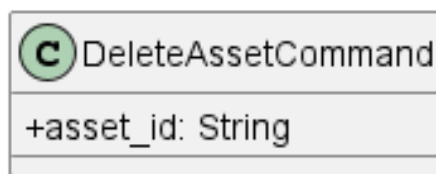


Figura 50: DeleteAssetCommand

Descrizione

DeleteAssetCommand è il Command Object utilizzato per trasportare i dati necessari all'eliminazione di un Asset dalla sessione di valutazione. Incapsula i parametri di input del metodo esposto da *DeleteAssetUseCase*.

Attributi

- + `Asset_id: String` — identificativo univoco del Asset da eliminare.

Metodi e funzioni

DeleteAssetCommand non definisce metodi propri.

5.3.3.2) DeleteAssetUseCase



Figura 51: DeleteAssetUseCase

Descrizione

DeleteAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'eliminazione di un Asset dalla sessione di valutazione. Viene implementata da *DeleteAssetService* e utilizzata da *WriteAssetController*.

Attributi

DeleteAssetUseCase non definisce attributi.

Metodi e funzioni

- + `delete_asset(command: DeleteAssetCommand): void` — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dai dati contenuti nel Command.

5.3.3.3) DeleteAssetService

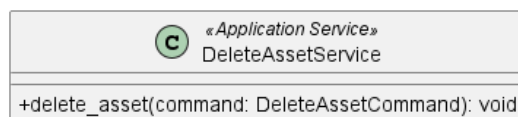


Figura 52: DeleteAssetService

Descrizione

DeleteAssetService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Asset. Implementa l'interfaccia *DeleteAssetUseCase*, riceve un *DeleteAssetCommand* contenente gli identificativi necessari, recupera la sessione tramite *GetSessionPort*, rimuove l'Asset corrispondente e persiste la sessione aggiornata tramite *SaveSessionPort*.

Attributi

DeleteAssetService non definisce attributi propri.

Metodi e funzioni

- `+ delete_asset(command: DeleteAssetCommand): void` — concretizza il contratto definito da *DeleteAssetUseCase*. Recupera la sessione attiva, individua e rimuove l'Asset corrispondente e ne persiste lo stato aggiornato.

5.3.4) GetAssetDetail

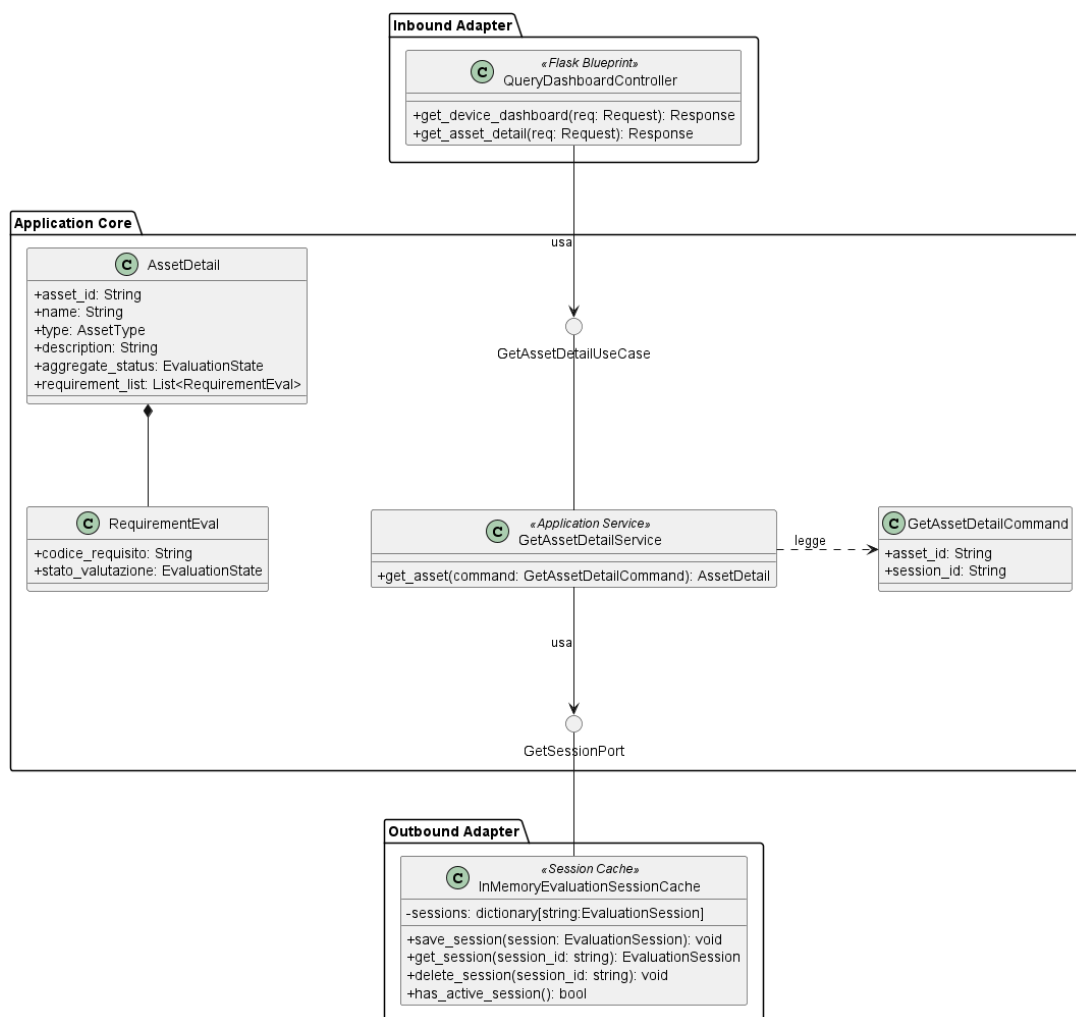


Figura 53: Caso d'uso GetAssetDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un Asset all'interno di una sessione di valutazione attiva.

- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.4.1) QueryDashboardController



Figura 54: QueryDashboardController

Descrizione

QueryDashboardController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di lettura relative alla dashboard e agli Asset. Delega la logica applicativa al livello sottostante tramite le rispettive interfacce.

Attributi

QueryDashboardController non definisce attributi propri.

Metodi e funzioni

- + `get_device_dashboard(req: Request): Response` — riceve la richiesta HTTP di recupero della dashboard del Dispositivo e restituisce una risposta HTTP con i dati aggregati.
- + `get_asset_detail(req: Request): Response` — riceve la richiesta HTTP di recupero del dettaglio di un Asset specifico e restituisce una risposta HTTP con i dati completi.

5.3.4.2) GetAssetDetailCommand

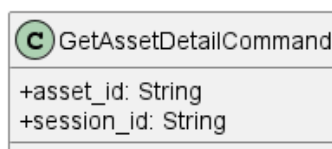


Figura 55: GetAssetDetailCommand

Descrizione

GetAssetDetailCommand è il Command Object utilizzato per trasportare i dati necessari al recupero del dettaglio di un Asset. Incapsula i parametri di input del metodo esposto da *GetAssetDetailUseCase*.

Attributi

- + `asset_id: String` — identificativo univoco dell'Asset di cui recuperare il dettaglio.
- + `session_id: String` — identificativo univoco della sessione di valutazione corrente.

Metodi e funzioni

GetAssetDetailCommand non definisce metodi propri.

5.3.4.3) GetAssetDetailUseCase



Figura 56: GetAssetDetailUseCase

Descrizione

GetAssetDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Asset. Viene implementata da *GetAssetDetailService* e utilizzata da *QueryDashboardController*.

Attributi

GetAssetDetailUseCase non definisce attributi.

Metodi e funzioni

- `+ get_asset(command: GetAssetDetailCommand): AssetDetail` — firma del metodo delegato al recupero del dettaglio dell'Asset corrispondente ai dati contenuti nel Command.

5.3.4.4) GetAssetDetailService



Figura 57: GetAssetDetailService

Descrizione

GetAssetDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Asset. Implementa l'interfaccia *GetAssetDetailUseCase*, recupera la sessione attiva tramite *GetSessionPort* e costruisce il *AssetDetail* aggregando le informazioni dell'Asset con il suo stato di valutazione.

Attributi

GetAssetDetailService non definisce attributi propri.

Metodi e funzioni

- `+ get_asset(command: GetAssetDetailCommand): AssetDetail` — concretizza il contratto definito da *GetAssetDetailUseCase*. Recupera la sessione attiva, individua l'Asset richiesto e ne costruisce la rappresentazione *AssetDetail*.

5.3.4.5) AssetDetail

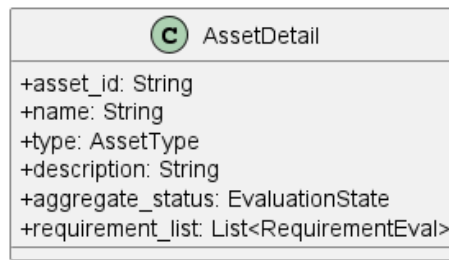


Figura 58: AssetDetail

Descrizione

AssetDetail è il Data Transfer Object che veicola la rappresentazione completa di un Asset verso il livello di presentazione. Aggrega le informazioni dell'Asset con il suo stato di valutazione e la lista dei requisiti valutati, evitando di esporre direttamente l'entità di dominio.

Attributi

- + `asset_id: String` — identificativo univoco dell'Asset.
- + `name: String` — nome dell'Asset.
- + `type: AssetType` — tipo dell'Asset.
- + `description: String` — descrizione testuale dell'Asset.
- + `aggregate_status: EvaluationState` — stato di valutazione aggregato dell'Asset.
- + `requirement_list: List<RequirementEval>` — lista dei requisiti valutati associati all'Asset.

Metodi e funzioni

AssetDetail non definisce metodi.

5.3.4.6) RequirementEval

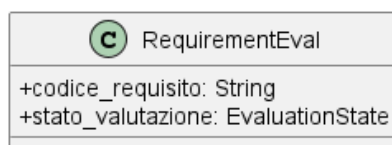


Figura 59: RequirementEval

Descrizione

RequirementEval è il Data Transfer Object che rappresenta la valutazione di un singolo requisito di conformità associato all'Asset. È contenuto nella lista *requirement_list* di *AssetDetail*.

Attributi

- + *codice_requisito*: String — codice identificativo del requisito di conformità.
- + *stato_valutazione*: EvaluationState — stato di valutazione del requisito.

Metodi e funzioni

RequirementEval non definisce metodi.

5.4) Import

5.4.1) ImportDevice

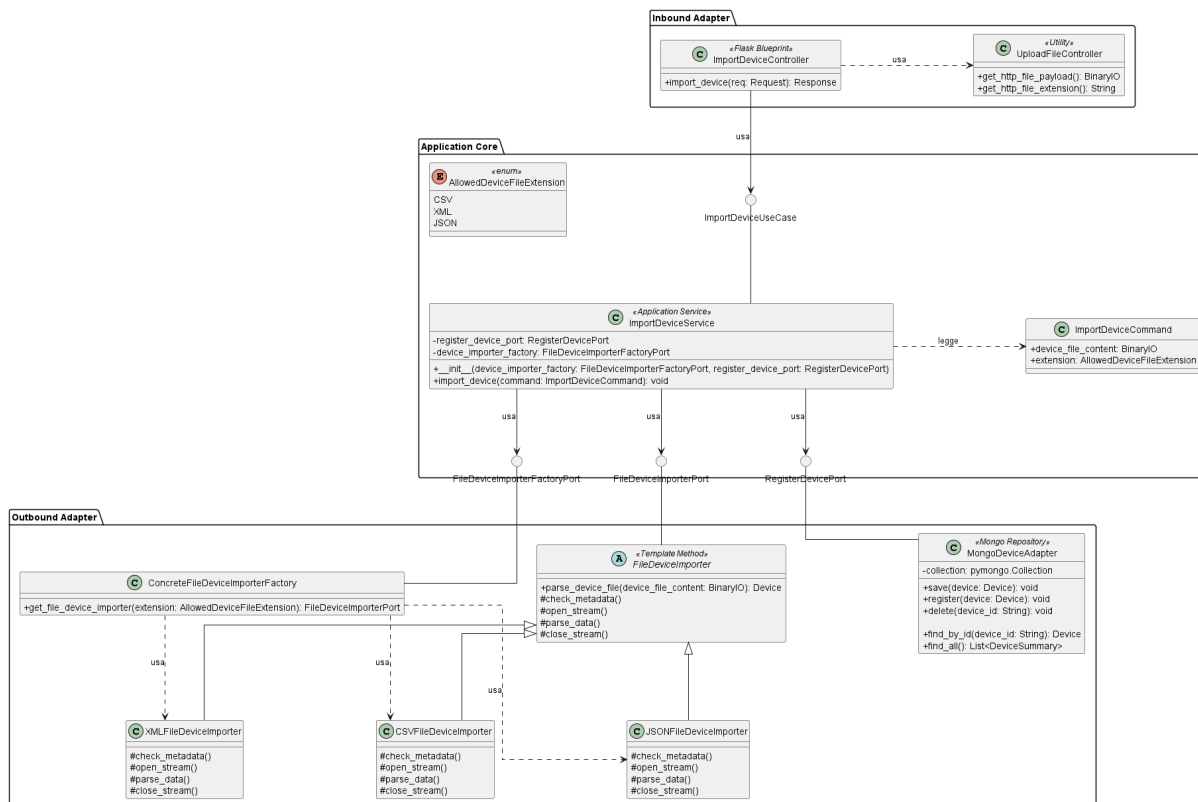


Figura 60: Caso d'uso ImportDevice

Il diagramma illustra l'architettura del modulo dedicato all'importazione di Dispositivi tramite file. Il modulo supporta tre formati — CSV, XML e JSON — gestiti tramite il pattern *Template Method* e una factory dedicata. Il componente *MongoDeviceAdapter* è già descritto nella sezione *CreateDevice*.

- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.
- Per la definizione di *RegisterDevicePort*, vedere la sezione **Sezione 5.2.2.6**

Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso.

5.4.1.1) UploadFileController



Figura 61: UploadFileController

Descrizione

UploadFileController è una classe di utilità appartenente all'Inbound Adapter che fornisce i metodi comuni per l'estrazione del contenuto e dell'estensione di un file dalla richiesta HTTP. Viene estesa da *ImportDeviceController*.

Attributi

UploadFileController non definisce attributi propri.

Metodi e funzioni

- *+ get_http_file_payload(): BinaryIO* — estrae il contenuto binario del file dalla richiesta HTTP.
- *+ get_http_file_extension(): String* — estrae l'estensione del file dalla richiesta HTTP.

5.4.1.2) ImportDeviceController



Figura 62: ImportDeviceController

Descrizione

ImportDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di importazione di Dispositivi tramite file. Utilizza *UploadFileController* per estrarre il contenuto binario e l'estensione del file dalla request.

Attributi

ImportDeviceController non definisce attributi propri.

Metodi e funzioni

- + `import_device(req: Request): Response` — riceve la richiesta HTTP di importazione, estrae il file e la sua estensione tramite *UploadFileController* e inoltra il Command al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.4.1.3) UploadFileController

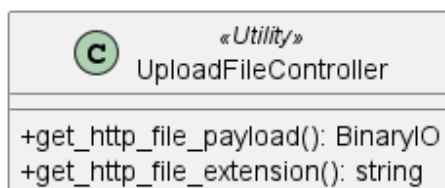


Figura 63: UploadFileController

Descrizione

UploadFileController è una classe di utilità appartenente all'Inbound Adapter che fornisce metodi di supporto per l'estrazione del file dalla request HTTP. Viene utilizzata da *ImportDeviceController*.

Attributi

UploadFileController non definisce attributi propri.

Metodi e funzioni

- + `get_http_file_payload(): BinaryIO` — estrae e restituisce il contenuto binario del file dalla request HTTP.
- + `get_http_file_extension(): String` — estrae e restituisce l'estensione del file dalla request HTTP.

5.4.1.4) ImportDeviceUseCase

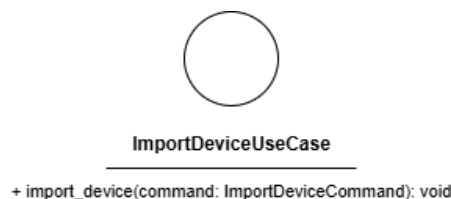


Figura 64: ImportDeviceUseCase

Descrizione

ImportDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'importazione di Dispositivi da file. Viene implementata da *ImportDeviceService* e utilizzata da *ImportDeviceController*.

Attributi

ImportDeviceUseCase non definisce attributi.

Metodi e funzioni

- + import_device(command: ImportDeviceCommand): void — firma del metodo delegato all'esecuzione della logica di importazione a partire dai dati contenuti nel Command.

5.4.1.5) ImportDeviceCommand

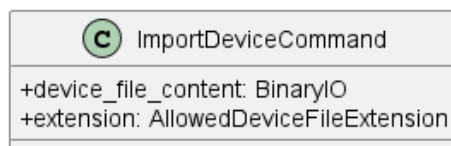


Figura 65: ImportDeviceCommand

Descrizione

ImportDeviceCommand è il Command Object che veicola i dati necessari all'importazione di Dispositivi dal controller al service.

Attributi

- + device_file_content: BinaryIO — contenuto binario del file da importare.
- + extension: AllowedDeviceFileExtension — estensione del file, vincolata ai valori dell'enumerazione *AllowedDeviceFileExtension*.

Metodi e funzioni

ImportDeviceCommand non definisce metodi propri.

5.4.1.6) AllowedDeviceFileExtension

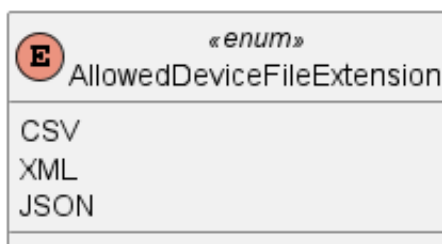


Figura 66: AllowedDeviceFileExtension

Descrizione

AllowedDeviceFileExtension è un'enumerazione che definisce i formati di file supportati per l'importazione di Dispositivi.

Valori

- CSV — formato CSV.
- XML — formato XML.
- JSON — formato JSON.

5.4.1.7) ImportDeviceService

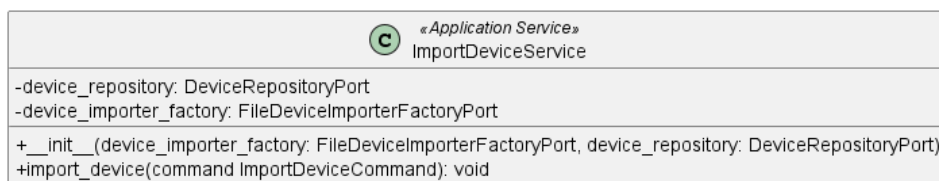


Figura 67: ImportDeviceService

Descrizione

ImportDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di importazione di Dispositivi da file. Implementa l'interfaccia *ImportDeviceUseCase* e coordina il parsing del file tramite *FileDeviceImporterFactoryPort*, la lettura del contenuto tramite *FileDeviceImporterPort* e la persistenza tramite *RegisterDevicePort*.

Attributi

- - `device_repository: DeviceRepositoryPort` — porta outbound per la persistenza dei Dispositivi importati.
- - `device_importer_factory: FileDeviceImporterFactoryPort` — porta outbound per l'ottenimento dell'importer appropriato in base al formato del file.

Metodi e funzioni

- + `__init__(device_importer_factory: FileDeviceImporterFactoryPort, device_repository: DeviceRepositoryPort)` — inializza il service con le dipendenze necessarie.
- + `import_device(command: ImportDeviceCommand): void` — concretizza il contratto definito da *ImportDeviceUseCase*. Ottiene l'importer appropriato tramite la factory, effettua il parsing del file e persiste i Dispositivi estratti tramite *DeviceRepositoryPort*.

5.4.1.8) FileDeviceImporterPort



Figura 68: FileDeviceImporterPort

Descrizione

FileDeviceImporterPort è l'interfaccia (Outbound Port) che definisce il contratto per il parsing di un file contenente dati di Dispositivi. Viene implementata dalle classi concrete *XMLFileDeviceImporter*, *JSONFileDeviceImporter* e *CSVFileDeviceImporter* tramite *FileDeviceImporter*.

Attributi

FileDeviceImporterPort non definisce attributi.

Metodi e funzioni

- + `parse_device_file(device_file_content: BinaryIO): Device` — firma del metodo che effettua il parsing del contenuto binario del file e restituisce l'entità *Device* estratta.

5.4.1.9) FileDeviceImporterFactoryPort

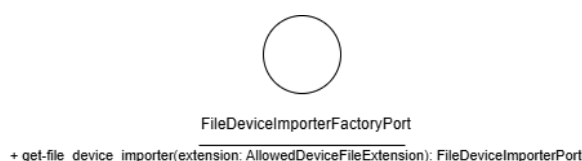


Figura 69: FileDeviceImporterFactoryPort

Descrizione

FileDeviceImporterFactoryPort è l'interfaccia (Outbound Port) che definisce il contratto per l'ottenimento dell'importer appropriato in base all'estensione del file. Viene implementata da *ConcreteFileDeviceImporterFactory*.

Attributi

FileDeviceImporterFactoryPort non definisce attributi.

Metodi e funzioni

- + `get_file_device_importer(extension: AllowedDeviceFileExtension): FileDeviceImporterPort` — restituisce l'istanza dell'importer appropriato per il formato specificato.

5.4.1.10) FileDeviceImporter

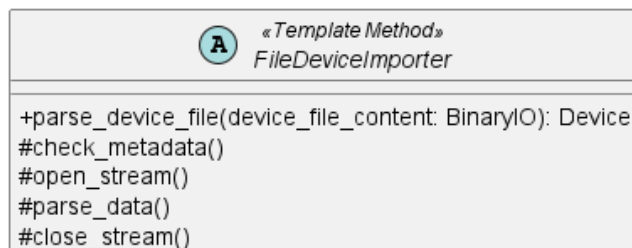


Figura 70: FileDeviceImporter

Descrizione

FileDeviceImporter è la classe astratta appartenente all'Outbound Adapter che implementa il pattern *Template Method* per il parsing dei file di Dispositivi. Definisce lo scheletro dell'algoritmo di importazione, delegando alle sottoclassi concrete l'implementazione dei passi specifici per ciascun formato.

Attributi

FileDeviceImporter non definisce attributi propri.

Metodi e funzioni

- + `parse_device_file(device_file_content: BinaryIO): Device` — metodo pubblico che orchestra l'algoritmo di parsing invocando in sequenza i metodi del template.
- # `check_metadata()` — metodo protetto che verifica i metadati del file.
- # `open_stream()` — metodo protetto che apre lo stream di lettura del file.
- # `parse_data()` — metodo protetto che effettua il parsing dei dati.
- # `close_stream()` — metodo protetto che chiude lo stream di lettura.

5.4.1.11) XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter

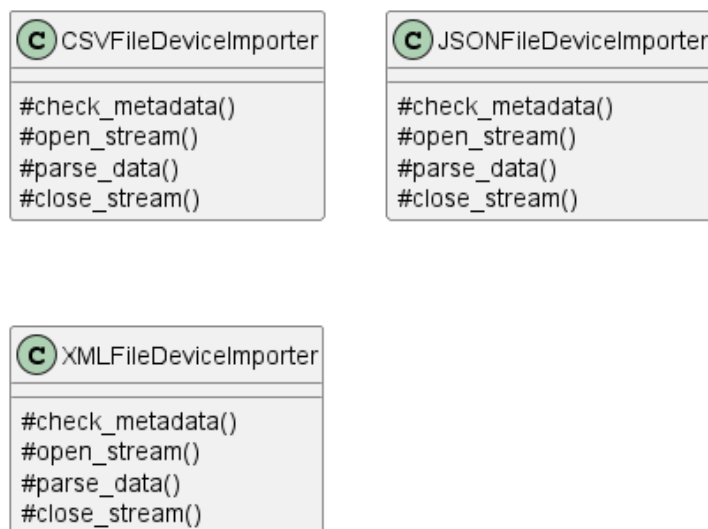


Figura 71: XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter

Descrizione

XMLFileDeviceImporter, *JSONFileDeviceImporter* e *CSVFileDeviceImporter* sono le implementazioni concrete di *FileDeviceImporter*, ciascuna specializzata nel parsing del rispettivo formato di file. Implementano i metodi protetti del template per la gestione dello stream e del parsing specifico del formato.

Attributi

Le tre classi non definiscono attributi propri.

Metodi e funzioni

Ciascuna classe implementa i metodi ereditati da *FileDeviceImporter*:

- # `check_metadata()` — verifica i metadati specifici del formato.
- # `open_stream()` — apre lo stream nel formato appropriato.
- # `parse_data()` — effettua il parsing dei dati nel formato specifico.
- # `close_stream()` — chiude lo stream di lettura.

5.4.1.12) ConcreteFileDeviceImporterFactory



Figura 72: ConcreteFileDeviceImporterFactory

Descrizione

ConcreteFileDeviceImporterFactory è la classe dell'Outbound Adapter che implementa *FileDeviceImporterFactoryPort*. Istanza e restituisce l'importer appropriato in base

all'estensione del file fornita, selezionando tra *XMLFileDeviceImporter*, *JSONFileDeviceImporter* e *CSVFileDeviceImporter*.

Attributi

ConcreteFileDeviceImporterFactory non definisce attributi propri.

Metodi e funzioni

- + `get_file_device_importer(extension: AllowedDeviceFileExtension): FileDeviceImporterPort` — restituisce l'istanza dell'importer corrispondente al formato specificato.

5.4.2) ImportStandard

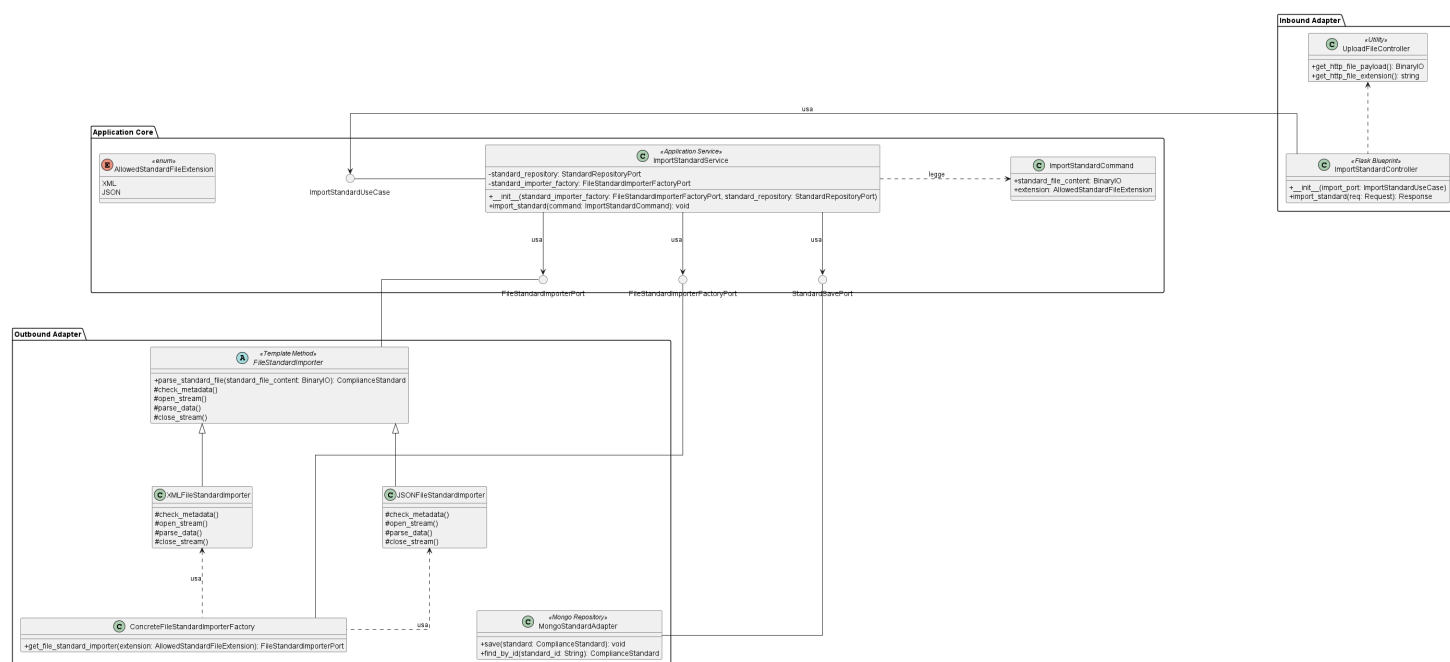


Figura 73: ImportStandard

Il diagramma illustra l'architettura del modulo dedicato all'importazione di standard di conformità tramite file. La struttura ricalca il pattern già adottato per *ImportDevice* (si veda **Figura 60**), con le dovute specializzazioni per la gestione degli standard.

- Per la definizione di *UploadFileController*, vedere la sezione **Sezione 5.4.1.1**.
- Per la definizione di *MongoStandardAdapter*, vedere la sezione **Sezione 5.6.1.8**. Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso.

5.4.2.1) ImportStandardController



Figura 74: ImportStandardController

Descrizione

ImportStandardController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di importazione di standard di conformità tramite file. Estende *UploadFileController* per la gestione del payload e delega la logica applicativa tramite *ImportStandardUseCase*.

Attributi

ImportStandardController non definisce attributi.

Metodi e funzioni

- + `__init__(import_port: ImportStandardUseCase)` — inializza il controller con la dipendenza verso il caso d’uso.
- + `import_standard(req: Request): Response` — riceve la richiesta HTTP di importazione, estrae il file e la sua estensione tramite i metodi ereditati da *UploadFileController* e inoltra il comando al livello applicativo; restituisce una risposta HTTP con l’esito dell’operazione.

5.4.2.2) ImportStandardUseCase

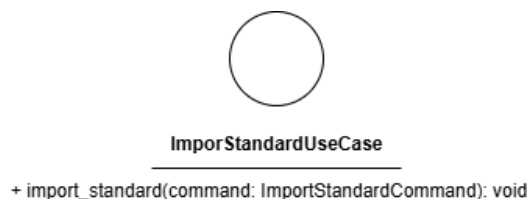


Figura 75: ImportStandardUseCase

Descrizione

ImportStandardUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'importazione di standard di conformità da file. Viene implementata da *ImportStandardService* e utilizzata da *ImportStandardController*.

Attributi

ImportStandardUseCase non definisce attributi.

Metodi e funzioni

- `+ import_standard(command: ImportStandardCommand): void` — firma del metodo delegato all'esecuzione della logica di importazione a partire dai dati contenuti nel comando.

5.4.2.3) ImportStandardCommand



Figura 76: ImportStandardCommand

Descrizione

ImportStandardCommand è il Command Object che veicola i dati necessari all'importazione di uno standard di conformità dal controller al service.

Attributi

- `+ standard_file_content: BinaryIO` — contenuto binario del file da importare.
- `+ extension: AllowedStandardFileExtension` — estensione del file, vincolata ai valori dell'enumerazione *AllowedStandardFileExtension*.

Metodi e funzioni

ImportStandardCommand non definisce metodi.

5.4.2.4) AllowedStandardFileExtension



Figura 77: AllowedStandardFileExtension

Descrizione

AllowedStandardFileExtension è un'enumerazione che definisce i formati di file supportati per l'importazione di standard di conformità.

Valori

- XML — formato XML.
- JSON — formato JSON.

5.4.2.5) ImportStandardService

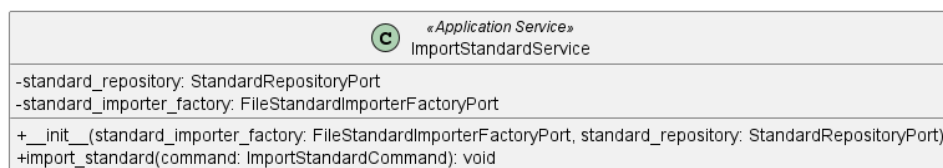


Figura 78: ImportStandardService

Descrizione

ImportStandardService è il service applicativo appartenente all'Application Core responsabile della logica di importazione di standard di conformità da file. Implementa l'interfaccia *ImportStandardUseCase* e coordina il parsing del file tramite *FileStandardImporterFactoryPort*, la lettura del contenuto tramite *FileStandardImporterPort* e la persistenza tramite *StandardSavePort*.

Attributi

- - *device_repository*: *StandardRepositoryPort* — porta outbound per la persistenza degli standard importati.
- - *device_importer_factory*: *FileStandardImporterFactoryPort* — porta outbound per l'ottenimento dell'importer appropriato in base al formato del file.

Metodi e funzioni

- + *__init__*(*device_importer_factory*: *FileStandardImporterFactoryPort*, *device_repository*: *StandardRepositoryPort*) — inizializza il service con le dipendenze necessarie.

- + `import_standard(command: ImportStandardCommand): void` — concretizza il contratto definito da *ImportStandardUseCase*. Ottiene l'importer appropriato tramite la factory, effettua il parsing del file e persiste lo standard estratto tramite *StandardSavePort*.

5.4.2.6) FileStandardImporterPort



Figura 79: FileStandardImporterPort

Descrizione

FileStandardImporterPort è l'interfaccia (Outbound Port) che definisce il contratto per il parsing di un file contenente dati di standard di conformità. Viene implementata dalle classi concrete *XMLFileStandardImporter* e *JSONFileStandardImporter* tramite *FileStandardImporter*.

Attributi

FileStandardImporterPort non definisce attributi.

Metodi e funzioni

- + `parse_standard_file(standard_file_content: BinaryIO): ComplianceStandard` — firma del metodo che effettua il parsing del contenuto binario del file e restituisce l'entità *ComplianceStandard* estratta.

5.4.2.7) StandardSavePort



Figura 80: StandardSavePort

Descrizione

StandardSavePort è l'interfaccia (Outbound Port) che definisce il contratto per la persistenza di uno standard di conformità nel database. Viene implementata da *MongoStandardAdapter* e utilizzata da *ImportStandardService*.

Attributi

StandardSavePort non definisce attributi.

Metodi e funzioni

- + `save(standard: ComplianceStandard): void` — firma del metodo che persiste lo standard di conformità nel database.

5.4.2.8) FileStandardImporterFactoryPort

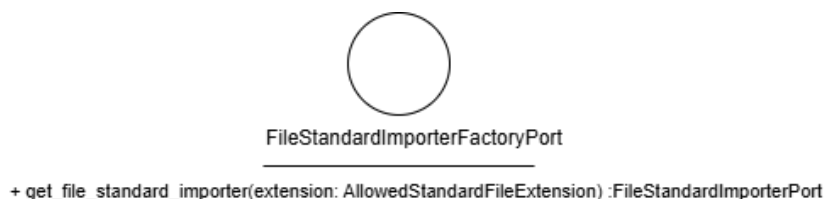


Figura 81: FileStandardImporterFactoryPort

Descrizione

FileStandardImporterFactoryPort è l'interfaccia (Outbound Port) che definisce il contratto per l'ottenimento dell'importer appropriato in base all'estensione del file. Viene implementata da *ConcreteFileStandardImporterFactory*.

Attributi

FileStandardImporterFactoryPort non definisce attributi.

Metodi e funzioni

- + `get_file_standard_importer(extension: AllowedStandardFileExtension): FileStandardImporterPort` — restituisce l'istanza dell'importer appropriato per il formato specificato.

5.4.2.9) FileStandardImporter

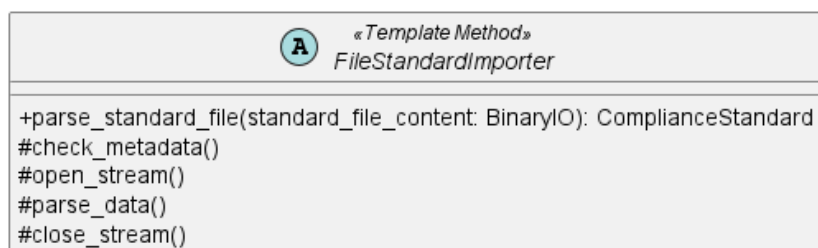


Figura 82: FileStandardImporter

Descrizione

FileStandardImporter è la classe astratta appartenente all'Outbound Adapter che implementa il pattern *Template Method* per il parsing dei file di standard di conformità. Definisce lo scheletro dell'algoritmo di importazione, delegando alle sottoclassi concrete l'implementazione dei passi specifici per ciascun formato.

Attributi

FileStandardImporter non definisce attributi propri.

Metodi e funzioni

- + `parse_standard_file(standard_file_content: BinaryIO): ComplianceStandard` — metodo pubblico che orchestra l'algoritmo di parsing invocando in sequenza i metodi del template.
- # `check_metadata()` — verifica i metadati del file.
- # `open_stream()` — apre lo stream di lettura del file.
- # `parse_data()` — effettua il parsing dei dati.
- # `close_stream()` — chiude lo stream di lettura.

5.4.2.10) JSONFileStandardImporter e XMLFileStandardImporter

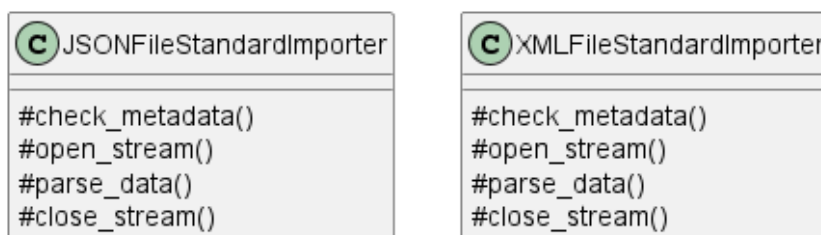


Figura 83: JSONFileStandardImporter e XMLFileStandardImporter

Descrizione

JSONFileStandardImporter e *XMLFileStandardImporter* sono le implementazioni concrete di *FileStandardImporter*, ciascuna specializzata nel parsing del rispettivo formato di file. Implementano i metodi protetti del template per la gestione dello stream e del parsing specifico del formato.

Attributi

Le due classi non definiscono attributi propri.

Metodi e funzioni

Ciascuna classe implementa i metodi ereditati da *FileStandardImporter*:

- # `check_metadata()` — verifica i metadati specifici del formato.
- # `open_stream()` — apre lo stream nel formato appropriato.
- # `parse_data()` — effettua il parsing dei dati nel formato specifico.
- # `close_stream()` — chiude lo stream di lettura.

5.4.2.11) ConcreteFileStandardImporterFactory



Figura 84: ConcreteFileStandardImporterFactory

Descrizione

ConcreteFileStandardImporterFactory è la classe dell'Outbound Adapter che implementa *FileStandardImporterFactoryPort*. Istanza e restituisce l'importer appropriato in base all'estensione del file fornita, selezionando tra *JSONFileStandardImporter* e *XMLFileStandardImporter*.

Attributi

ConcreteFileStandardImporterFactory non definisce attributi propri.

Metodi e funzioni

- `+ get_file_standard_importer(extension: AllowedStandardFileExtension): FileStandardImporterPort` — restituisce l'istanza dell'importer corrispondente al formato specificato.

5.5) Dashboard

5.5.1) GetDeviceDashboard

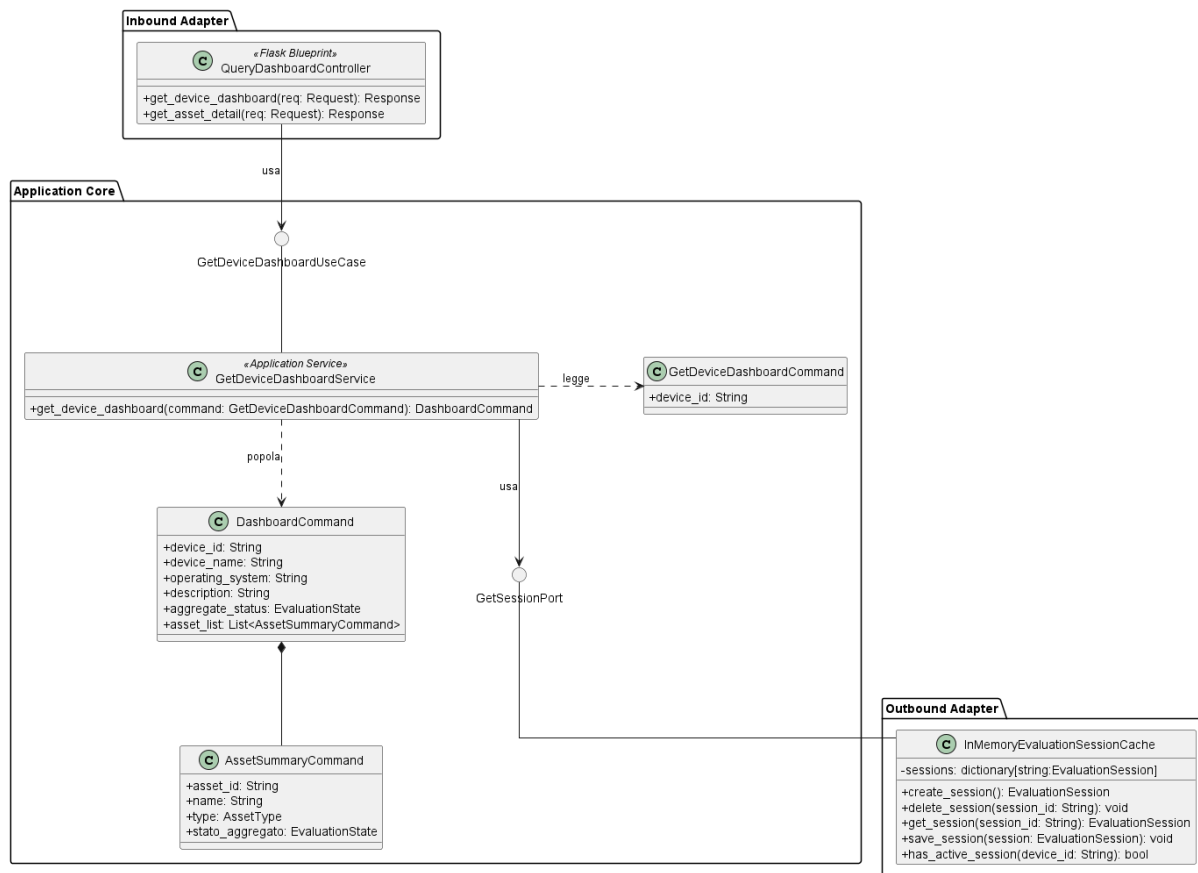


Figura 85: Caso d'uso GetDeviceDashboard

Il diagramma illustra l'architettura del modulo dedicato al recupero della dashboard di un Dispositivo, che aggrega le informazioni della sessione di valutazione attiva in una vista sintetica.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.
- Per la definizione di *QueryDashboardController*, vedere la sezione **Sezione 5.3.4.1**.

5.5.1.1) GetDeviceDashboardCommand

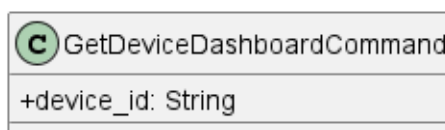


Figura 86: GetDeviceDashboardCommand

Descrizione

GetDeviceDashboardCommand è il Command Object utilizzato per trasportare i dati necessari al recupero della dashboard di un Dispositivo. Incapsula i parametri di input del metodo esposto da *GetDeviceDashboardUseCase*.

Attributi

- + `device_id`: String — identificativo univoco del Dispositivo di cui recuperare la dashboard.

Metodi e funzioni

GetDeviceDashboardCommand non definisce metodi propri.

5.5.1.2) GetDeviceDashboardUseCase



Figura 87: GetDeviceDashboardUseCase

Descrizione

GetDeviceDashboardUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero della dashboard di un Dispositivo. Viene implementata da *GetDeviceDashboardService* e utilizzata da *QueryDashboardController*.

Attributi

GetDeviceDashboardUseCase non definisce attributi.

Metodi e funzioni

- + `get_device_dashboard(command: GetDeviceDashboardCommand): DashboardCommand` — firma del metodo delegato al recupero e all'aggregazione delle informazioni della sessione di valutazione in una vista dashboard.

5.5.1.3) GetDeviceDashboardService



Figura 88: GetDeviceDashboardService

Descrizione

GetDeviceDashboardService è il service applicativo appartenente all'Application Core responsabile del recupero della dashboard di un Dispositivo. Implementa l'interfaccia *GetDeviceDashboardUseCase*, recupera la sessione attiva tramite *GetSessionPort* e costruisce il *DashboardCommand* con le informazioni aggregate.

Attributi

GetDeviceDashboardService non definisce attributi propri.

Metodi e funzioni

- + `get_device_dashboard(command: GetDeviceDashboardCommand): DashboardCommand` — concretizza il contratto definito da *GetDeviceDashboardUseCase*. Recupera la sessione attiva, aggrega le informazioni del Dispositivo e dei suoi Asset e restituisce la rappresentazione *DashboardCommand*.

5.5.1.4) DashboardCommand

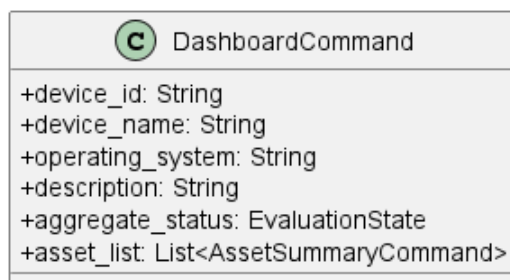


Figura 89: DashboardCommand

Descrizione

DashboardCommand è il Data Transfer Object che veicola la rappresentazione aggregata della dashboard di un Dispositivo verso il livello di presentazione. Espone le informazioni del Dispositivo, il suo stato di valutazione complessivo e la lista sintetica degli Asset associati.

Attributi

- + `device_id: String` — identificativo univoco del Dispositivo.
- + `device_name: String` — nome del Dispositivo.
- + `operating_system: String` — sistema operativo del Dispositivo.
- + `description: String` — descrizione testuale del Dispositivo.
- + `aggregate_status: EvaluationState` — stato di valutazione aggregato del Dispositivo.
- + `asset_list: List<AssetSummaryCommand>` — lista sintetica degli Asset associati al Dispositivo.

Metodi e funzioni

DashboardCommand non definisce metodi.

5.5.1.5) AssetSummaryCommand

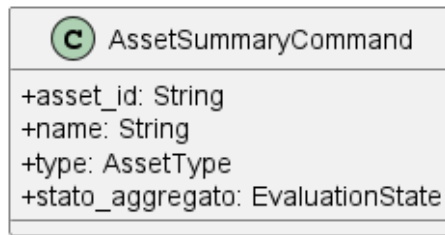


Figura 90: AssetSummaryCommand

Descrizione

AssetSummaryCommand è il Data Transfer Object che rappresenta la vista sintetica di un Asset all'interno della dashboard. È contenuto nella lista `asset_list` di *DashboardCommand*.

Attributi

- `+ asset_id: String` — identificativo univoco dell'Asset.
- `+ name: String` — nome dell'Asset.
- `+ type: AssetType` — tipo dell'Asset.
- `+ stato_aggregato: EvaluationState` — stato di valutazione aggregato dell'Asset.

Metodi e funzioni

AssetSummaryCommand non definisce metodi.

5.6) Session

5.6.1) OpenEvaluationSession

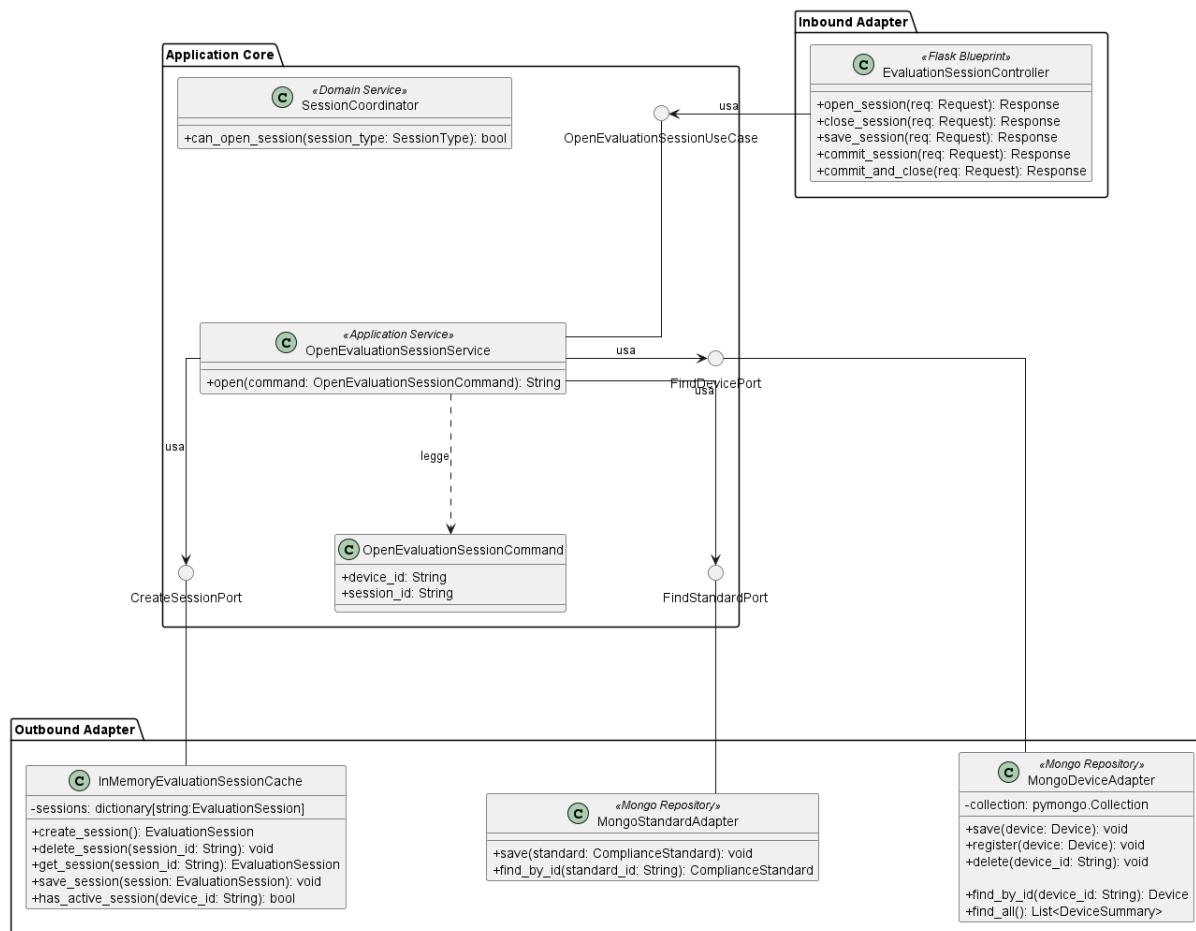


Figura 91: Caso d'uso OpenEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato all'apertura di una sessione di valutazione.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.
- Per la definizione di *FindDevicePort*, vedere la sezione **Sezione 5.2.7.3**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.6.1.1) EvaluationSessionController

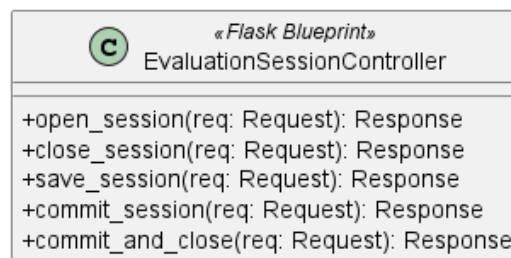


Figura 92: EvaluationSessionController

Descrizione

EvaluationSessionController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione del ciclo di vita della sessione di valutazione e le inoltra al livello applicativo.

Attributi

EvaluationSessionController non definisce attributi propri.

Metodi e funzioni

- `+ open_session(req: Request): Response` — riceve la richiesta HTTP di apertura di una nuova sessione di valutazione e restituisce una risposta HTTP con l'identificativo della sessione creata.
- `+ close_session(req: Request): Response` — riceve la richiesta HTTP di chiusura della sessione corrente e restituisce una risposta HTTP con l'esito dell'operazione.
- `+ save_session(req: Request): Response` — riceve la richiesta HTTP di salvataggio dello stato corrente della sessione e restituisce una risposta HTTP con l'esito dell'operazione.
- `+ commit_session(req: Request): Response` — riceve la richiesta HTTP di commit della sessione e restituisce una risposta HTTP con l'esito dell'operazione.
- `+ commit_and_close(req: Request): Response` — riceve la richiesta HTTP di commit e chiusura contestuale della sessione e restituisce una risposta HTTP con l'esito dell'operazione.

5.6.1.2) OpenEvaluationSessionCommand

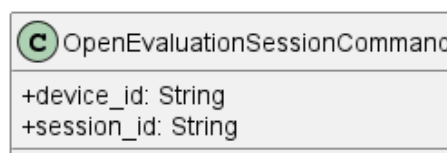


Figura 93: OpenEvaluationSessionCommand

Descrizione

OpenEvaluationSessionCommand è il Command Object utilizzato per trasportare i dati necessari all'apertura di una nuova sessione di valutazione. Incapsula i parametri di input del metodo esposto da *OpenEvaluationSessionUseCase*.

Attributi

- + device_id: String — identificativo univoco del Dispositivo per cui aprire la sessione.
- + session_id: String — identificativo univoco della sessione di valutazione da aprire.

Metodi e funzioni

OpenEvaluationSessionCommand non definisce metodi propri.

5.6.1.3) OpenEvaluationSessionUseCase

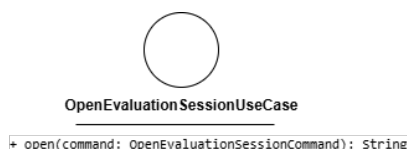


Figura 94: OpenEvaluationSessionUseCase

Descrizione

OpenEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'apertura di una nuova sessione di valutazione. Viene implementata da *OpenEvaluationSessionService* e utilizzata da *EvaluationSessionController*.

Attributi

OpenEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- + open(command: OpenEvaluationSessionCommand): String — firma del metodo delegato all'esecuzione della logica di apertura della sessione a partire dai dati contenuti nel Command.

5.6.1.4) OpenEvaluationSessionService

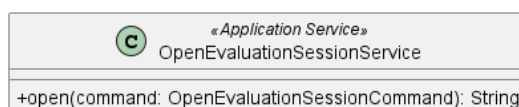


Figura 95: OpenEvaluationSessionService

Descrizione

OpenEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di apertura di una sessione di valutazione. Implementa

l'interfaccia *OpenEvaluationSessionUseCase* e coordina il recupero del Dispositivo tramite *FindDevicePort*, il recupero dello standard di conformità tramite *FindStandardPort* e la creazione della sessione tramite *CreateSessionPort*.

Attributi

OpenEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- + `open(command: OpenEvaluationSessionCommand): String` — concretizza il contratto definito da *OpenEvaluationSessionUseCase*. Recupera il Dispositivo e lo standard associato, inizializza una nuova *EvaluationSession* e ne richiede la creazione tramite *CreateSessionPort*; restituisce l'identificativo della sessione creata.

5.6.1.5) SessionCoordinator



Figura 96: SaveAssetService

Descrizione

SessionCoordinator è il Domain Service che coordina la logica di dominio relativa alla gestione della sessione di valutazione. Verifica le precondizioni necessarie all'apertura di una sessione, come l'assenza di sessioni attive per il Dispositivo.

Attributi

SessionCoordinator non definisce attributi propri.

Metodi e funzioni

- + `can_open_session(session_type: SessionType): bool` — verifica se è possibile aprire una nuova sessione del tipo specificato, restituendo `true` se le precondizioni sono soddisfatte.

5.6.1.6) CreateSessionPort

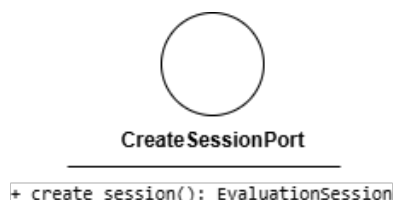


Figura 97: CreateSessionPort

Descrizione

CreateSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per la creazione di una nuova sessione di valutazione nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da *OpenEvaluationSessionService*.

Attributi

CreateSessionPort non definisce attributi.

Metodi e funzioni

- + `create_session(): EvaluationSession` — firma del metodo che inizializza e registra una nuova sessione di valutazione nel sistema in memoria.

5.6.1.7) FindStandardPort

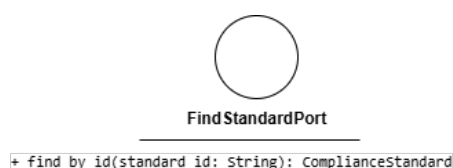


Figura 98: FindStandardPort

Descrizione

FindStandardPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero di uno standard di conformità dal sistema di persistenza. Viene implementata da *MongoStandardAdapter* e utilizzata da *OpenEvaluationSessionService*.

Attributi

FindStandardPort non definisce attributi.

Metodi e funzioni

- + `find_by_id(standard_id: String): ComplianceStandard` — firma del metodo che recupera lo standard di conformità corrispondente all'identificativo fornito.

5.6.1.8) MongoStandardAdapter



Figura 99: MongoStandardAdapter

Descrizione

MongoStandardAdapter è la classe dell'Outbound Adapter annotata come *Mongo Repository* che implementa *FindStandardPort*, traducendo le operazioni di recupero degli standard di conformità in interazioni concrete con MongoDB.

Attributi

MongoStandardAdapter non definisce attributi propri nel diagramma.

Metodi e funzioni

- + `save(standard: ComplianceStandard): void` — persiste uno standard di conformità nel database.
- + `find_by_id(standard_id: String): ComplianceStandard` — recupera lo standard di conformità corrispondente all'identificativo fornito.

5.6.2) SaveEvaluationSession

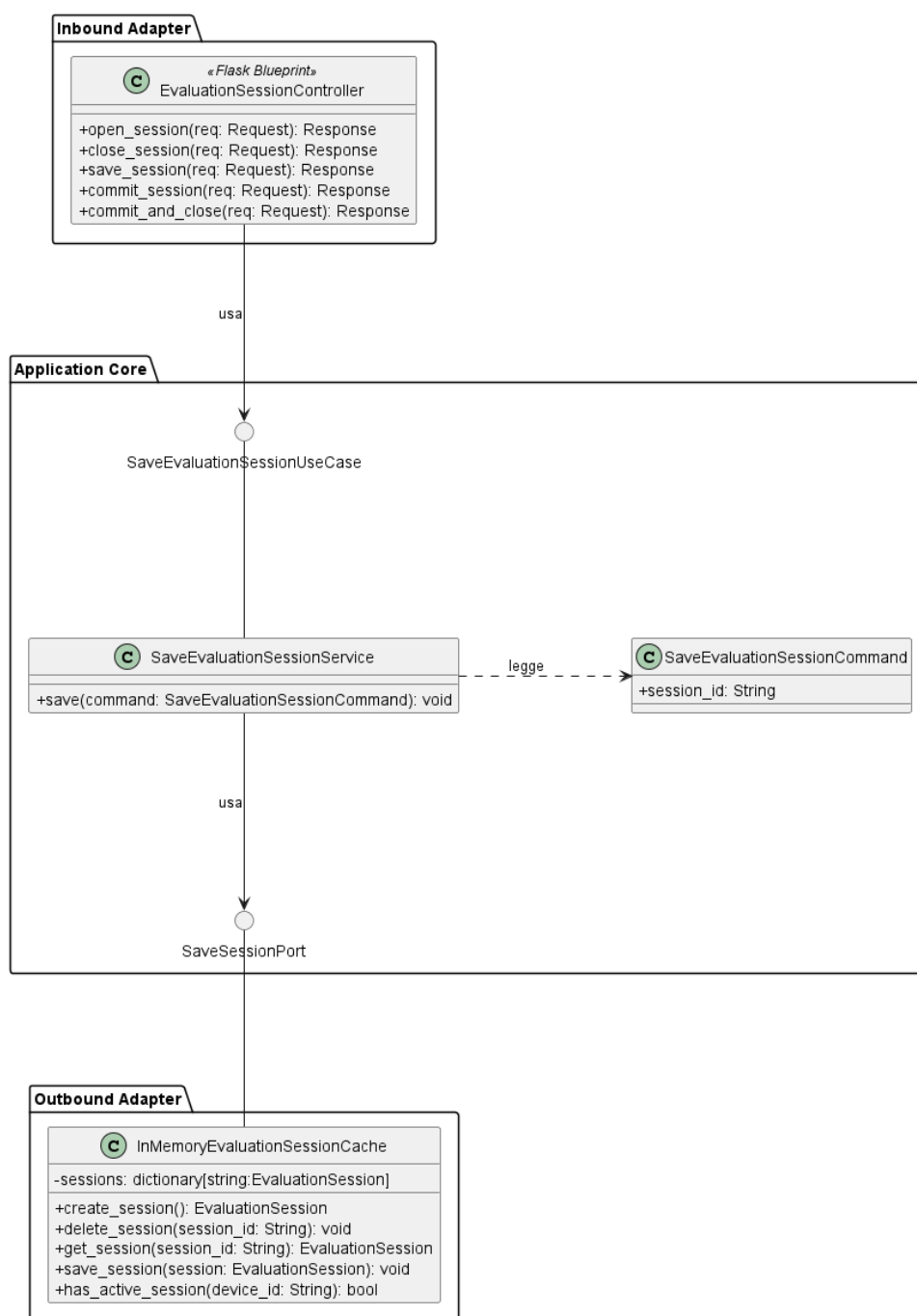


Figura 100: Caso d'uso SaveEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato al salvataggio dello stato di una sessione di valutazione corrente.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.6.1.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.6.2.1) SaveEvaluationSessionUseCase



Figura 101: SaveEvaluationSessionUseCase

Descrizione

SaveEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il salvataggio dello stato di una sessione di valutazione attiva. Viene implementata da *SaveEvaluationSessionService* e utilizzata da *EvaluationSessionController*.

Attributi

SaveEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- + save(command: SaveEvaluationSessionCommand): void — firma del metodo delegato all'esecuzione della logica di salvataggio della sessione a partire dal comando ricevuto in input.

5.6.2.2) SaveEvaluationSessionService

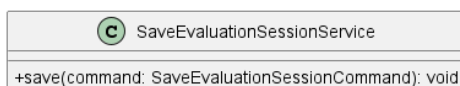


Figura 102: SaveEvaluationSessionService

Descrizione

SaveEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile del coordinamento dell'operazione di salvataggio. Implementa l'interfaccia *SaveEvaluationSessionUseCase*. Recupera la sessione corrente e ne richiede la persistenza aggiornata tramite la porta di uscita *SaveSessionPort*.

Attributi

SaveEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- + `save(command: SaveEvaluationSessionCommand): void` — concretizza il contratto definito da *SaveEvaluationSessionUseCase*. Coordina il salvataggio dello stato attuale della sessione di valutazione utilizzando i parametri incapsulati nel comando.

5.6.2.3) SaveEvaluationSessionCommand

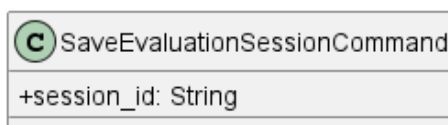


Figura 103: *SaveEvaluationSessionCommand*

Descrizione

SaveEvaluationSessionCommand incapsula i parametri necessari per richiedere il salvataggio di una sessione di valutazione. Serve a disaccoppiare i dati di input dalle firme dei metodi dei service.

Attributi

- + `session_id: String` — l'identificativo univoco della sessione di valutazione di cui si richiede il salvataggio.

Metodi e funzioni

SaveEvaluationSessionCommand non definisce metodi.

5.6.2.4) SaveSessionPort



Figura 104: *SaveSessionPort*

SaveSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per la persistenza di una sessione di valutazione nel sistema di archiviazione (in questo caso, la cache in memoria). Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da *SaveEvaluationSessionService*.

Attributi

SaveSessionPort non definisce attributi.

Metodi e funzioni

- + save_session(session: EvaluationSession): void — firma del metodo che sovrascrive o aggiorna lo stato di una sessione di valutazione nel sistema di persistenza.

5.6.3) CloseEvaluationSession

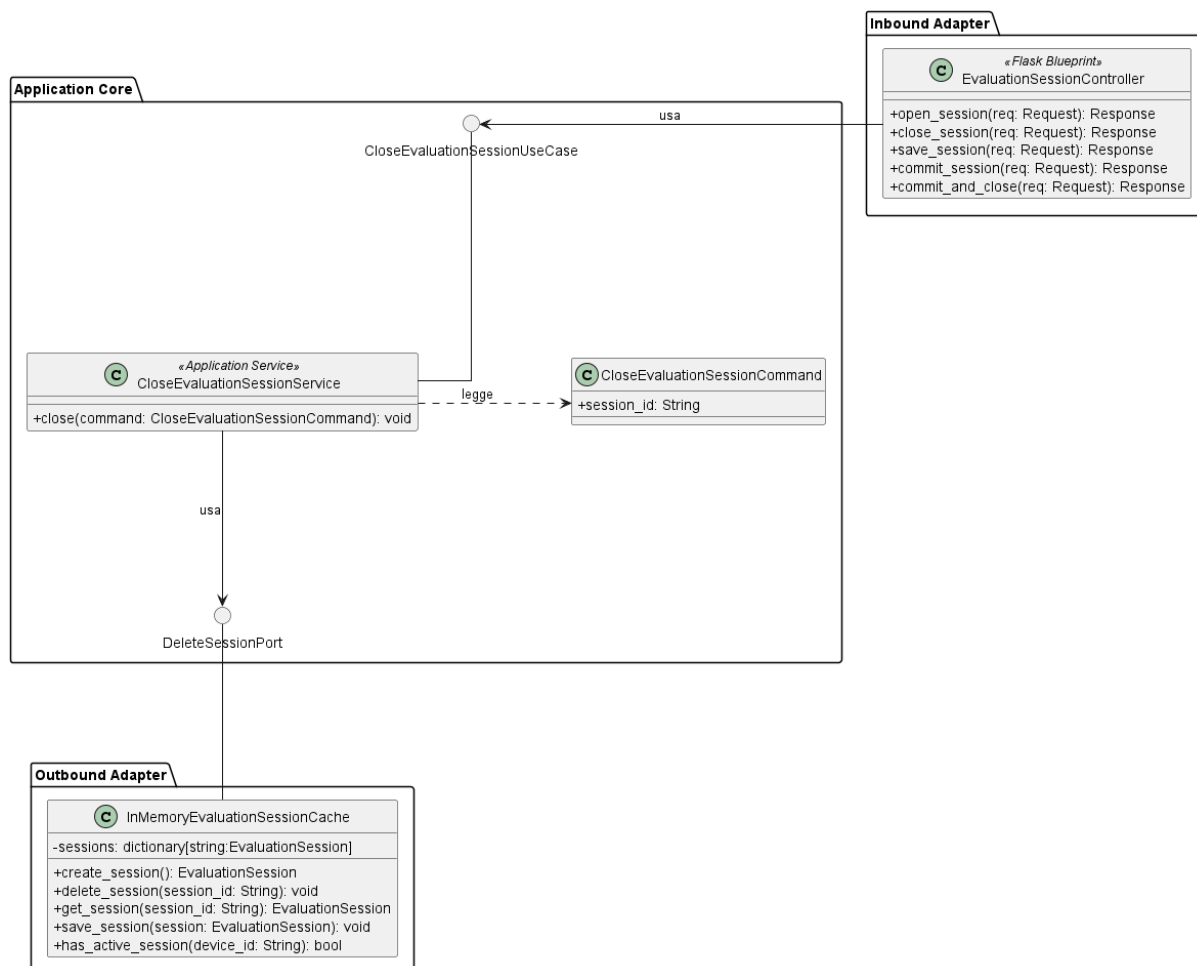


Figura 105: Caso d'uso CloseEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato alla chiusura e all'eliminazione di una sessione di valutazione.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.6.1.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.6.3.1) CloseEvaluationSessionUseCase

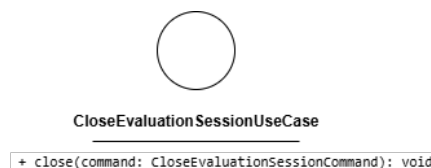


Figura 106: CloseEvaluationSessionUseCase

Descrizione

CloseEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la chiusura definitiva di una sessione di valutazione attiva. Rappresenta l'operazione conclusiva del ciclo di vita della sessione e viene implementata da *CloseEvaluationSessionService* e utilizzata dal controller *EvaluationSessionController*.

Attributi

CloseEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- `+ close(command: CloseEvaluationSessionCommand): void` — firma del metodo delegato all'esecuzione della logica di chiusura della sessione a partire dal comando ricevuto in input.

5.6.3.2) CloseEvaluationSessionService



Figura 107: CloseEvaluationSessionService

Descrizione

CloseEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di chiusura della sessione. Implementa l'interfaccia *CloseEvaluationSessionUseCase*. Una volta eseguite le operazioni di dominio necessarie, richiede la rimozione della sessione dal sistema di persistenza richiamando la porta in uscita *DeleteSessionPort*.

Attributi

CloseEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- `+ close(command: CloseEvaluationSessionCommand): void` — concretizza il contratto definito da *CloseEvaluationSessionUseCase*. Coordina le operazioni di chiusura e inoltra la richiesta di eliminazione della sessione utilizzando i parametri incapsulati nel comando.

5.6.3.3) CloseEvaluationSessionCommand

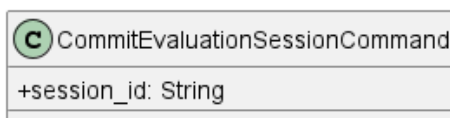


Figura 108: CloseEvaluationSessionCommand

Descrizione

CloseEvaluationSessionCommand incapsula i parametri necessari per richiedere la chiusura di una sessione di valutazione. Serve a disaccoppiare i dati di input dalle firme dei metodi dei service applicativi.

Attributi

- + `session_id: String` — l'identificativo univoco della sessione di valutazione di cui si richiede la chiusura e l'eliminazione.

Metodi e funzioni

CloseEvaluationSessionCommand non definisce metodi.

5.6.3.4) DeleteSessionPort

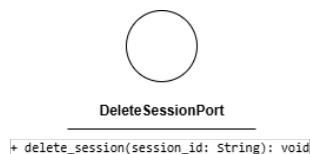


Figura 109: DeleteSessionPort

Descrizione

DeleteSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per l'eliminazione di una sessione di valutazione dal sistema di archiviazione (es. la cache in memoria). Viene utilizzata da *CloseEvaluationSessionService* e implementata nel livello di adapter da *InMemoryEvaluationSessionCache*.

Attributi

DeleteSessionPort non definisce attributi.

Metodi e funzioni

- + `delete_session(session_id: String): void` — firma del metodo che si occupa di rimuovere o invalidare lo stato di una specifica sessione di valutazione dal sistema di persistenza.

5.6.4) CommitEvaluationSession

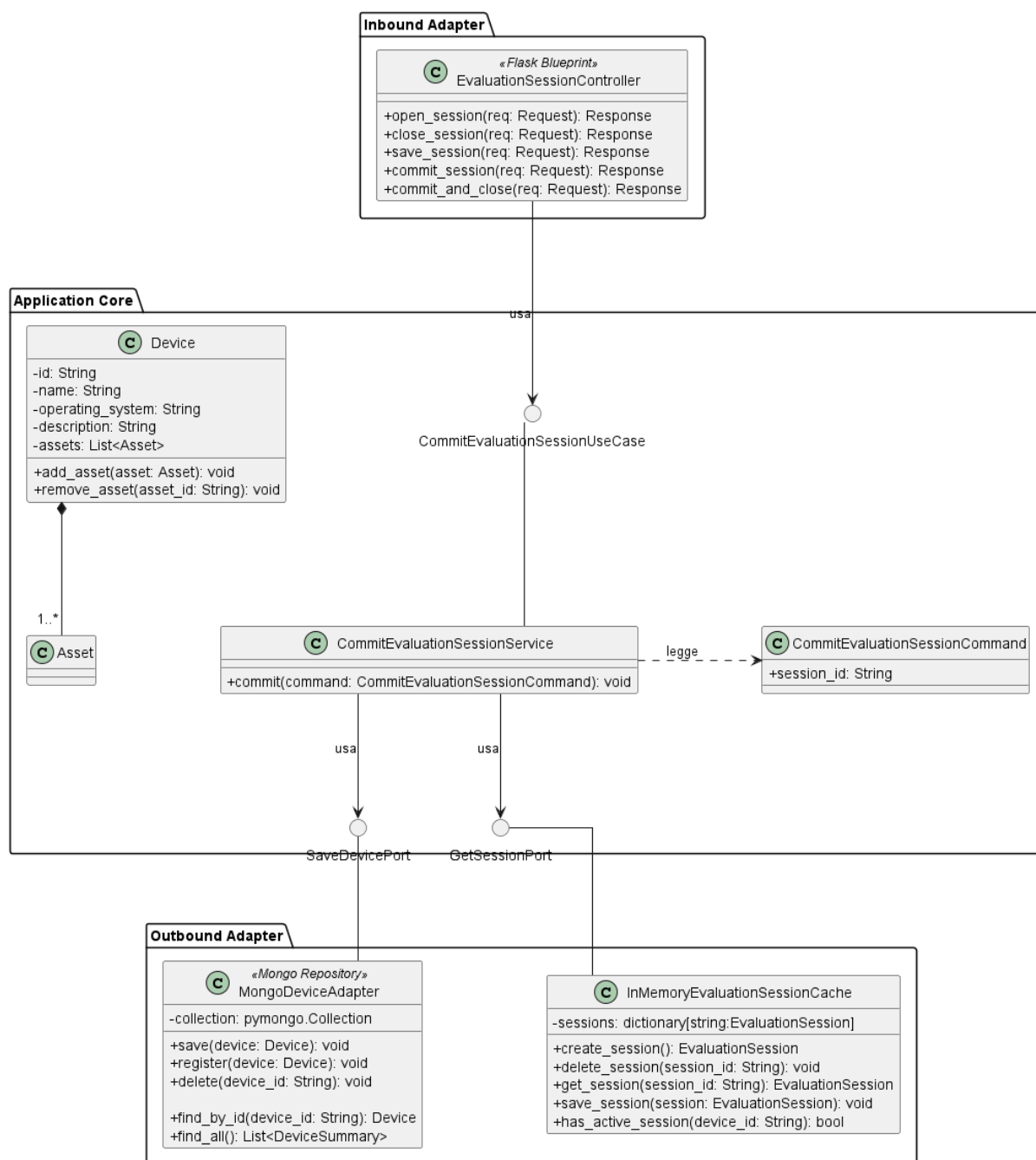


Figura 110: Caso d'uso CommitEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato esclusivamente al consolidamento (commit) dei dati di una sessione di valutazione verso il dispositivo, senza richiederne la chiusura o l'eliminazione dalla memoria temporanea.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.6.1.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

- Per la definizione di *SaveDevicePort*, vedere la sezione **Sezione 5.2.4.4**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.

Di seguito vengono documentati esclusivamente i componenti specifici introdotti per questo flusso operativo.

block(breakable: false)[

5.6.4.1) CommitEvaluationSessionService

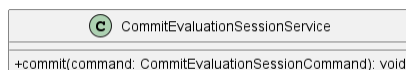


Figura 111: CommitEvaluationSessionService

CommitEvaluationSessionService è il service applicativo dell'Application Core responsabile di orchestrare l'operazione di commit. Implementa l'interfaccia *CommitEvaluationSessionUseCase*. Coordina il recupero della sessione attualmente in corso (tramite la porta *GetSessionPort*), legge i dati necessari dal *CommitEvaluationSessionCommand* e applica le modifiche definitive delegando il salvataggio all'entità dispositivo (tramite *SaveDevicePort*).

Attributi

CommitEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- + commit(command: CommitEvaluationSessionCommand): void — concretizza la logica di business relativa al consolidamento dei dati. Utilizza i parametri incapsulati nel comando per applicare le modifiche allo stato persistente del dispositivo.

]

5.6.4.2) CommitEvaluationSessionUseCase



Figura 112: CommitEvaluationSessionUseCase

Descrizione

CommitEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per eseguire il consolidamento (commit) dei dati di una sessione di valutazione attiva, applicando definitivamente le modifiche all'entità dispositivo associata. Viene implementata a livello applicativo e utilizzata dal controller *EvaluationSessionController*.

Attributi

CommitEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- + `commit(command: CommitEvaluationSessionCommand): void` — firma del metodo delegato all'esecuzione della logica di consolidamento dei dati della sessione di valutazione a partire dal comando ricevuto in input.

5.6.4.3) CommitEvaluationSessionCommand

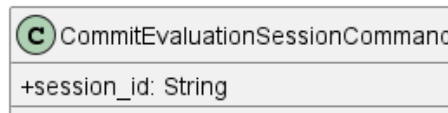


Figura 113: `CommitEvaluationSessionCommand`

Descrizione

CommitEvaluationSessionCommand incapsula i parametri necessari per richiedere il consolidamento (`commit`) di una sessione di valutazione. Serve a disaccoppiare i dati di input dalle firme dei metodi dei service applicativi.

Attributi

- + `session_id: String` — l'identificativo univoco della sessione di valutazione di cui si richiede il consolidamento dei dati.

Metodi e funzioni

CommitEvaluationSessionCommand non definisce metodi.

5.6.4.4) CommitCloseSession

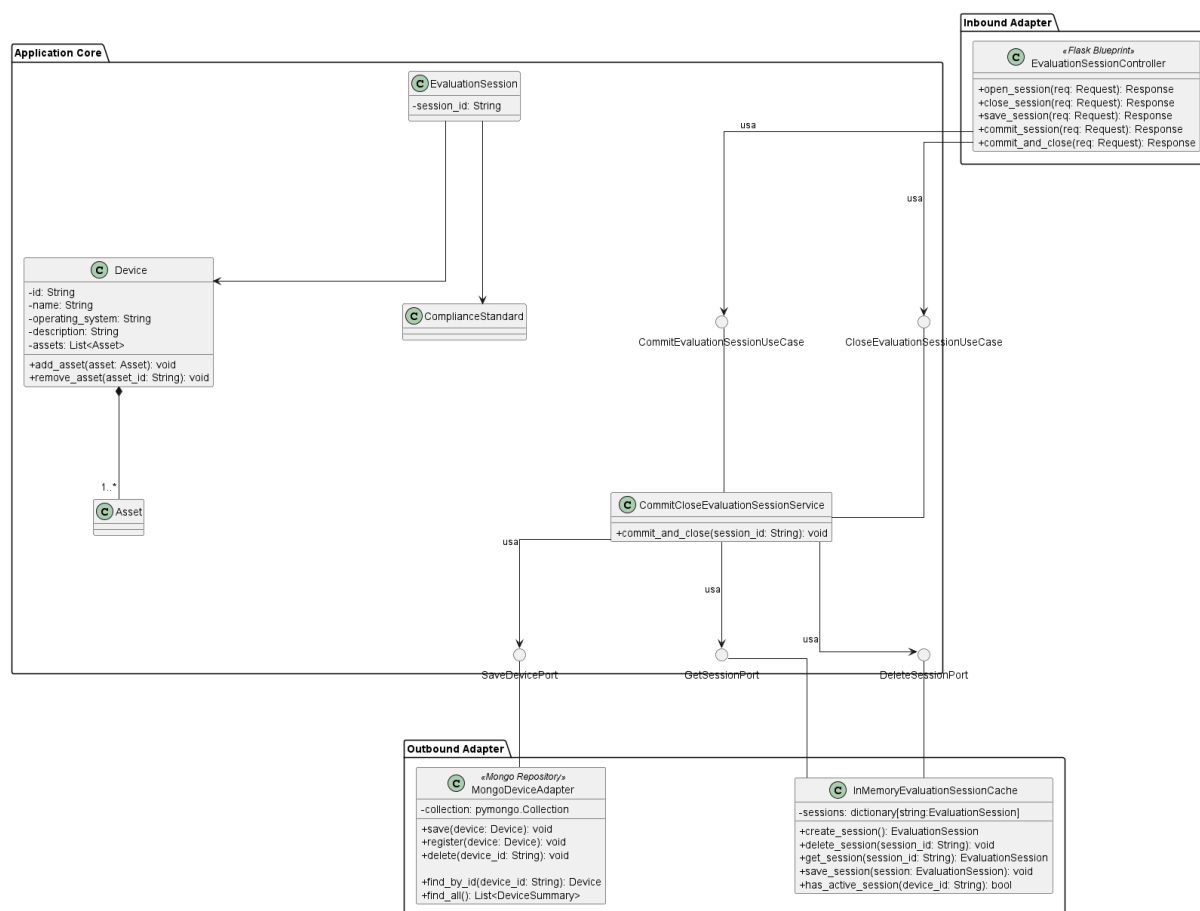


Figura 114: Caso d'uso CommitCloseSession

Il diagramma illustra l'architettura del modulo dedicato al consolidamento (commit) e alla contestuale chiusura di una sessione di valutazione.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.6.1.1.**
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5.**
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7.**
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7.**
- Per la definizione di *CloseEvaluationSessionUseCase*, vedere la sezione **Sezione 5.6.3.1.**
- Per la definizione di *DeleteSessionPort*, vedere la sezione **Sezione 5.6.3.4.**
- Per la definizione di *SaveDevicePort*, vedere la sezione **Sezione 5.2.4.4.**
- Per la definizione di *CommitEvaluationSessionUseCase*, vedere la sezione **Sezione 5.6.4.2.**

Di seguito vengono documentati esclusivamente i componenti introdotti o aggregati specificamente per questo flusso operativo.

5.6.4.5) CommitCloseEvaluationSessionService

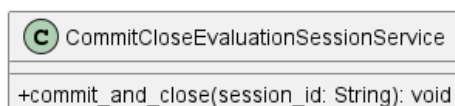


Figura 115: CommitCloseEvaluationSessionService

Descrizione

CommitCloseEvaluationSessionService è il service applicativo dell'Application Core responsabile di orchestrare le operazioni combinate. Funendo da implementazione per i casi d'uso di commit e chiusura, coordina il recupero della sessione in corso (tramite *GetSessionPort*), il salvataggio dei risultati consolidati sul dispositivo (tramite *SaveDevicePort*) e, in caso di successo, la rimozione della sessione attiva (tramite *DeleteSessionPort*).

Attributi

CommitCloseEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- `+ commit_and_close(session_id: String): void` — concretizza la logica di business principale. Prende in input l'identificativo della sessione, ne effettua il commit dei dati sull'entità *Device* e successivamente procede con l'eliminazione della sessione dalla memoria temporanea.

5.7) Valutazione

5.7.1) EvaluateDecisionNode

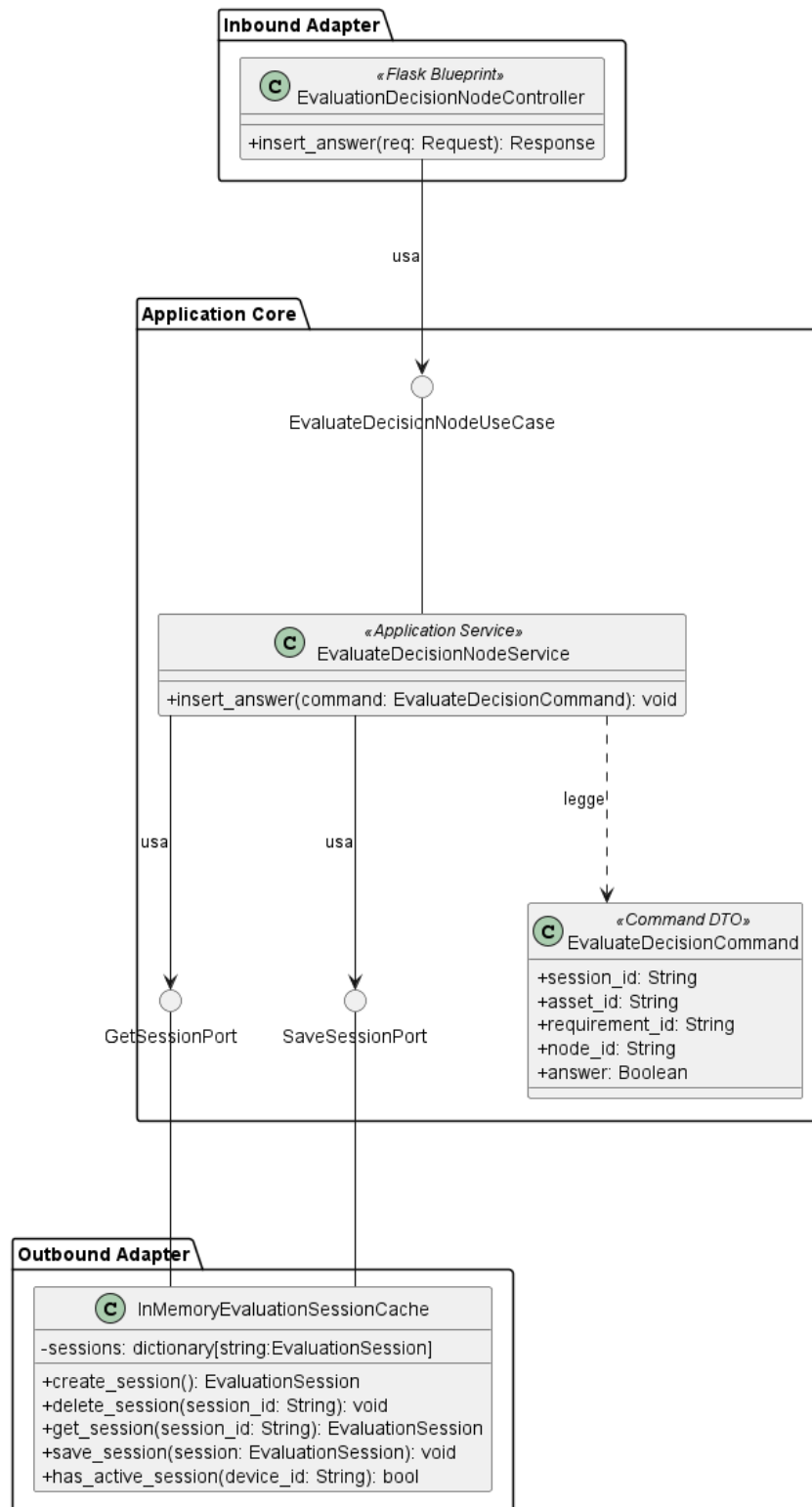


Figura 116: Caso d'uso EvaluateDecisionNode

Il diagramma illustra l'architettura del modulo dedicato alla valutazione di un nodo decisionale durante la verifica di conformità.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.
- Per la definizione di *SaveSessionPort*, vedere la sezione **Sezione 5.3.1.8**.

5.7.1.1) EvaluationDecisionNodeController

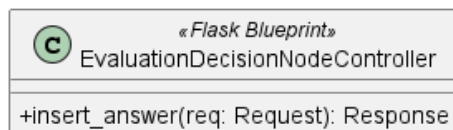


Figura 117: EvaluationDecisionNodeController

Descrizione

EvaluationDecisionNodeController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP per la valutazione di un nodo decisionale e le inoltra al livello applicativo.

Attributi

EvaluationDecisionNodeController non definisce attributi propri.

Metodi e funzioni

- + *evaluate_node*(req: Request): Response — riceve la richiesta HTTP, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l’esito dell’operazione.

5.7.1.2) EvaluateDecisionNodeUseCase

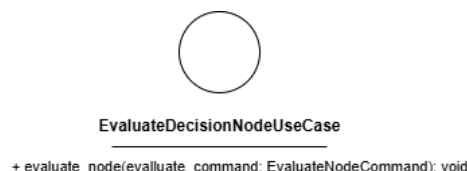


Figura 118: EvaluateDecisionNodeUseCase

Descrizione

EvaluateDecisionNodeUseCase è l’interfaccia (Inbound Port) che definisce il contratto per la valutazione di un nodo decisionale. Viene implementata da *EvaluateDecisionNodeService* e utilizzata da *EvaluationDecisionNodeController*.

Attributi

EvaluateDecisionNodeUseCase non definisce attributi.

Metodi e funzioni

- + evaluate_node(evaluate_command: EvaluationNodeCommand): void — firma del metodo delegato all'esecuzione della logica di valutazione a partire dai dati contenuti nel comando.

5.7.1.3) EvaluationNodeCommand

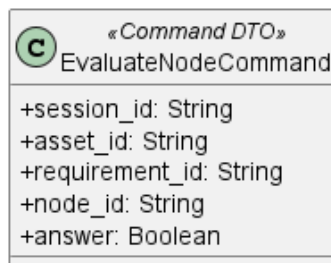


Figura 119: EvaluateNodeCommand

Descrizione

EvaluationNodeCommand è il Command Object annotato come *Command DTO* che veicola i dati necessari alla valutazione di un nodo decisionale dal controller al service.

Attributi

- + session_id: String — identificativo della sessione di valutazione attiva.
- + asset_id: String — identificativo dell'Asset oggetto di valutazione.
- + requirement_id: String — identificativo del requisito in fase di valutazione.
- + node_id: String — identificativo del nodo decisionale.
- + answer: Boolean — valore della risposta fornita per il nodo decisionale.

Metodi e funzioni

EvaluationNodeCommand non definisce metodi comportamentali, agendo esclusivamente come struttura dati.

5.7.1.4) EvaluateDecisionNodeService

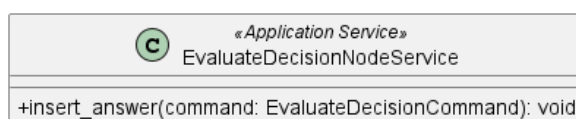


Figura 120: EvaluateDecisionNodeService

Descrizione

EvaluateDecisionNodeService è il service applicativo appartenente all'Application Core responsabile della logica di valutazione di un nodo decisionale. Implementa l'interfaccia *EvaluateDecisionNodeUseCase*, recupera la sessione attiva tramite *GetSessionPort*, applica la valutazione al nodo corrispondente e persiste la sessione aggiornata tramite *SaveSessionPort*.

Attributi

EvaluateDecisionNodeService non definisce attributi propri.

Metodi e funzioni

- + `evaluate_node(evaluate_command: EvaluationNodeCommand): void` — concretizza il contratto definito da *EvaluateDecisionNodeUseCase*. Recupera la sessione attiva, individua il nodo decisionale corrispondente, elabora la risposta e persiste la sessione aggiornata.

5.7.2) GetRequirement

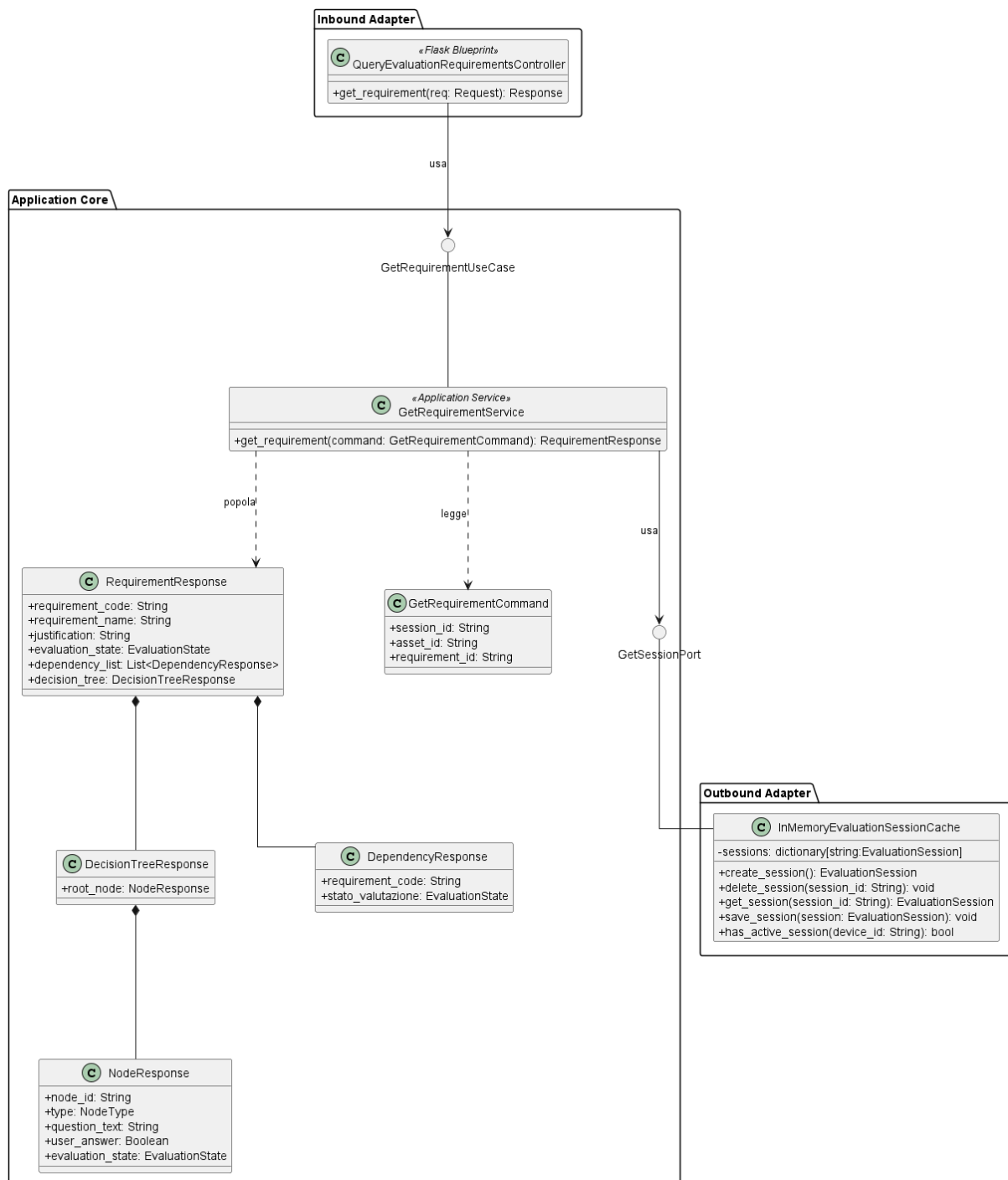


Figura 121: Caso d'uso GetRequirement

Il diagramma illustra l'architettura del modulo dedicato al recupero di un requisito di conformità con il relativo albero decisionale e le dipendenze associate.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.

5.7.2.1) QueryEvaluationRequirementsController

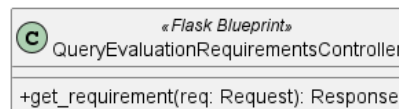


Figura 122: QueryEvaluationRequirementsController

Descrizione

QueryEvaluationRequirementsController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di recupero di un requisito di conformità e le inoltra al livello applicativo.

Attributi

QueryEvaluationRequirementsController non definisce attributi propri.

Metodi e funzioni

- + `get_requirement(req: Request): Response` — riceve la richiesta HTTP di recupero di un requisito, estrae i parametri dalla richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con i dati del requisito.

5.7.2.2) GetRequirementUseCase



Figura 123: GetRequirementUseCase

Descrizione

GetRequirementUseCase è l’interfaccia (Inbound Port) che definisce il contratto per il recupero di un requisito di conformità. Viene implementata da *GetRequirementService* e utilizzata da *QueryEvaluationRequirementsController*.

Attributi

GetRequirementUseCase non definisce attributi.

Metodi e funzioni

- + `get_requirement(command: GetRequirementCommand): RequirementResponse` — firma del metodo delegato al recupero del requisito corrispondente ai parametri incapsulati nel comando fornito in input.

5.7.2.3) GetRequirementCommand

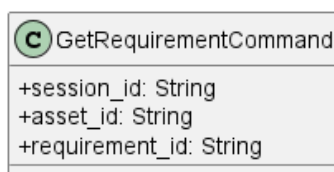


Figura 124: GetRequirementCommand

Descrizione

GetRequirementCommand è l'oggetto che veicola i parametri necessari al recupero di un requisito dal controller al service.

Attributi

- + `session_id: String` — identificativo della sessione di valutazione attiva.
- + `asset_id: String` — identificativo dell'Asset oggetto di valutazione.
- + `requirement_id: String` — identificativo del requisito da recuperare.

Metodi e funzioni

GetRequirementCommand non definisce metodi.

5.7.2.4) GetRequirementService

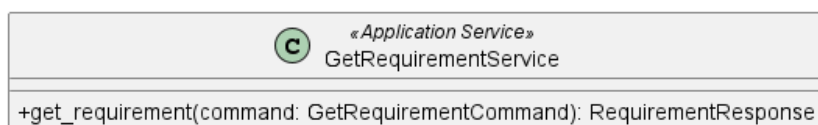


Figura 125: GetRequirementService

Descrizione

GetRequirementService è il service applicativo appartenente all'Application Core responsabile del recupero di un requisito di conformità. Implementa l'interfaccia *GetRequirementUseCase*, legge i parametri dal *GetRequirementCommand*, recupera la sessione attiva tramite *GetSessionPort* e si occupa di costruire il *RequirementResponse* contenente i dati richiesti.

Attributi

GetRequirementService non definisce attributi propri.

Metodi e funzioni

- + `get_requirement(command: GetRequirementCommand): RequirementResponse` — concretizza il contratto definito da *GetRequirementUseCase*. Recupera la sessione attiva, individua il requisito richiesto utilizzando i parametri incapsulati nel comando e ne costruisce la relativa rappresentazione sotto forma di *RequirementResponse*.

5.7.2.5) RequirementResponse

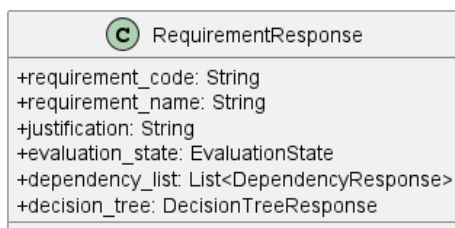


Figura 126: RequirementResponse

Descrizione

RequirementResponse è il Data Transfer Object che veicola la rappresentazione completa di un requisito di conformità verso il livello di presentazione, includendo il suo stato di valutazione, l'albero decisionale associato e le dipendenze con altri requisiti.

Attributi

- + requirement_code: String — codice identificativo del requisito.
- + requirement_name: String — nome del requisito.
- + justification: String — giustificazione associata al requisito.
- + evaluation_state: EvaluationState — stato di valutazione del requisito.
- + dependency_list: List<DependencyResponse> — lista delle dipendenze con altri requisiti.
- + decision_tree: DecisionTreeResponse — albero decisionale associato al requisito.

Metodi e funzioni

RequirementResponse non definisce metodi.

5.7.2.6) DecisionTreeResponse

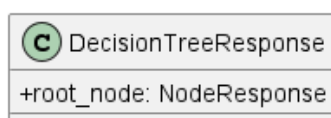


Figura 127: DecisionTreeResponse

Descrizione

DecisionTreeResponse è il Data Transfer Object che rappresenta l'albero decisionale associato a un requisito, strutturato a partire dal nodo radice.

Attributi

- + root_node: NodeResponse — nodo radice dell'albero decisionale.

Metodi e funzioni

DecisionTreeResponse non definisce metodi.

5.7.2.7) NodeResponse

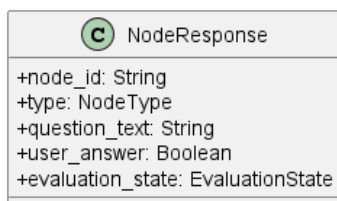


Figura 128: NodeResponse

Descrizione

NodeResponse è il Data Transfer Object che rappresenta un singolo nodo dell'albero decisionale, con il testo della domanda, la risposta fornita dall'utente e lo stato di valutazione associato.

Attributi

- + `node_id`: String — identificativo univoco del nodo.
- + `type`: NodeType — tipo del nodo nell'albero decisionale.
- + `question_text`: String — testo della domanda associata al nodo.
- + `user_answer`: Boolean — risposta fornita dall'utente per questo nodo.
- + `evaluation_state`: EvaluationState — stato di valutazione del nodo.

Metodi e funzioni

NodeResponse non definisce metodi.

5.7.2.8) DependencyResponse

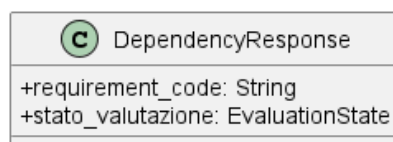


Figura 129: DependencyResponse

Descrizione

DependencyResponse è il Data Transfer Object che rappresenta la dipendenza di un requisito da un altro, includendo il codice del requisito dipendente e il suo stato di valutazione.

Attributi

- + `requirement_code`: String — codice del requisito da cui dipende il requisito corrente.
- + `stato_valutazione`: EvaluationState — stato di valutazione del requisito dipendente.

Metodi e funzioni

DependencyResponse non definisce metodi.

5.7.3) InsertJustification

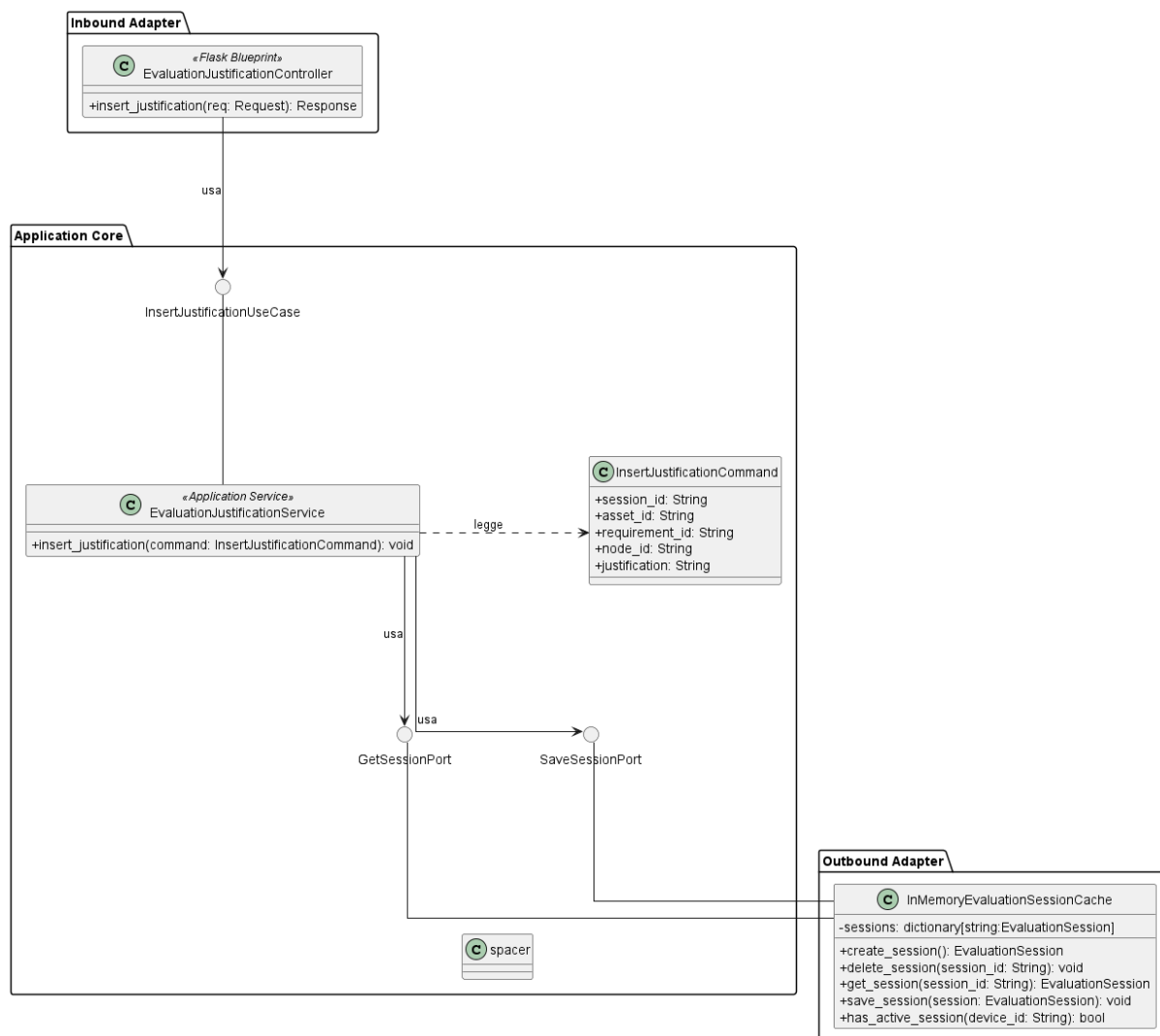


Figura 130: Caso d'uso InsertJustification

Il diagramma illustra l'architettura del modulo dedicato all'inserimento di una giustificazione testuale per un requisito di conformità durante la sessione di valutazione.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.
- Per la definizione di *SaveSessionPort*, vedere la sezione **Sezione 5.3.1.8**.

Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso.

5.7.3.1) EvaluationJustificationController

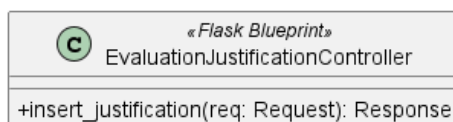


Figura 131: EvaluationJustificationController

Descrizione

EvaluationJustificationController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di inserimento di una giustificazione per un requisito e le inoltra al livello applicativo.

Attributi

EvaluationJustificationController non definisce attributi propri.

Metodi e funzioni

- + `insert_justification(req: Request): Response` — riceve la richiesta HTTP di inserimento della giustificazione, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l’esito dell’operazione.

5.7.3.2) InsertJustificationUseCase



Figura 132: InsertJustificationUseCase

Descrizione

InsertJustificationUseCase è l’interfaccia (Inbound Port) che definisce il contratto per l’inserimento di una giustificazione testuale associata a un requisito di conformità. Viene implementata da *EvaluationJustificationService* e utilizzata da *EvaluationJustificationController*.

Attributi

InsertJustificationUseCase non definisce attributi.

Metodi e funzioni

- + `insert_justification(command: InsertJustificationCommand): void` — firma del metodo delegato all’esecuzione della logica di inserimento della giustificazione a partire dai dati contenuti nel comando.

5.7.3.3) InsertJustificationCommand

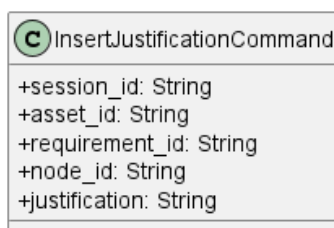


Figura 133: InsertJustificationCommand

Descrizione

InsertJustificationCommand è il Command Object che veicola i dati necessari all'inserimento di una giustificazione dal controller al service.

Attributi

- + session_id: String — identificativo della sessione di valutazione attiva.
- + asset_id: String — identificativo dell'Asset oggetto di valutazione.
- + requirement_id: String — identificativo del requisito a cui si associa la giustificazione.
- + node_id: String — identificativo del nodo decisionale correlato.
- + justification: String — testo della giustificazione da inserire.

Metodi e funzioni

InsertJustificationCommand non definisce metodi.

5.7.3.4) EvaluationJustificationService



Figura 134: EvaluationJustificationService

Descrizione

EvaluationJustificationService è il service applicativo appartenente all'Application Core responsabile della logica di inserimento di una giustificazione per un requisito di conformità. Implementa l'interfaccia *InsertJustificationUseCase*, recupera la sessione attiva tramite *GetSessionPort*, associa la giustificazione al requisito specificato e persiste la sessione aggiornata tramite *SaveSessionPort*.

Attributi

EvaluationJustificationService non definisce attributi propri.

Metodi e funzioni

- + insert_justification(command: InsertJustificationCommand): void — concretizza il contratto definito da *InsertJustificationUseCase*. Recupera la sessione attiva,

individua il requisito corrispondente ai parametri incapsulati nel comando, registra la giustificazione fornita e persiste la sessione aggiornata.

5.8) Export

5.8.1) ExportDevice

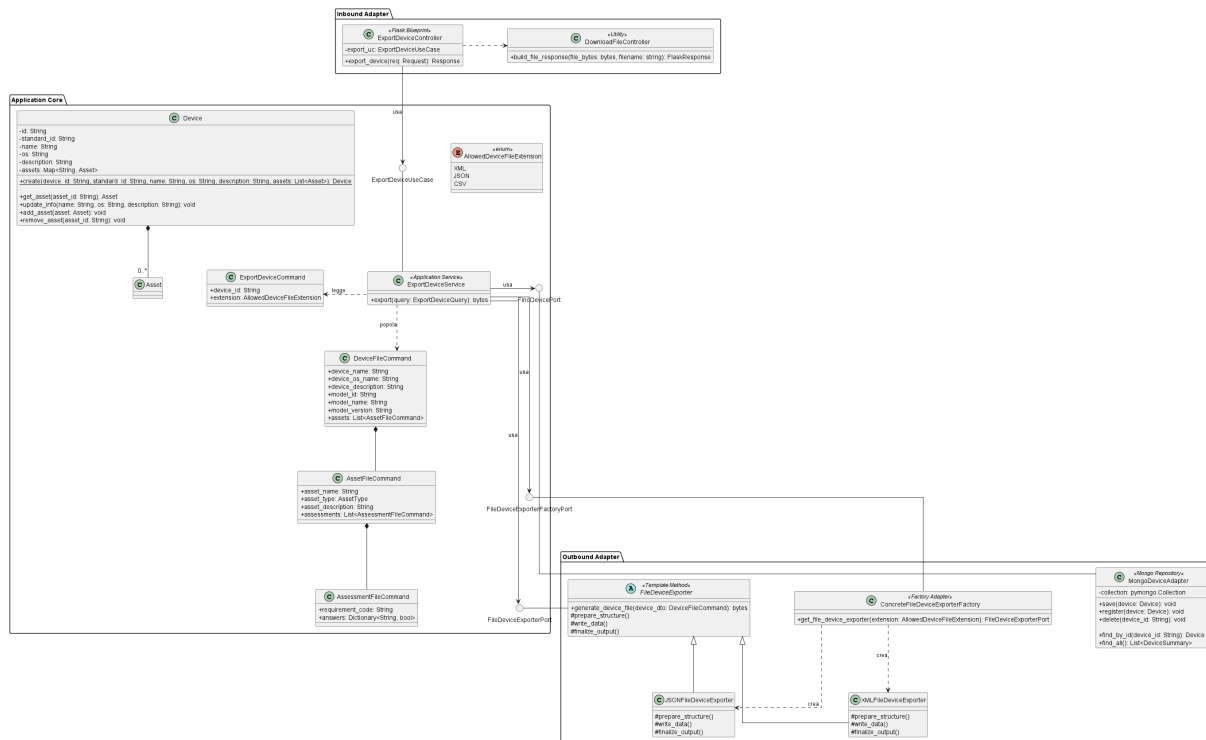


Figura 135: Caso d'uso ExportDevice

Il diagramma illustra l'architettura del modulo dedicato all'esportazione dei dati di un dispositivo (inclusi i suoi asset e le relative valutazioni di conformità) in un file scaricabile. Il sistema supporta diversi formati (es. JSON, XML, CSV) utilizzando il pattern Factory per la creazione dell'esportatore corretto e il pattern Template Method per standardizzare l'algoritmo di generazione del file.

- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7.**
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5.**
- Per la definizione di *FindDevicePort*, vedere la sezione **Sezione 5.2.7.3.**

5.8.1.1) ExportDeviceController



Figura 136: ExportDeviceController

Descrizione

ExportDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di esportazione di un dispositivo e le inoltra al livello applicativo.

Attributi

- - `export_uc`: *ExportDeviceUseCase* — riferimento all'interfaccia (porta di ingresso) del caso d'uso, iniettata nel controller per delegare l'esecuzione della logica di business.

Metodi e funzioni

- + `export_device(req: Request): Response` — riceve la richiesta HTTP, estrae l'identificativo del dispositivo e il formato desiderato, invoca il caso d'uso di esportazione e delega la costruzione della risposta HTTP al *DownloadFileController*.

5.8.1.2) DownloadFileController

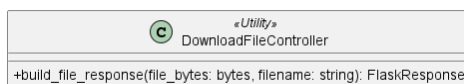


Figura 137: DownloadFileController

Descrizione

DownloadFileController è una classe di utilità dell'Inbound Adapter responsabile della formattazione dell'output binario in una risposta HTTP valida per il download di file.

Attributi

DownloadFileController non definisce attributi propri.

Metodi e funzioni

- + `build_file_response(file_bytes: bytes, filename: string): FlaskResponse` — impacchetta l'array di byte generato dall'Application Core all'interno di una *FlaskResponse* configurata con gli header corretti per forzare il download del file lato client.

5.8.1.3) ExportDeviceUseCase



Figura 138: ExportDeviceUseCase

Descrizione

ExportDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'esportazione dei dati di un dispositivo in formato byte. Viene implementata da *ExportDeviceService* e utilizzata da *ExportDeviceController*.

5.8.1.4) ExportDeviceCommand

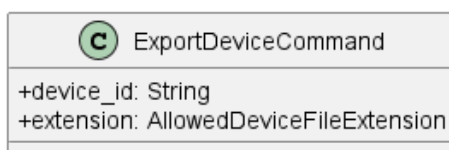


Figura 139: ExportDeviceCommand

Descrizione

ExportDeviceCommand è il Command Object che veicola i parametri della richiesta di esportazione dal controller al service.

Attributi

- + `device_id`: String — identificativo univoco del dispositivo da esportare.
- + `extension`: AllowedDeviceFileExtension — formato del file richiesto per l'esportazione (es. XML, JSON).

5.8.1.5) AllowedDeviceFileExtension

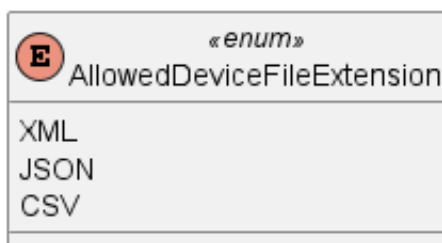


Figura 140: AllowedDeviceFileExtension

Descrizione

AllowedDeviceFileExtension è un'enumerazione che definisce i formati di file supportati dal sistema per l'esportazione dei dispositivi.

Valori

- XML
- JSON
- CSV

5.8.1.6) ExportDeviceService



Figura 141: ExportDeviceService

Descrizione

ExportDeviceService è il service applicativo responsabile di orchestrare il processo di esportazione. Legge i parametri della richiesta, recupera il dispositivo tramite *FindDevicePort*, converte i dati, richiede l'esportatore corretto tramite *FileDeviceExporterFactoryPort* e avvia la generazione del file tramite *FileDeviceExporterPort*.

Attributi

ExportDeviceService non definisce attributi propri.

Metodi e funzioni

- `+ export(query: ExportDeviceQuery): bytes` — concretizza il contratto definito da *ExportDeviceUseCase*, restituendo il file generato sotto forma di array di byte. (**Nota: il parametro riceve i dati strutturati dal Command**).

5.8.1.7) DeviceFileCommand, AssetFileCommand e AssessmentFileCommand

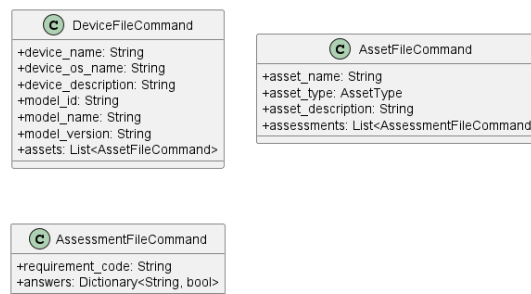


Figura 142: DDeviceFileCommand, AssetFileCommand e AssessmentFileCommand

Descrizione

Questo gruppo di classi rappresenta i Data Transfer Object (DTO) gerarchici creati dal service per passare i dati di dominio all'Adapter di esportazione in un formato disaccoppiato dalle entità core.

Attributi principali

- **DeviceFileCommand:** Contiene le anagrafiche del dispositivo (`device_name`, `device_os_name`, `model_name`, ecc.) e una lista di *AssetFileCommand*.
- **AssetFileCommand:** Contiene i dettagli del singolo asset (`asset_name`, `asset_type`, ecc.) e una lista di *AssessmentFileCommand*.
- **AssessmentFileCommand:** Contiene il codice del requisito valutato (`requirement_code`) e un dizionario con le relative risposte dell'utente (`answers`).

5.8.1.8) ConcreteFileDeviceExporterFactory



Figura 143: ConcreteFileDeviceExporterFactory

Descrizione

ConcreteFileDeviceExporterFactory è l'Outbound Adapter che implementa il pattern Factory. Si occupa di istanziare la classe di esportazione corretta in base all'estensione richiesta.

Metodi e funzioni

- + `get_file_device_exporter(extension: AllowedDeviceFileExtension): FileDeviceExporterPort` — restituisce l'istanza specifica dell'esportatore (es. JSON o XML) associata al formato richiesto.

5.8.1.9) FileDeviceExporter

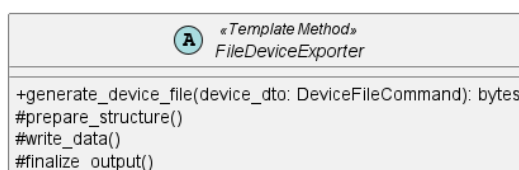


Figura 144: FileDeviceExporter

Descrizione

FileDeviceExporter è una classe astratta nell'Outbound Adapter che implementa *FileDeviceExporterPort* definendo lo scheletro dell'algoritmo di esportazione tramite il pattern Template Method. Le sottoclassi concrete (*JSONFileDeviceExporter*, *XMLFileDeviceExporter*) implementano i passaggi specifici per il formato di destinazione.

Metodi e funzioni

- + `generate_device_file(device_dto: DeviceFileCommand): bytes` — metodo *Template* principale che orchestra le chiamate ai metodi protetti sottostanti.
- # `prepare_structure()` — metodo astratto per l'inizializzazione della struttura del documento (es. root XML o dizionario JSON).
- # `write_data()` — metodo astratto per l'inserimento dei dati nel documento.
- # `finalize_output()` — metodo astratto per la chiusura del documento e la conversione finale in array di byte.

5.8.1.10) JSONFileDeviceExporter e XMLFileDeviceExporter



Figura 145: JSONFileDeviceExporter e XMLFileDeviceExporter

Descrizione

JSONFileDeviceExporter e *XMLFileDeviceExporter* sono le classi concrete dell'Outbound Adapter che estendono la classe base astratta *FileDeviceExporter*. Ognuna di esse implementa i passaggi specifici dell'algoritmo di esportazione definito tramite il pattern

Template Method, occupandosi rispettivamente della formattazione e serializzazione dei dati nei formati JSON e XML.

Attributi

Queste classi non definiscono attributi.

Metodi e funzioni

- # `prepare_structure()` — implementazione concreta del passaggio di inizializzazione della struttura del file (es. la creazione del nodo root in XML o del dizionario/oggetto base in JSON).
- # `write_data()` — implementazione concreta del passaggio in cui i dati, estratti precedentemente nel DTO, vengono mappati e scritti all'interno della struttura specifica del documento.
- # `finalize_output()` — implementazione concreta della chiusura del documento (se necessaria) e della sua conversione (serializzazione) in un array di byte finale, pronto per essere inviato al client.

5.8.1.11) FileDeviceExporterFactoryPort



Figura 146: FileDeviceExporterFactoryPort

Descrizione

FileDeviceExporterFactoryPort è l'interfaccia (Outbound Port) definita nell'Application Core che stabilisce il contratto per istanziare dinamicamente l'esportatore di file corretto. Viene utilizzata dal service per disaccoppiare la logica di business dalla creazione concreta degli adapter ed è implementata da *ConcreteFileDeviceExporterFactory*.

Attributi

FileDeviceExporterFactoryPort non definisce attributi.

Metodi e funzioni

- + `get_file_device_exporter(extension: AllowedDeviceFileExtension): FileDeviceExporterPort` — firma del metodo delegato alla creazione dell'esportatore. Riceve in input l'estensione desiderata e restituisce la porta *FileDeviceExporterPort* specifica e pronta all'uso per quel formato.

5.8.1.12) FileDeviceExporterPort



Figura 147: FileDeviceExporterPort

Descrizione

FileDeviceExporterPort è l'interfaccia (Outbound Port) definita nell'Application Core che stabilisce il contratto per la generazione materiale del file di esportazione. Viene utilizzata dal service per avviare la serializzazione dei dati ed è implementata dalla classe *FileDeviceExporter* dell'Outbound Adapter.

Attributi

FileDeviceExporterPort non definisce attributi.

Metodi e funzioni

- `+ generate_device_file(device_dto: DeviceFileCommand): bytes` — firma del metodo delegato all'esportazione vera e propria dei dati. Riceve in input la struttura gerarchica dei DTO (radicata in *DeviceFileCommand*) contenente le informazioni del dispositivo e restituisce il file generato sotto forma di array di byte.

5.8.2) ExportReport

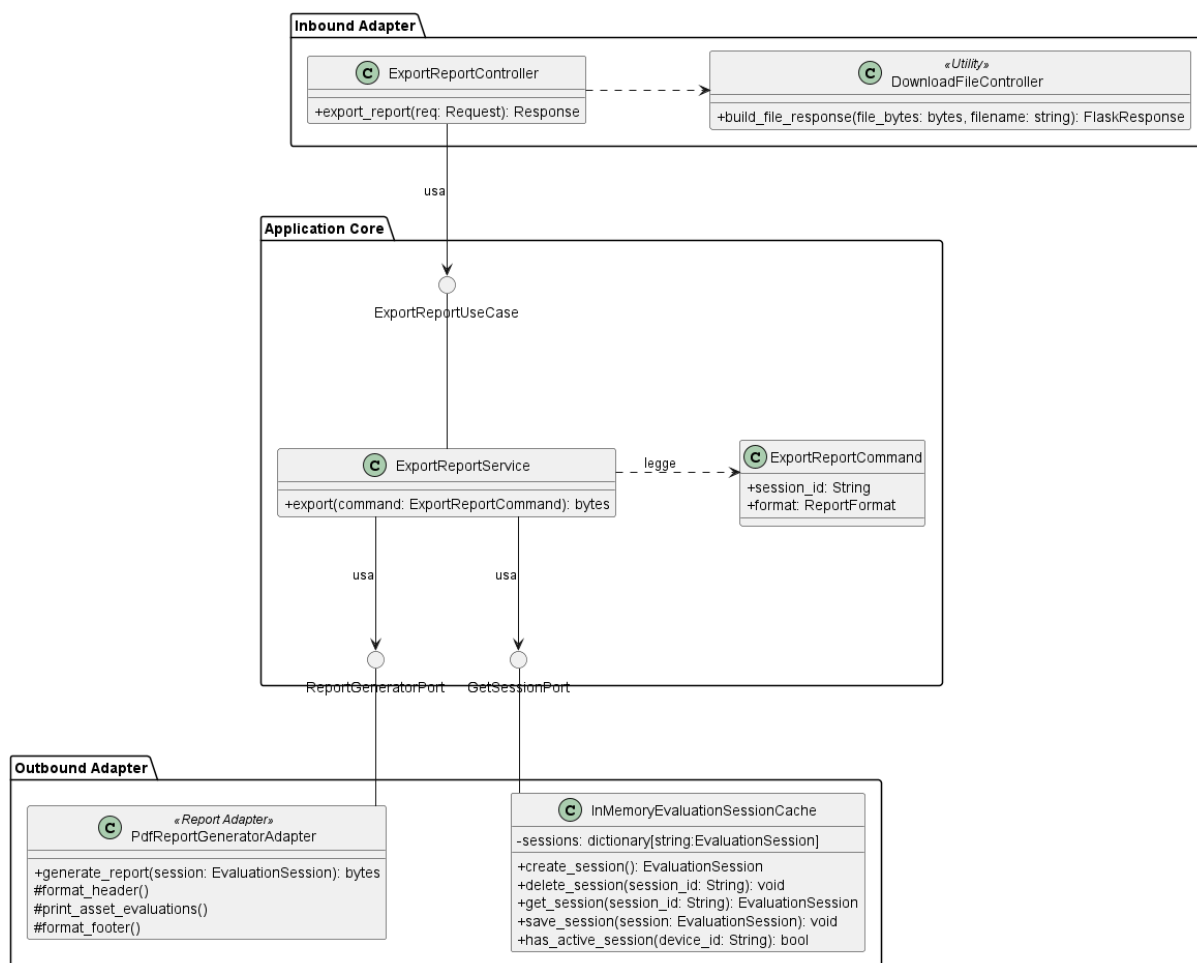


Figura 148: Caso d'uso ExportReport

Il diagramma illustra l'architettura del modulo dedicato alla generazione e all'esportazione dei report riassuntivi di una valutazione. Il flusso permette di recuperare i dati di una sessione e di compilare dinamicamente un documento (ad esempio in formato PDF) da restituire all'utente tramite download.

- Per la definizione di `InMemoryEvaluationSessionCache`, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di `GetSessionPort`, vedere la sezione **Sezione 5.3.1.9**.
- Per la definizione di `DownloadFileController`, vedere la sezione **Sezione 5.8.1.2**

Di seguito vengono analizzati nel dettaglio i componenti specifici introdotti per questo caso d'uso.

5.8.2.1) ExportReportController

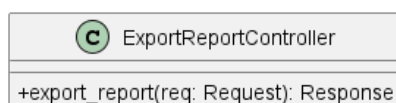


Figura 149: ExportReportController

Descrizione

ExportReportController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP per l'esportazione del report di una sessione di valutazione.

Attributi

ExportReportController non definisce attributi propri.

Metodi e funzioni

- + `export_report(req: Request): Response` — riceve la richiesta HTTP, estrae i parametri, invoca il caso d'uso per la generazione del report e delega alla classe di utilità *DownloadFileController* la costruzione della risposta HTTP configurata per il download del file.

5.8.2.2) ExportReportUseCase



Figura 150: ExportReportUseCase

Descrizione

ExportReportUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'esportazione di un report. Viene implementata da *ExportReportService* e utilizzata da *ExportReportController*.

Attributi

ExportReportUseCase non definisce attributi.

Metodi e funzioni

- + `export(command: ExportReportCommand): bytes` — firma del metodo delegato all'esecuzione della logica di esportazione a partire dai parametri forniti.

5.8.2.3) ExportReportCommand

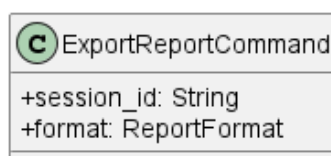


Figura 151: ExportReportCommand

Descrizione

ExportReportCommand è il Command Object che veicola i parametri necessari alla generazione del report dal controller al service.

Attributi

- + `session_id`: String — identificativo univoco della sessione di valutazione da cui generare il report.
- + `format`: ReportFormat — formato desiderato per il report in uscita (es. PDF).

Metodi e funzioni

ExportReportCommand non definisce metodi.

5.8.2.4) ExportReportService



Figura 152: ExportReportService

Descrizione

ExportReportService è il service applicativo appartenente all'Application Core responsabile dell'orchestrazione del processo di generazione ed esportazione.

Attributi

ExportReportService non definisce attributi propri.

Metodi e funzioni

- + `export(command: ExportReportCommand): bytes` — concretizza il contratto definito da *ExportReportUseCase*. Legge l'identificativo della sessione dal comando, recupera l'entità della sessione tramite *GetSessionPort* e delega la creazione materiale del documento a *ReportGeneratorPort*, restituendo l'array di byte finale.

5.8.2.5) ReportGeneratorPort

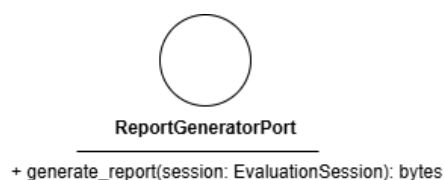


Figura 153: ReportGeneratorPort

Descrizione

ReportGeneratorPort è l'interfaccia (Outbound Port) definita nell'Application Core che stabilisce il contratto per la generazione di un report a partire dai dati di una sessione di valutazione.

Attributi

ReportGeneratorPort non definisce attributi.

Metodi e funzioni

- + `generate_report(session: EvaluationSession): bytes` — firma del metodo delegato alla generazione materiale del file del report a partire dall'entità di dominio della sessione.

5.8.2.6) PdfReportGeneratorAdapter

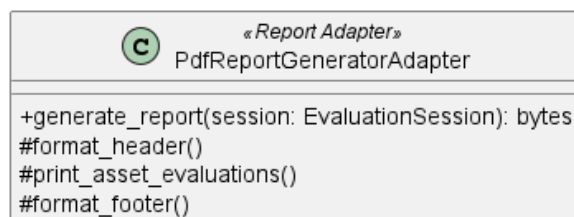


Figura 154: PdfReportGeneratorAdapter

Descrizione

PdfReportGeneratorAdapter è l'Outbound Adapter che implementa *ReportGeneratorPort*. Si occupa della formattazione e della generazione materiale del file del report in formato PDF a partire dai dati di dominio estratti dalla sessione.

Attributi

PdfReportGeneratorAdapter non definisce attributi propri.

Metodi e funzioni

- + `generate_report(session: EvaluationSession): bytes` — implementa il metodo dell'interfaccia, avviando il processo di creazione del PDF e restituendo i byte generati.
- # `format_header()` — metodo di utilità interno per la formattazione dell'intestazione grafica e testuale del documento.
- # `print_asset_evaluations()` — metodo di utilità interno per l'iterazione e l'inserimento nel documento delle valutazioni relative ai singoli asset.
- # `format_footer()` — metodo di utilità interno per la formattazione del piè di pagina del documento.

5.8.3) ExportStandard

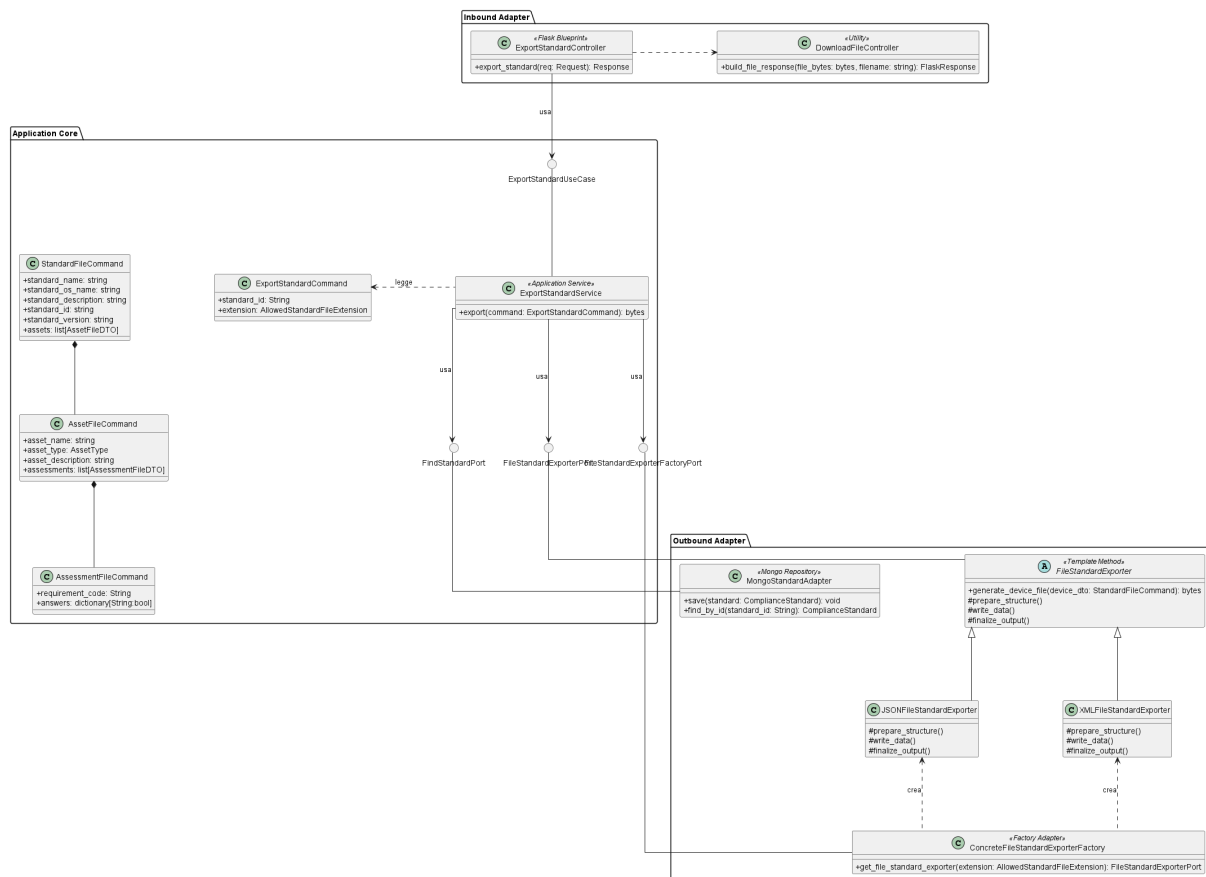


Figura 155: Caso d'uso ExportStandard

Il diagramma illustra l'architettura del modulo dedicato all'esportazione dei dati relativi a uno Standard di Conformità (inclusi i requisiti e le valutazioni associate) sotto forma di file scaricabile. Analogamente al caso d'uso di esportazione del dispositivo, il sistema supporta l'esportazione in vari formati ricorrendo ai pattern Factory e Template Method nel livello degli Outbound Adapter per garantire flessibilità ed estensibilità.

- Per la definizione di *MongoStandardAdapter*, vedere la sezione **Sezione 5.6.1.8.**
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9.**
- Per la definizione di *DownloadFileController*, vedere la sezione **Sezione 5.8.1.2**
- Per la definizione di *FindStandardPort*, vedere la sezione **Sezione 5.6.1.7**

Di seguito vengono analizzati nel dettaglio i componenti introdotti per questo caso d'uso.

5.8.3.1) ExportStandardController



Figura 156: ExportStandardController

Descrizione

ExportStandardController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di esportazione di uno standard e le inoltra al livello applicativo.

Attributi

ExportStandardController non definisce attributi propri.

Metodi e funzioni

- + `export_standard(req: Request): Response` — riceve la richiesta HTTP, estrae l'identificativo dello standard e l'estensione desiderata, invoca il caso d'uso e restituisce la risposta HTTP delegando la costruzione al controller di utilità.

5.8.3.2) ExportStandardUseCase



Figura 157: ExportStandardUseCase

Descrizione

ExportStandardUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'esportazione dei dati di uno standard di conformità. Viene implementata da *ExportStandardService*.

Attributi

ExportStandardUseCase non definisce attributi.

Metodi e funzioni

- + `export(command: ExportStandardCommand): bytes` — firma del metodo delegato all'esecuzione della logica di esportazione a partire dai parametri forniti.

5.8.3.3) ExportStandardCommand

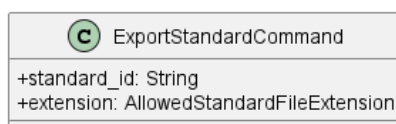


Figura 158: ExportStandardCommand

Descrizione

ExportStandardCommand è il Command Object che veicola i parametri necessari all'esportazione dello standard dal controller al service.

Attributi

- + standard_id: String — identificativo univoco dello standard da esportare.
- + extension: AllowedStandardFileExtension — formato richiesto per l'esportazione.

Metodi e funzioni

ExportStandardCommand non definisce metodi.

5.8.3.4) ExportStandardService



Figura 159: ExportStandardService

Descrizione

ExportStandardService è il service applicativo appartenente all'Application Core responsabile dell'orchestrazione del processo di esportazione dello standard.

Attributi

ExportStandardService non definisce attributi propri.

Metodi e funzioni

- + export(command: ExportStandardCommand): bytes — concretizza il contratto definito da *ExportStandardUseCase*. Recupera i dati dello standard, costruisce la struttura dei DTO, richiede l'esportatore corretto e avvia la generazione del file.

5.8.3.5) StandardFileCommand

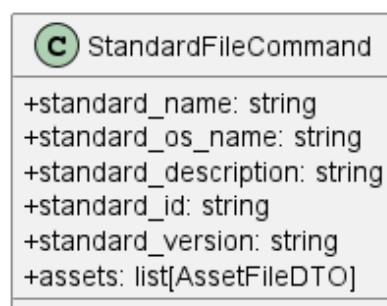


Figura 160: StandardFileCommand

Descrizione

StandardFileCommand è il Data Transfer Object (DTO) radice utilizzato per disaccoppiare i dati di dominio dello standard di conformità passandoli all'Outbound Adapter per l'esportazione.

Attributi

- + `standard_id`: string — identificativo dello standard.
- + `standard_name`: string — nome dello standard.
- + `standard_version`: string — versione dello standard.
- + `standard_os_name`: string — sistema operativo di riferimento (se applicabile).
- + `standard_description`: string — descrizione generale.
- + `assets`: list[AssetFileDTO] — lista degli asset e delle rispettive valutazioni associate allo standard.

Metodi e funzioni

StandardFileCommand non definisce metodi.

5.8.3.6) AssetFileCommand

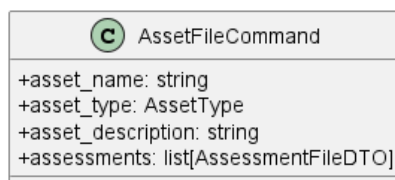


Figura 161: AssetFileCommand

Descrizione

AssetFileCommand è il DTO che rappresenta le informazioni di un singolo asset all'interno del report di esportazione dello standard.

Attributi

- + `asset_name`: string — nome dell'asset.
- + `asset_type`: AssetType — tipologia dell'asset.
- + `asset_description`: string — descrizione dell'asset.
- + `assessments`: list[AssessmentFileDTO] — lista delle valutazioni effettuate sull'asset.

Metodi e funzioni

AssetFileCommand non definisce metodi.

5.8.3.7) AssessmentFileCommand

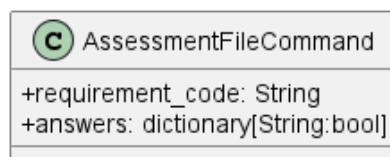


Figura 162: AssessmentFileCommand

Descrizione

AssessmentFileCommand è il DTO foglia che incapsula l'esito della valutazione di uno specifico requisito di conformità.

Attributi

- + `requirement_code: String` — codice del requisito valutato.
- + `answers: dictionary[String:bool]` — dizionario contenente le risposte fornite durante la valutazione.

Metodi e funzioni

AssessmentFileCommand non definisce metodi.

5.8.3.8) FileStandardExporterFactoryPort



Figura 163: FileStandardExporterFactoryPort

Descrizione

FileStandardExporterFactoryPort è l'interfaccia (Outbound Port) che stabilisce il contratto per l'istanziatura dinamica dell'esportatore di standard corretto.

Attributi

FileStandardExporterFactoryPort non definisce attributi.

Metodi e funzioni

- + `get_file_standard_exporter(extension: AllowedStandardFileExtension): FileStandardExporterPort` — firma del metodo che restituisce l'istanza corretta dell'esportatore in base al formato richiesto.

5.8.3.9) FileStandardExporterPort



Figura 164: FileStandardExporterPort

Descrizione

FileStandardExporterPort è l'interfaccia (Outbound Port) definita nell'Application Core che stabilisce il contratto per la generazione materiale del file dello standard.

Attributi

FileStandardExporterPort non definisce attributi.

Metodi e funzioni

- + generate_device_file(device_dto: StandardFileCommand): bytes — firma del metodo delegato all'esportazione vera e propria dei dati partendo dal DTO fornito.

5.8.3.10) MongoStandardAdapter



Figura 165: MongoStandardAdapter

Descrizione

MongoStandardAdapter è l'Outbound Adapter che implementa le interfacce di persistenza per gli standard di conformità (inclusa *FindStandardPort*), interfacciandosi con un database MongoDB.

Attributi

MongoStandardAdapter non definisce attributi.

Metodi e funzioni

- + save(standard: ComplianceStandard): void — persiste uno standard di conformità nel database.
- + find_by_id(standard_id: String): ComplianceStandard — implementa l'interfaccia per il recupero di uno standard in base all'ID.

5.8.3.11) ConcreteFileStandardExporterFactory



Figura 166: ConcreteFileStandardExporterFactory

Descrizione

ConcreteFileStandardExporterFactory è l'Outbound Adapter che implementa il pattern Factory per istanziare la classe di esportazione corretta.

Attributi

ConcreteFileStandardExporterFactory non definisce attributi.

Metodi e funzioni

- + `get_file_standard_exporter(extension: AllowedStandardFileExtension): FileStandardExporterPort` — restituisce un'istanza di *JSONFileStandardExporter* o *XMLFileStandardExporter* in base all'estensione fornita.

5.8.3.12) JSONFileStandardExporter e XMLFileStandardExporter



Figura 167: JSON e XML FileStandardExporter

Descrizione

JSONFileStandardExporter e *XMLFileStandardExporter* sono le classi concrete che estendono *FileStandardExporter* fornendo l'implementazione specifica per i formati JSON e XML dei passaggi previsti dal Template Method.

Attributi

Non definiscono attributi propri.

Metodi e funzioni

- # `prepare_structure()` — implementazione concreta del passaggio di inizializzazione della struttura del file.
- # `write_data()` — implementazione concreta della mappatura e scrittura dei dati.
- # `finalize_output()` — implementazione concreta della chiusura e serializzazione del documento in byte.

5.9) Architettura Frontend: Modulo Decision Tree

Il frontend per la valutazione dei requisiti è progettato per essere reattivo e disaccoppiato. Per evitare di sovraccaricare il server a ogni interazione, una porzione della logica di dominio viene eseguita direttamente lato client, orchestrata da uno State Management centralizzato.

L'architettura si articola su quattro pilastri:

1. **Core di Valutazione (EvaluationEngine):** Il client non si limita a mostrare dati passivi, ma ricalcola istantaneamente il percorso attivo dell'albero in base alle risposte dell'utente, offrendo un feedback visivo immediato tramite una struttura a oggetti polimorfica.
2. **Gestione dello Stato (DecisionTreeStore):** Basato su Pinia, funge da «Single Source of Truth». Mantiene in memoria le risposte, lo stato dell'interfaccia, il nodo selezionato e delega i calcoli complessi all'EvaluationEngine.
3. **Interfaccia Utente (UI):** Suddivisa in due macro-aree cooperanti. Il **Tree Canvas** disegna la topologia dell'albero delegando i calcoli geometrici al D3LayoutEngine, mentre la **Tree Sidebar** funge da pannello interattivo per leggere le domande e inserire le risposte, comunicando esclusivamente con lo Store.
4. **Integrazione Backend (EvaluationApiClient):** Astrattizza la comunicazione HTTP con il server, assicurando che lo stato locale dell'applicazione sia costantemente sincronizzato con il database remoto in modo asincrono.

Nei paragrafi successivi verranno analizzati nel dettaglio i diagrammi delle classi dei singoli sottosistemi.

5.9.1) Architettura Interattiva della Sidebar

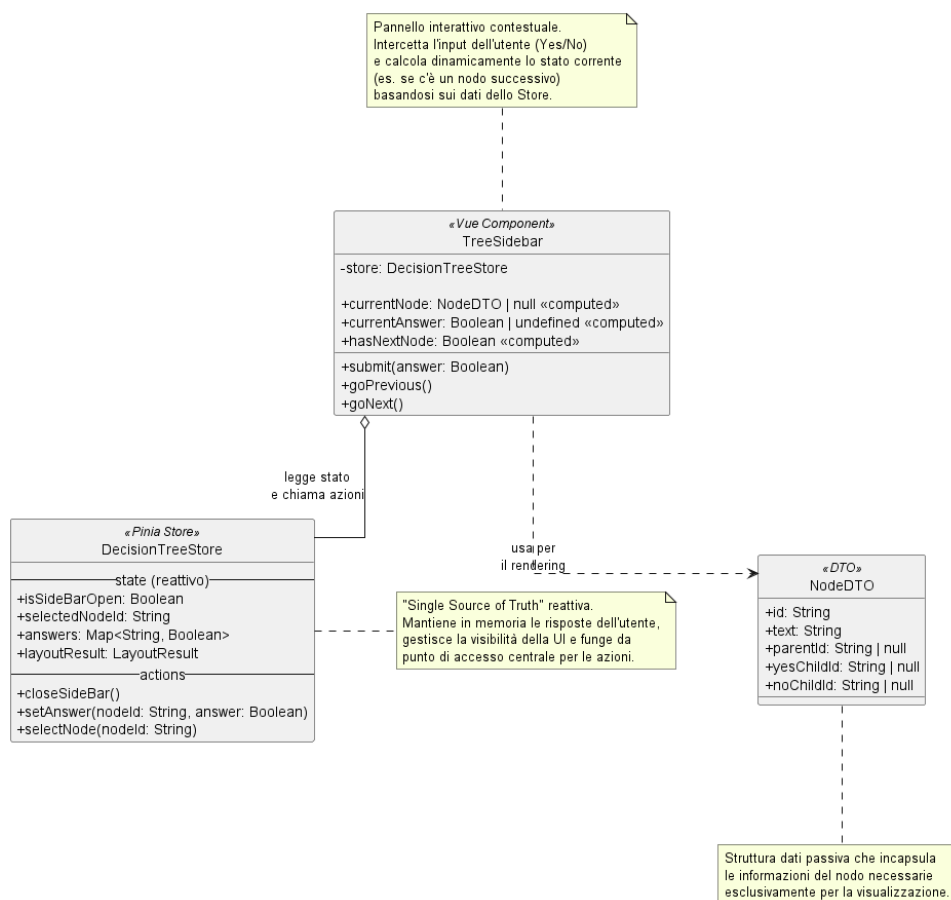


Figura 168: Architettura del componente TreeSidebar e del suo Store

Descrizione

Il diagramma illustra la relazione e la separazione delle responsabilità tra l'interfaccia utente laterale (*TreeSidebar*), il gestore dello stato globale (*DecisionTreeStore*) e l'oggetto di trasferimento dati (*NodeDTO*). Il componente visivo si interfaccia esclusivamente con lo Store per leggere lo stato reattivo e invocare le azioni di aggiornamento, utilizzando il DTO in modo puramente passivo per estrarre le informazioni testuali e topologiche necessarie al rendering a schermo.

Di seguito il dettaglio dei singoli componenti coinvolti nel flusso.

5.9.1.1) DecisionTreeStore

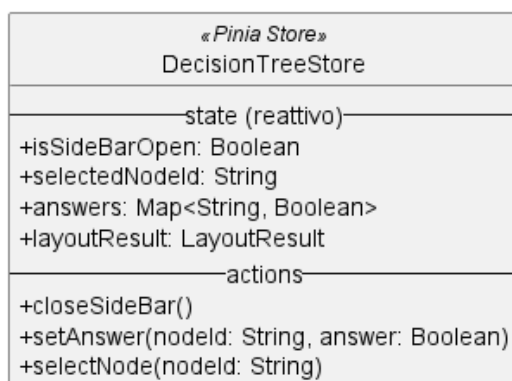


Figura 169: DecisionTreeStore

Descrizione

DecisionTreeStore è il modulo basato su Pinia che funge da «Single Source of Truth» (singola fonte di verità) per il frontend. Mantiene in memoria in modo reattivo le risposte dell'utente, gestisce la visibilità della UI e accentra il punto di accesso per tutte le modifiche di stato.

Stato Reattivo (State)

- + `isSideBarOpen: Boolean` — flag che determina se il pannello laterale è attualmente aperto o nascosto.
- + `selectedNodeId: String` — l'identificativo del nodo correntemente selezionato o cliccato dall'utente.
- + `answers: Map<String, Boolean>` — dizionario che associa l'ID di ogni nodo decisionale alla rispettiva risposta (Yes/No) fornita dall'utente.
- + `layoutResult: LayoutResult` — struttura dati che incapsula i risultati del calcolo geometrico (nodi e archi) necessari a renderizzare l'albero.

Metodi e funzioni (Actions)

- + `closeSideBar()` — azione per chiudere il pannello laterale.
- + `setAnswer(nodeId: String, answer: Boolean)` — registra o aggiorna la risposta dell'utente per un dato nodo decisionale.
- + `selectNode(nodeId: String)` — azione per impostare un nodo come «selezionato», scatenando gli aggiornamenti reattivi sulla UI.

5.9.1.2) TreeSidebar

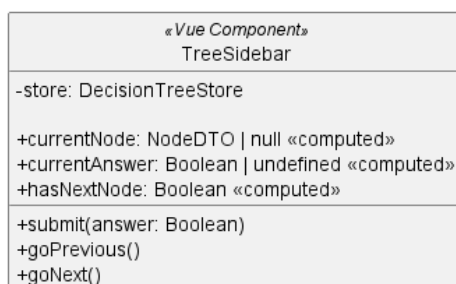


Figura 170: TreeSidebar

Descrizione

TreeSidebar è il componente Vue.js che implementa il pannello laterale interattivo. Il suo compito è intercettare l'input dell'utente e presentare le informazioni del nodo selezionato calcolando dinamicamente la navigazione basandosi sullo Store centrale.

Proprietà Compute (Computed)

- + `currentNode: NodeDTO | null` — recupera dinamicamente le informazioni del nodo selezionato in quel momento.
- + `currentAnswer: Boolean | undefined` — estrae dallo Store la risposta precedentemente data (se presente) per il nodo corrente.
- + `hasNextNode: Boolean` — calcola dinamicamente se esiste un nodo successivo navigabile partendo dalla situazione corrente.

Metodi e funzioni

- + `submit(answer: Boolean)` — gestisce l'invio della risposta per la domanda a schermo, delegandone il salvataggio all'azione dello Store.
- + `goPrevious()` — innesca la navigazione per tornare al nodo precedente lungo il percorso di valutazione.
- + `goNext()` — innesca la navigazione per avanzare al nodo logico successivo all'interno del flusso.

5.9.1.3) NodeDTO

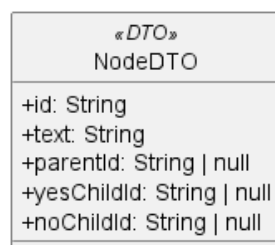


Figura 171: NodeDTO

Descrizione

NodeDTO (Data Transfer Object) è una struttura dati passiva utilizzata esclusivamente per il trasferimento e la formattazione dei dati. Incapsula le informazioni «grezze» del nodo in modo che il componente visivo possa stamparle a schermo senza dover accedere alla complessa logica di dominio.

Attributi

- + id: String — identificativo univoco del nodo.
- + text: String — il contenuto testuale della domanda o dell’esito.
- + parentId: String | null — riferimento all’ID del nodo padre, necessario per calcolare la navigazione a ritroso.
- + yesChildId: String | null — riferimento all’ID del nodo figlio in caso di risposta affermativa.
- + noChildId: String | null — riferimento all’ID del nodo figlio in caso di risposta negativa.

Metodi e funzioni

NodeDTO non espone metodi, trattandosi di un oggetto dedicato al solo trasporto dati.

5.9.2) Architettura Visiva del Canvas (Tree Canvas)

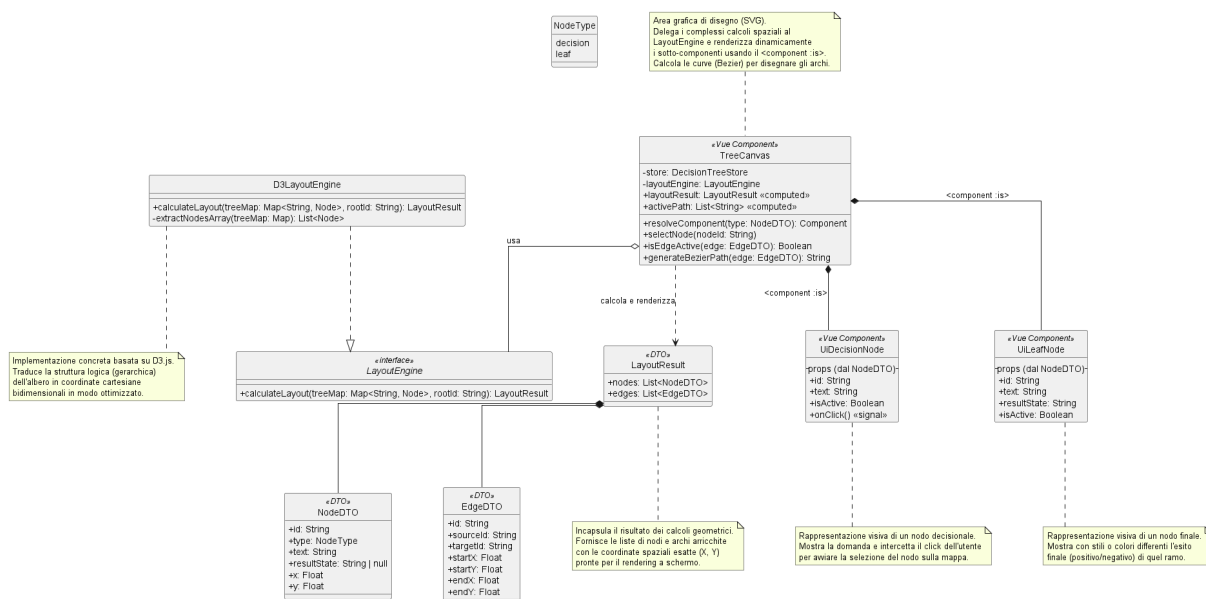


Figura 172: Architettura del componente TreeCanvas e del Layout Engine

Descrizione

Il diagramma illustra l'architettura dedicata alla renderizzazione topologica dell'albero decisionale. Per garantire prestazioni ottimali e codice pulito, il sistema separa nettamente il calcolo matematico delle coordinate visive (delegato a *D3LayoutEngine*) dal rendering effettivo a schermo (gestito dal componente *TreeCanvas* e dai suoi sotto-componenti *UiDecisionNode* e *UiLeafNode*).

Di seguito vengono analizzati nel dettaglio i componenti e le strutture dati che partecipano a questo flusso.

5.9.2.1) LayoutResult e i DTO Geometrici

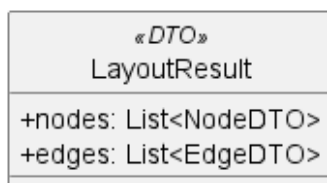


Figura 173: LayoutResult, NodeDTO ed EdgeDTO

Descrizione

Questi Data Transfer Object (DTO) rappresentano le strutture dati arricchite con le informazioni spaziali. A differenza del NodeDTO usato dalla Sidebar, qui i nodi includono coordinate assolute per il posizionamento sulla mappa.

Attributi

- **NodeDTO:**
 - + id: String — identificativo del nodo.
 - + type: NodeType — enumerativo che distingue tra nodo decisionale (decision) e nodo finale (leaf).
 - + text: String — testo da mostrare.
 - + resultState: String | null — eventuale esito finale (se di tipo leaf).
 - + x: Float, + y: Float — coordinate cartesiane calcolate per il centro del nodo.
- **EdgeDTO:**
 - + id: String — identificativo dell'arco.
 - + sourceId: String, + targetId: String — ID dei nodi collegati.
 - + startX, startY, endX, endY: Float — coordinate esatte dei punti di inizio e fine della linea di collegamento.
- **LayoutResult:**
 - + nodes: List<NodeDTO> — lista di tutti i nodi posizionati.
 - + edges: List<EdgeDTO> — lista di tutti gli archi posizionati.

5.9.2.2) LayoutEngine e D3LayoutEngine

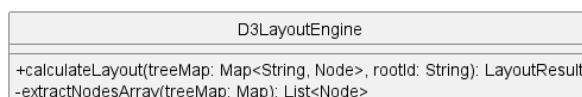


Figura 174: D3LayoutEngine

Descrizione

LayoutEngine è l'interfaccia che definisce il contratto per il calcolo spaziale. *D3LayoutEngine* ne è l'implementazione concreta, che sfrutta la libreria matematica

D3.js per trasformare la struttura logica e gerarchica dell'albero in coordinate cartesiane bidimensionali, evitando sovrapposizioni tra i nodi.

Metodi e funzioni

- + `calculateLayout(treeMap: Map<String, Node>, rootId: String): LayoutResult` — riceve la mappa logica dei nodi e restituisce il `LayoutResult` contenente nodi e archi con le coordinate X e Y calcolate.
- - `extractNodesArray(treeMap: Map): List<Node>` — metodo di utilità interno per la conversione della mappa in array.

5.9.2.3) TreeCanvas

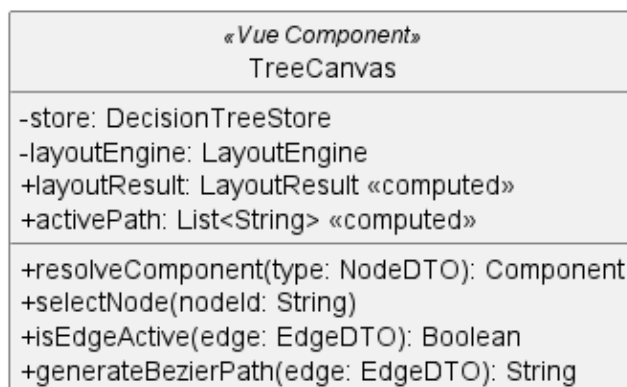


Figura 175: TreeCanvas

Descrizione

TreeCanvas è il componente Vue.js principale per la visualizzazione dell'albero. Interroga lo Store, passa i dati al Layout Engine per il calcolo geometrico e si occupa di orchestrare il rendering dinamico di linee e nodi a schermo.

Proprietà Compute e Stato

- - `store: DecisionTreeStore` — riferimento allo stato globale.
- - `layoutEngine: LayoutEngine` — riferimento al motore matematico.
- + `layoutResult: LayoutResult` — (computed) si aggiorna automaticamente se l'albero cambia, innescando un nuovo ricalcolo geometrico.
- + `activePath: List<String>` — (computed) recupera l'elenco degli ID dei nodi correntemente attivi in base alle risposte dell'utente.

Metodi e funzioni

- + `resolveComponent(type: NodeDTO): Component` — determina dinamicamente quale sotto-componente Vue renderizzare (*UiDecisionNode* o *UiLeafNode*) in base al tipo del nodo.
- + `selectNode(nodeId: String)` — gestisce il click su un nodo della mappa, notificando lo Store.

- + `isEdgeActive(edge: EdgeDTO): Boolean` — verifica se la linea di collegamento appartiene al percorso attivo per applicare stili di evidenziazione.
- + `generateBezierPath(edge: EdgeDTO): String` — calcola la stringa del tracciato SVG per disegnare una curva morbida tra due nodi.

5.9.2.4) UiDecisionNode e UiLeafNode

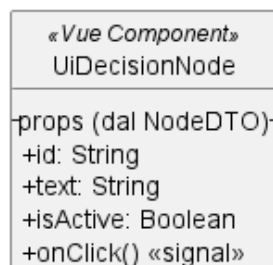


Figura 176: UiDecisionNode e UiLeafNode

Descrizione

UiDecisionNode e *UiLeafNode* sono i micro-componenti visivi montati dinamicamente dal *TreeCanvas*. Si occupano unicamente di stampare a schermo l'interfaccia di un singolo nodo, applicando stili CSS differenti a seconda che il nodo sia attivo o meno, e intercettando le interazioni dell'utente.

Proprietà (Props passate dal padre)

- + `id: String` — identificativo del nodo.
- + `text: String` — testo da mostrare (domanda o esito).
- + `isActive: Boolean` — flag iniettato dal *TreeCanvas* per indicare se il nodo è parte del percorso selezionato.
- + `resultState: String` — (Solo in *UiLeafNode*) indica l'esito finale della valutazione per colorare opportunamente il nodo (es. verde/rosso).

Eventi (Signals)

- + `onClick()` — (Solo in *UiDecisionNode*) emesso quando l'utente fa click sulla forma geometrica del nodo, catturato dal *TreeCanvas*.

5.9.3) Core di Valutazione (Domain Logic Client-side)

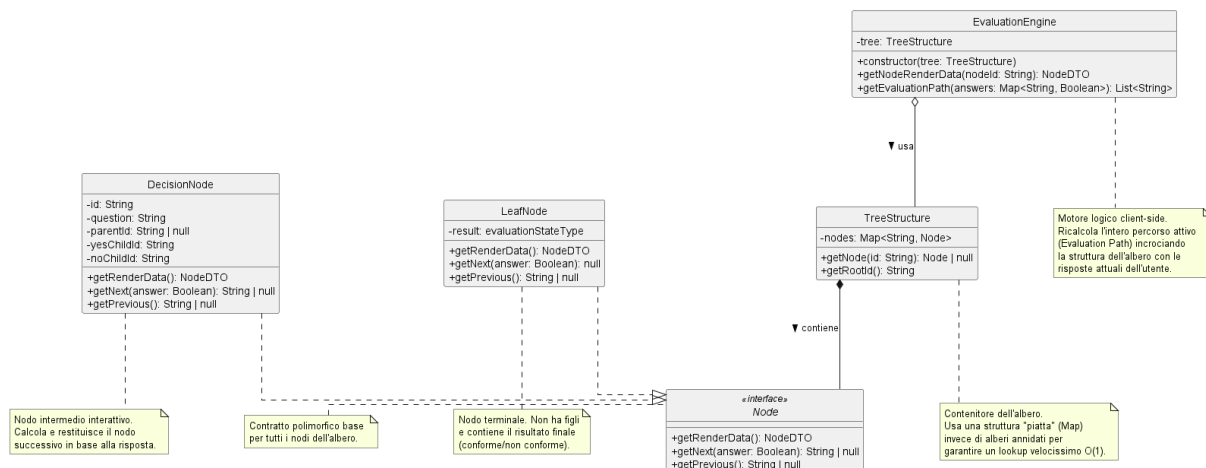


Figura 177: Diagramma delle classi del Motore di Valutazione

Descrizione

Il «Cervello» del modulo frontend risiede in una gerarchia di classi che implementano la logica di navigazione dell'albero decisionale direttamente nel browser. Questo approccio permette di ricalcolare il percorso di valutazione istantaneamente a ogni clic dell'utente, senza richiedere l'intervento del server. Il sistema utilizza il polimorfismo per distinguere il comportamento tra nodi di domanda e nodi di esito.

5.9.3.1) Interfaccia Node

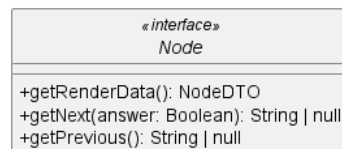


Figura 178: Interfaccia Node

Descrizione

Node è l'interfaccia (o contratto) base che definisce il comportamento comune a tutti i tipi di nodi presenti nell'albero. Garantisce che ogni nodo, indipendentemente dalla sua natura, possa fornire i propri dati di rendering e gestire la navigazione.

Metodi e funzioni

- + `getRenderData(): NodeDTO` — restituisce i dati necessari al componente visivo per mostrare il nodo.
- + `getNext(answer: Boolean): String | null` — calcola l'ID del nodo successivo in base alla risposta ricevuta.
- + `getPrevious(): String | null` — restituisce l'ID del nodo genitore per permettere la navigazione a ritroso.

5.9.3.2) DecisionNode

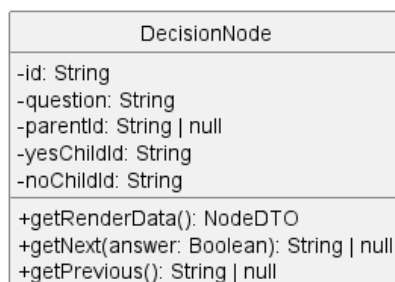


Figura 179: DecisionNode

Descrizione

DecisionNode è la classe che implementa un nodo intermedio (domanda). Contiene la logica decisionale binaria (Sì/No) per determinare quale ramo dell'albero debba essere percorso.

Attributi

- - id, - question: String — identificativo e testo della domanda.
- - parentId: String | null — riferimento al nodo precedente.
- - yesChildId, - noChildId: String — riferimenti ai due possibili nodi successivi.

Metodi e funzioni

- + getNext(answer: Boolean) — implementazione concreta che restituisce yesChildId se la risposta è true, altrimenti noChildId.

5.9.3.3) LeafNode

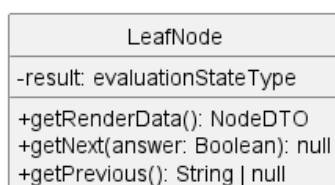


Figura 180: LeafNode

Descrizione

LeafNode rappresenta il punto terminale di un ramo dell'albero (foglia). Non permette ulteriori avanzamenti e contiene lo stato finale della valutazione per quel determinato percorso.

Attributi

- - result: evaluationStateType — definisce l'esito finale (es. «Conforme», «Non Conforme»).

Metodi e funzioni

- + getNext(answer: Boolean) — restituisce sempre null, segnando la fine del percorso.

5.9.3.4) TreeStructure

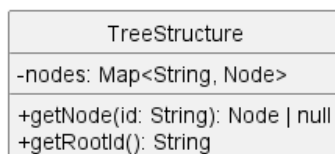


Figura 181: TreeStructure

Descrizione

TreeStructure funge da contenitore organizzato per l'intero albero decisionale. Al fine di massimizzare le prestazioni di ricerca lato client, utilizza una rappresentazione «piatta» basata su una mappa anziché una struttura nidificata.

Attributi

- - nodes: Map<String, Node> — mappa che indicizza tutti i nodi tramite il loro ID univoco per un accesso immediato ($O(1)$).

Metodi e funzioni

- + getNode(id: String): Node | null — recupera l'oggetto nodo corrispondente all'ID fornito.
- + getRootId(): String — restituisce l'ID del punto di partenza della valutazione.

5.9.3.5) EvaluationEngine

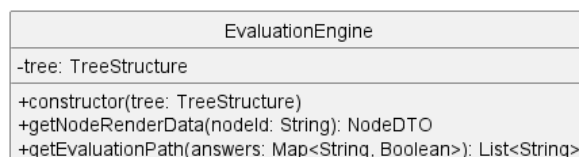


Figura 182: EvaluationEngine

Descrizione

EvaluationEngine è il motore logico principale che orchestra la valutazione. È il componente che traduce la massa di risposte dell'utente in un percorso visivo coerente e navigabile.

Attributi

- - tree: TreeStructure — la struttura dell'albero su cui operare.

Metodi e funzioni

- + `getEvaluationPath(answers: Map<String, Boolean>): List<String>` — incrocia la mappa delle risposte fornite dall'utente con la struttura dell'albero per generare la lista ordinata di ID che compongono l'attuale «Percorso Attivo» (Active Path).
- + `getNodeRenderData(nodeId: String): NodeDTO` — metodo di utilità per ottenere i dati grafici di un nodo specifico tramite il motore.

5.9.4) DecisionTreeStore

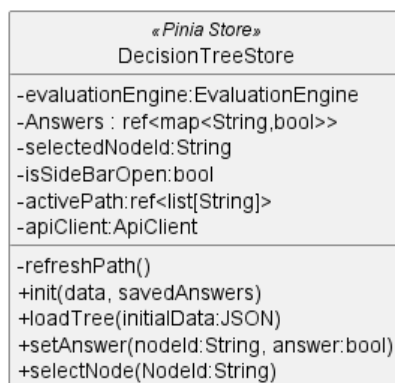


Figura 183: Dettaglio del DecisionTreeStore (Pinia)

Descrizione

DecisionTreeStore rappresenta il cuore reattivo dell'applicazione. È lo Store globale che permette la comunicazione disaccoppiata tra i vari componenti Vue (Canvas e Sidebar). Invece di passare dati tramite «props» e «events» complessi, i componenti leggono e scrivono direttamente in questo store, che si occupa di mantenere la coerenza dei dati.

Attributi (State)

- - `evaluationEngine: EvaluationEngine` — istanza del motore logico utilizzata per i calcoli di navigazione.
- - `answers: ref<Map<String, Boolean>>` — stato reattivo delle risposte fornite dall'utente durante la sessione corrente.
- - `selectedNodeId: String` — identifica il nodo su cui l'utente sta interagendo o che ha cliccato nel Canvas.
- - `activePath: ref<List<String>>` — lista reattiva degli ID dei nodi che compongono il percorso corrente, aggiornata automaticamente al variare delle risposte.
- - `apiClient: ApiClient` — riferimento al client per le chiamate verso le API del backend.

Metodi e funzioni (Actions)

- + `init(data, savedAnswers)` — inizializza lo store caricando la struttura dell'albero e le eventuali risposte già salvate nel database.
- + `setAnswer(nodeId, answer)` — aggiorna una risposta nello stato locale, invoca il ricalcolo del percorso e avvia la persistenza asincrona tramite l'API.

- + `selectNode(nodeId)` — aggiorna il nodo selezionato, permettendo alla Sidebar di mostrare le informazioni contestuali corrette.
- - `refreshPath()` — metodo privato invocato internamente per aggiornare l'attributo `activePath` interpellando l'*EvaluationEngine*.

5.9.5) EvaluationApiClient

EvaluationApiClient
-baseUrl:String
+saveAnswer(answer:bool, device_id:String, asset_id:String, requirement_id:String,node_id:String):Promise<JSON> +fetchRequirementEvaluationState(device_id:String, asset_id:String, requirement_id:String):Promise<JSON>

Figura 184: Dettaglio dell'EvaluationApiClient

Descrizione

EvaluationApiClient è l'Outbound Adapter del frontend. Questa classe incapsula tutta la logica di comunicazione HTTP, isolando il resto dell'applicazione (Store e Componenti) dai dettagli implementativi delle chiamate API. Utilizza il pattern delle *Promise* per gestire l'asincronia tipica delle richieste di rete.

Attributi

- - `baseUrl: String` — l'indirizzo radice del server API.

Metodi e funzioni

- + `saveAnswer(answer, device_id, asset_id, requirement_id, node_id): Promise<JSON>` — invia una richiesta asincrona al server per salvare la risposta fornita a un determinato nodo dell'albero. Riceve tutti i parametri di contesto necessari (dispositivo, asset e requisito) per garantire la corretta associazione dei dati nel database.
- + `fetchRequirementEvaluationState(device_id, asset_id, requirement_id): Promise<JSON>` — recupera dal backend lo stato attuale della valutazione per un determinato requisito. Questo metodo è fondamentale durante la fase di inizializzazione per ripristinare il «Percorso Attivo» basandosi sui dati salvati in precedenza.

6) Tracciamento

7) Qualità architettuale

8) Gestione Errori e Logging

9) Sicurezza

10) Performance e Scalabilità